Based on https://www.youtube.com/watch?v=UjOsM5iUWwY

Housing Prices

Strategy will be

- Import packages
- Import Data -labeled/unlabed
- Check Data
  - Target Feature Distribution
  - Check Outliers
  - Check Nan values
  - Check Correlations
- Data clean up (NO LEAKAGE)
  - Rename columns
- Split train/holdout datasets
- Data clean up (RISK OF LEAKAGE)
  - Imput missing data
  - Outliers
  - Changing Data Types
- Feature Engineering
  - Create new features
  - Deal with skew in the target variable (and maybe features depending on model)
  - Feature Transformation (scaling). Here it is not needed
  - Encoding Categorical through ordered encoding, label encoding, target encoding or one-hot encoding
  - Feature Selection
- Machine Learning
  - Hyperparameter tuning
  - Model training
  - Prediction
  - Analysis

Import Packages

In [1]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

```
import numpy as np
plt.style.use('ggplot')
pd.set_option('display.max_columns', 200)
from sklearn.model_selection import train_test_split
import scipy.stats

import lightgbm as lgb
import xgboost as xgb
from sklearn.model_selection import KFold, cross_val_score

import optuna
from xgboost import XGBRegressor
import lightgbm as lgb
from lightgbm import LGBMRegressor
from sklearn.linear_model import LinearRegression
```

Import Data

In [2]:
```
df_train = pd.read_csv("./input/train.csv",index_col = "Id")
df_inference = pd.read_csv("./input/test.csv",index_col = "Id")
```

Check Data

In [3]:
```
df_train
```

Out[3]:

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandCor
Id								
1	60	RL	65.0	8450	Pave	NaN	Reg	
2	20	RL	80.0	9600	Pave	NaN	Reg	
3	60	RL	68.0	11250	Pave	NaN	IR1	
4	70	RL	60.0	9550	Pave	NaN	IR1	
5	60	RL	84.0	14260	Pave	NaN	IR1	
...	...	...	...	...	...	...	...	
1456	60	RL	62.0	7917	Pave	NaN	Reg	
1457	20	RL	85.0	13175	Pave	NaN	Reg	
1458	70	RL	66.0	9042	Pave	NaN	Reg	
1459	20	RL	68.0	9717	Pave	NaN	Reg	
1460	20	RL	75.0	9937	Pave	NaN	Reg	

1460 rows × 80 columns

In [4]:
```
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1460 entries, 1 to 1460
Data columns (total 80 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   MSSubClass     1460 non-null   int64
 1   MSZoning       1460 non-null   object
 2   LotFrontage    1201 non-null   float64
 3   LotArea        1460 non-null   int64
 4   Street         1460 non-null   object
 5   Alley          91 non-null     object
 6   LotShape       1460 non-null   object
 7   LandContour    1460 non-null   object
 8   Utilities      1460 non-null   object
 9   LotConfig      1460 non-null   object
 10  LandSlope      1460 non-null   object
 11  Neighborhood   1460 non-null   object
 12  Condition1     1460 non-null   object
 13  Condition2     1460 non-null   object
 14  BldgType       1460 non-null   object
 15  HouseStyle     1460 non-null   object
 16  OverallQual    1460 non-null   int64
 17  OverallCond    1460 non-null   int64
 18  YearBuilt      1460 non-null   int64
 19  YearRemodAdd   1460 non-null   int64
 20  RoofStyle      1460 non-null   object
 21  RoofMatl       1460 non-null   object
 22  Exterior1st    1460 non-null   object
 23  Exterior2nd    1460 non-null   object
 24  MasVnrType     588 non-null    object
 25  MasVnrArea     1452 non-null   float64
 26  ExterQual      1460 non-null   object
 27  ExterCond      1460 non-null   object
 28  Foundation     1460 non-null   object
 29  BsmtQual       1423 non-null   object
 30  BsmtCond       1423 non-null   object
 31  BsmtExposure   1422 non-null   object
 32  BsmtFinType1   1423 non-null   object
 33  BsmtFinSF1     1460 non-null   int64
 34  BsmtFinType2   1422 non-null   object
 35  BsmtFinSF2     1460 non-null   int64
 36  BsmtUnfSF      1460 non-null   int64
 37  TotalBsmtSF    1460 non-null   int64
 38  Heating        1460 non-null   object
 39  HeatingQC      1460 non-null   object
 40  CentralAir     1460 non-null   object
 41  Electrical     1459 non-null   object
 42  1stFlrSF       1460 non-null   int64
 43  2ndFlrSF       1460 non-null   int64
 44  LowQualFinSF   1460 non-null   int64
 45  GrLivArea      1460 non-null   int64
 46  BsmtFullBath   1460 non-null   int64
 47  BsmtHalfBath   1460 non-null   int64
 48  FullBath       1460 non-null   int64
 49  HalfBath       1460 non-null   int64
 50  BedroomAbvGr   1460 non-null   int64
 51  KitchenAbvGr   1460 non-null   int64
 52  KitchenQual    1460 non-null   object
 53  TotRmsAbvGrd   1460 non-null   int64
 54  Functional     1460 non-null   object
```

```
 55  Fireplaces     1460 non-null   int64
 56  FireplaceQu    770 non-null    object
 57  GarageType     1379 non-null   object
 58  GarageYrBlt    1379 non-null   float64
 59  GarageFinish   1379 non-null   object
 60  GarageCars     1460 non-null   int64
 61  GarageArea     1460 non-null   int64
 62  GarageQual     1379 non-null   object
 63  GarageCond     1379 non-null   object
 64  PavedDrive     1460 non-null   object
 65  WoodDeckSF     1460 non-null   int64
 66  OpenPorchSF    1460 non-null   int64
 67  EnclosedPorch  1460 non-null   int64
 68  3SsnPorch      1460 non-null   int64
 69  ScreenPorch    1460 non-null   int64
 70  PoolArea       1460 non-null   int64
 71  PoolQC         7 non-null      object
 72  Fence          281 non-null    object
 73  MiscFeature    54 non-null     object
 74  MiscVal        1460 non-null   int64
 75  MoSold         1460 non-null   int64
 76  YrSold         1460 non-null   int64
 77  SaleType       1460 non-null   object
 78  SaleCondition  1460 non-null   object
 79  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(34), object(43)
memory usage: 923.9+ KB
```

In [5]:
```
df_inference.info()
```

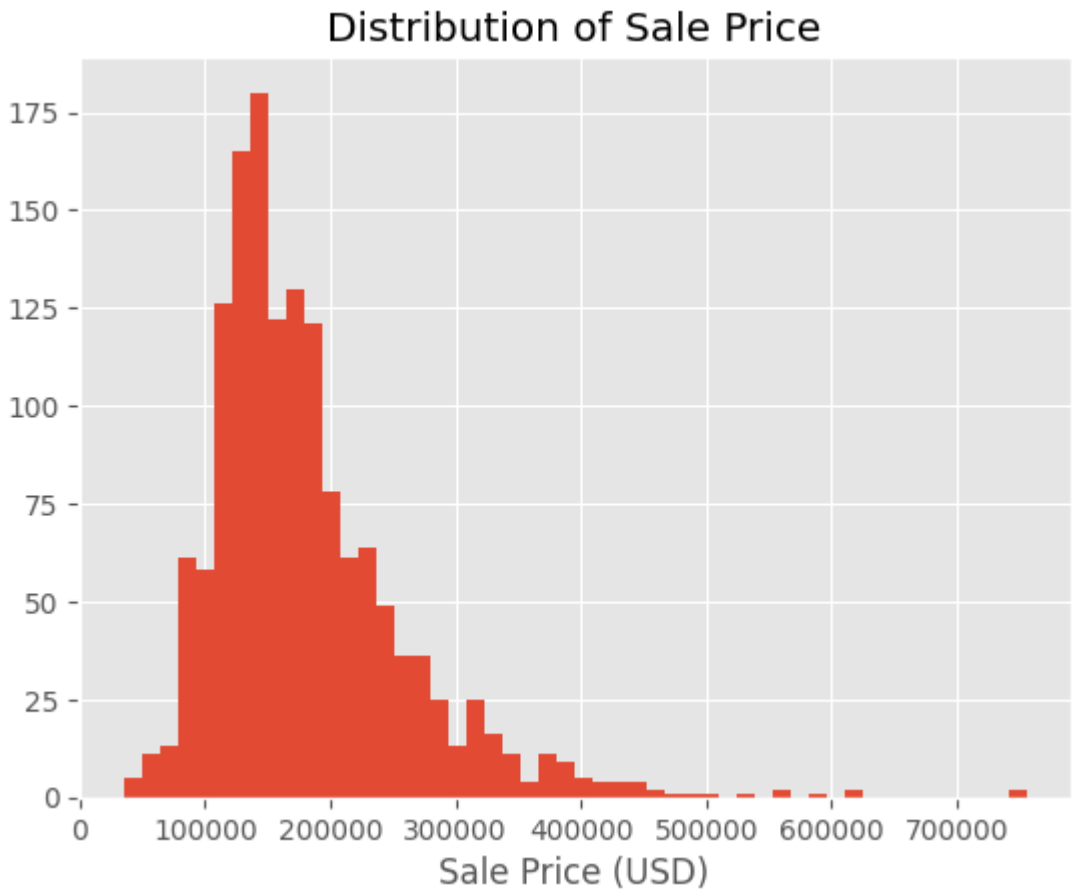
```
<class 'pandas.core.frame.DataFrame'>
Index: 1459 entries, 1461 to 2919
Data columns (total 79 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   MSSubClass     1459 non-null   int64
 1   MSZoning       1455 non-null   object
 2   LotFrontage    1232 non-null   float64
 3   LotArea        1459 non-null   int64
 4   Street         1459 non-null   object
 5   Alley          107 non-null    object
 6   LotShape       1459 non-null   object
 7   LandContour    1459 non-null   object
 8   Utilities      1457 non-null   object
 9   LotConfig      1459 non-null   object
 10  LandSlope      1459 non-null   object
 11  Neighborhood   1459 non-null   object
 12  Condition1     1459 non-null   object
 13  Condition2     1459 non-null   object
 14  BldgType       1459 non-null   object
 15  HouseStyle     1459 non-null   object
 16  OverallQual    1459 non-null   int64
 17  OverallCond    1459 non-null   int64
 18  YearBuilt      1459 non-null   int64
 19  YearRemodAdd   1459 non-null   int64
 20  RoofStyle      1459 non-null   object
 21  RoofMatl       1459 non-null   object
 22  Exterior1st    1458 non-null   object
 23  Exterior2nd    1458 non-null   object
 24  MasVnrType     565 non-null    object
 25  MasVnrArea     1444 non-null   float64
 26  ExterQual      1459 non-null   object
 27  ExterCond      1459 non-null   object
 28  Foundation     1459 non-null   object
 29  BsmtQual       1415 non-null   object
 30  BsmtCond       1414 non-null   object
 31  BsmtExposure   1415 non-null   object
 32  BsmtFinType1   1417 non-null   object
 33  BsmtFinSF1     1458 non-null   float64
 34  BsmtFinType2   1417 non-null   object
 35  BsmtFinSF2     1458 non-null   float64
 36  BsmtUnfSF      1458 non-null   float64
 37  TotalBsmtSF    1458 non-null   float64
 38  Heating        1459 non-null   object
 39  HeatingQC      1459 non-null   object
 40  CentralAir     1459 non-null   object
 41  Electrical     1459 non-null   object
 42  1stFlrSF       1459 non-null   int64
 43  2ndFlrSF       1459 non-null   int64
 44  LowQualFinSF   1459 non-null   int64
 45  GrLivArea      1459 non-null   int64
 46  BsmtFullBath   1457 non-null   float64
 47  BsmtHalfBath   1457 non-null   float64
 48  FullBath       1459 non-null   int64
 49  HalfBath       1459 non-null   int64
 50  BedroomAbvGr   1459 non-null   int64
 51  KitchenAbvGr   1459 non-null   int64
 52  KitchenQual    1458 non-null   object
 53  TotRmsAbvGrd   1459 non-null   int64
 54  Functional     1457 non-null   object
```

```
 55  Fireplaces     1459 non-null   int64
 56  FireplaceQu    729 non-null    object
 57  GarageType     1383 non-null   object
 58  GarageYrBlt    1381 non-null   float64
 59  GarageFinish   1381 non-null   object
 60  GarageCars     1458 non-null   float64
 61  GarageArea     1458 non-null   float64
 62  GarageQual     1381 non-null   object
 63  GarageCond     1381 non-null   object
 64  PavedDrive     1459 non-null   object
 65  WoodDeckSF     1459 non-null   int64
 66  OpenPorchSF    1459 non-null   int64
 67  EnclosedPorch  1459 non-null   int64
 68  3SsnPorch      1459 non-null   int64
 69  ScreenPorch    1459 non-null   int64
 70  PoolArea       1459 non-null   int64
 71  PoolQC         3 non-null      object
 72  Fence          290 non-null    object
 73  MiscFeature    51 non-null     object
 74  MiscVal        1459 non-null   int64
 75  MoSold         1459 non-null   int64
 76  YrSold         1459 non-null   int64
 77  SaleType       1458 non-null   object
 78  SaleCondition  1459 non-null   object
dtypes: float64(11), int64(25), object(43)
memory usage: 911.9+ KB
```

Target distribution

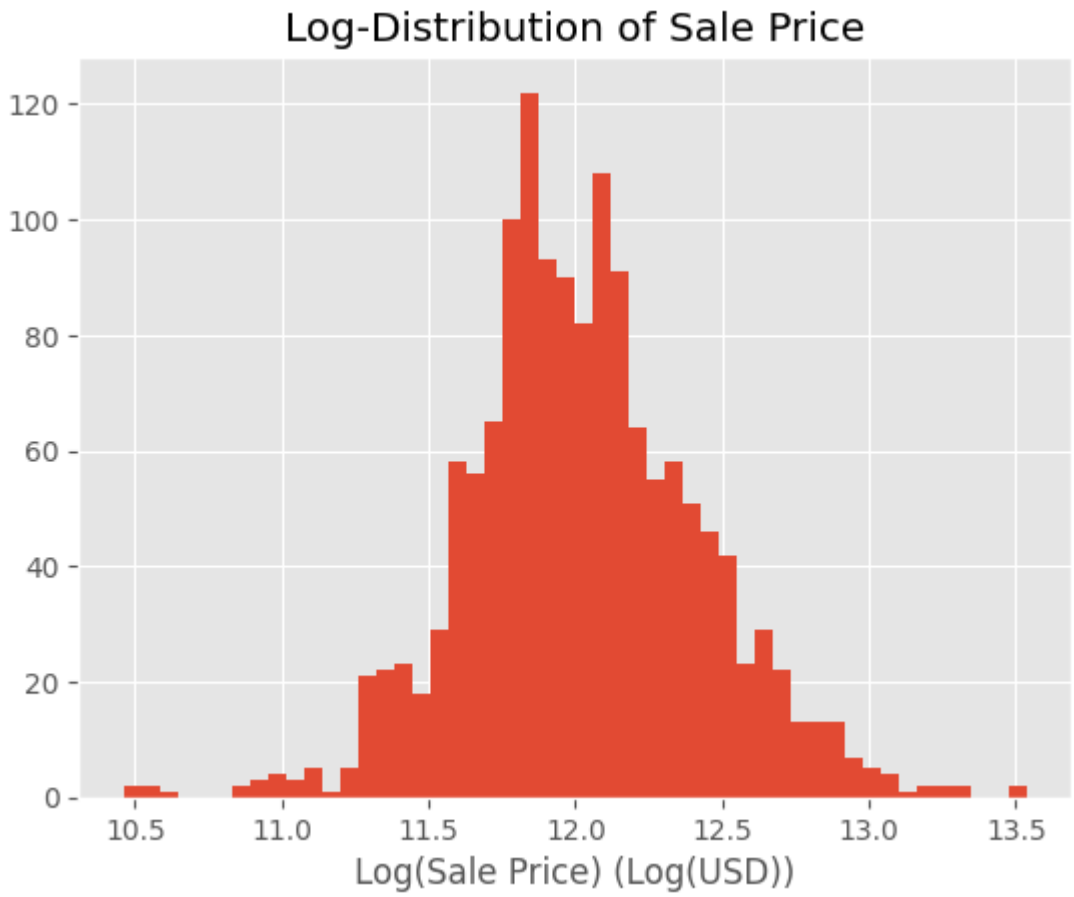
In [6]:
```
ax = df_train["SalePrice"].hist(bins = 50)
ax.set_title("Distribution of Sale Price")
ax.set_xlabel("Sale Price (USD)")
```

Out[6]:
```
Text(0.5, 0, 'Sale Price (USD)')
```



In [7]:
```
ax = df_train["SalePrice"].apply(lambda x : np.log(x)).hist(bins = 50)
ax.set_title("Log-Distribution of Sale Price")
ax.set_xlabel("Log(Sale Price) (Log(USD))")
```

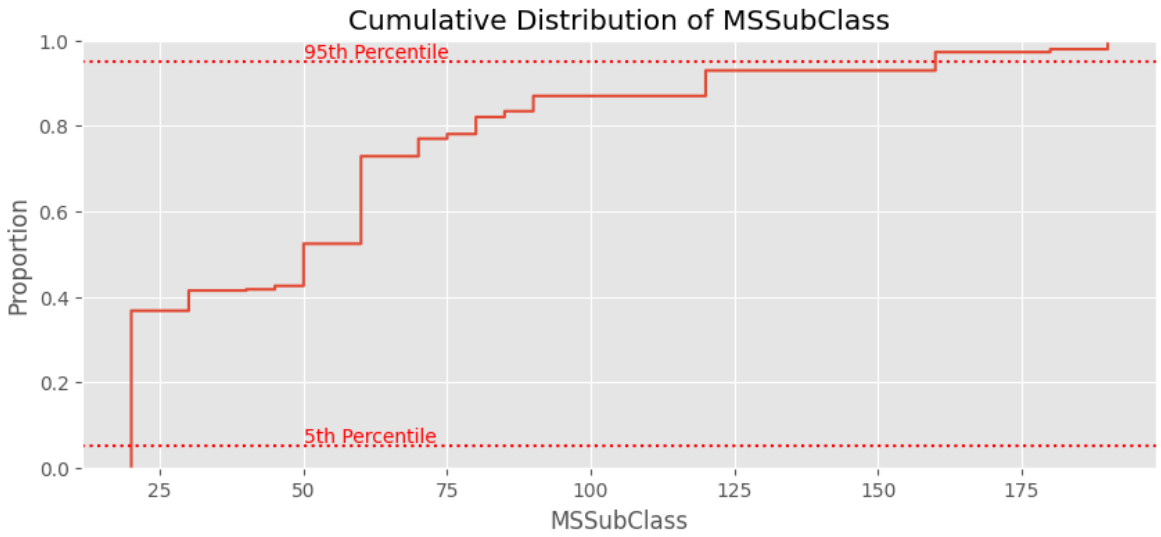
Out[7]:
```
Text(0.5, 0, 'Log(Sale Price) (Log(USD))')
```

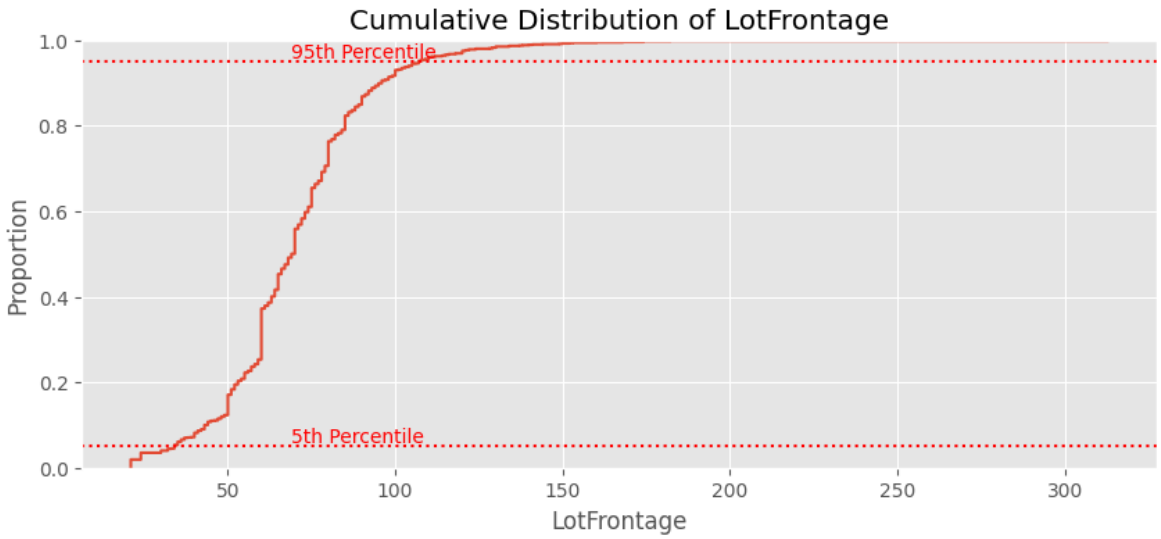


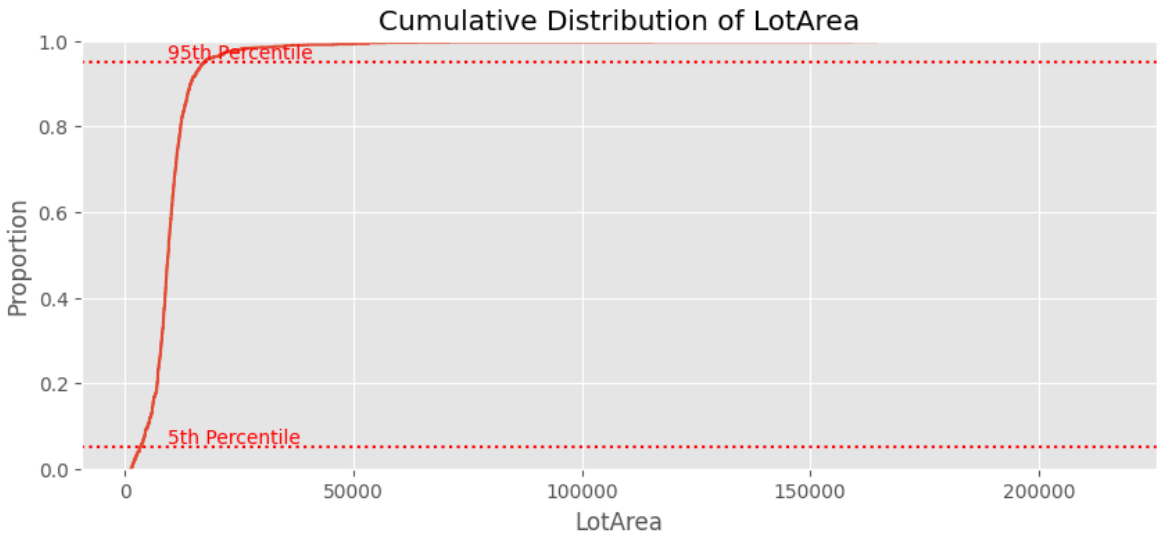
Check outliers

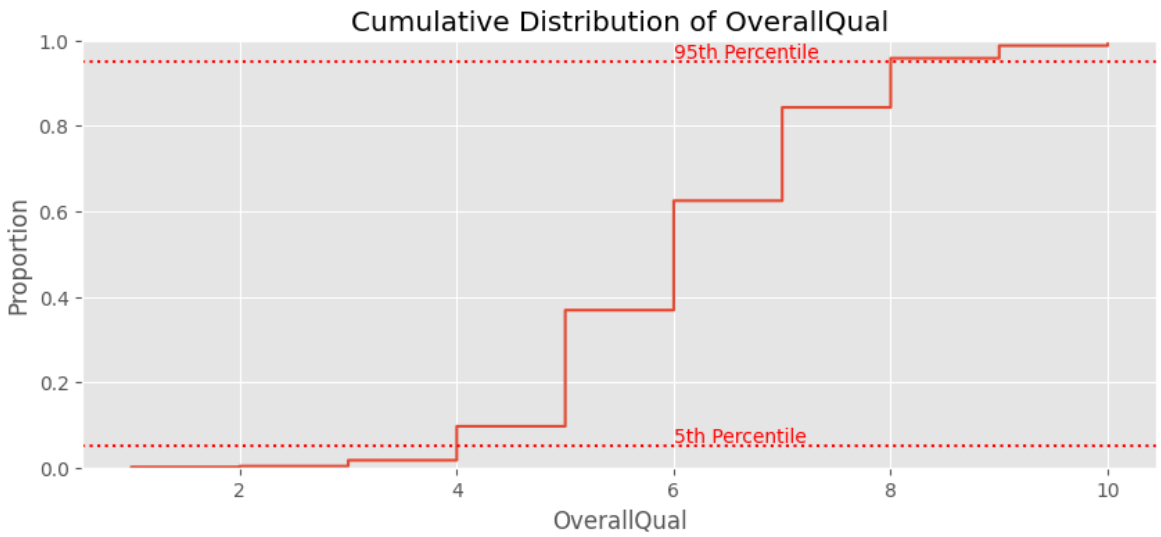
In [8]:
```
numerical_df = df_train.select_dtypes(exclude=['object'])
# numerical_df = numerical_df.drop(["Id"], axis=1)
```

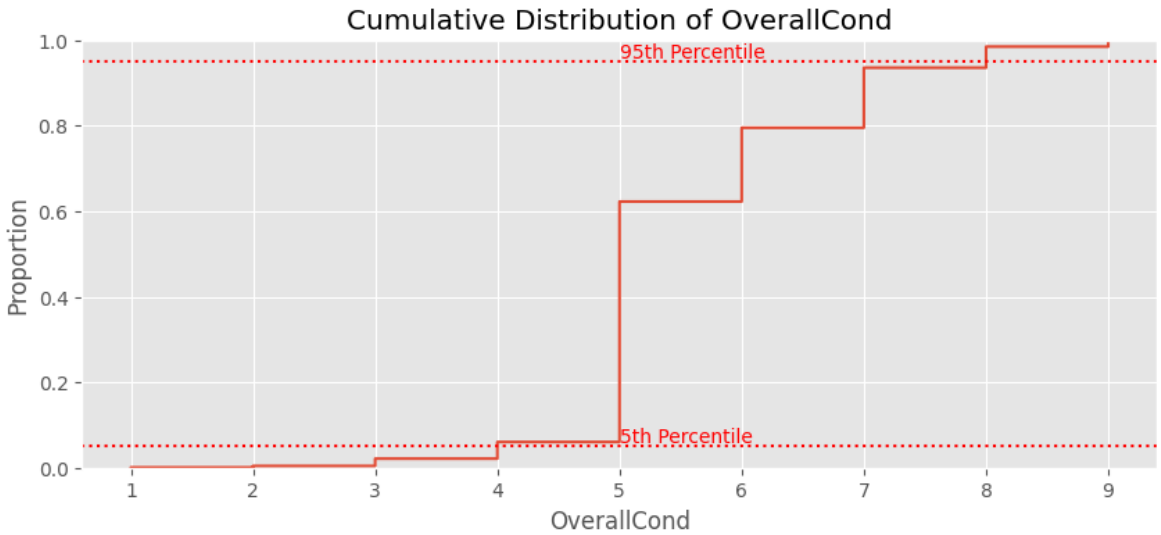
In [9]:
```
for column in numerical_df:
    plt.figure(figsize=(10, 4))
    sns.ecdfplot(data=numerical_df, x=column)

    # Add a line at 95% to see the "cutoff" for the extreme tail
    plt.axhline(0.95, color='red', linestyle=':')
    plt.annotate('95th Percentile', xy=(numerical_df[column].median(), 0.
    plt.axhline(0.05, color='red', linestyle=':')
    plt.annotate('5th Percentile', xy=(numerical_df[column].median(), 0.0

    plt.title(f'Cumulative Distribution of {column}')
    plt.show()
```

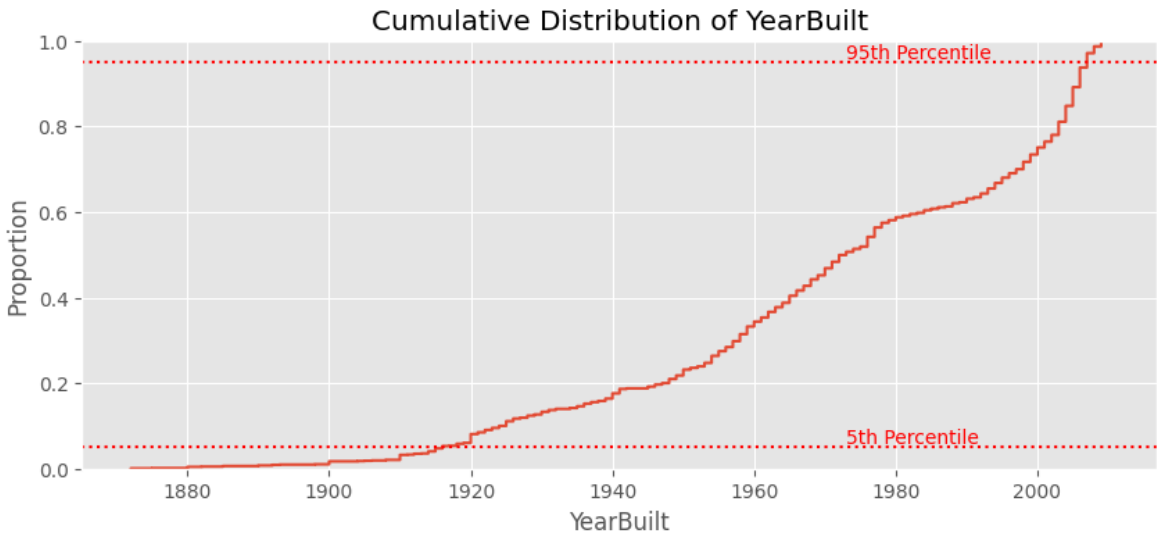

Cumulative Distribution of MSSubClass
95th Percentile
5th Percentile
Proportion
MSSubClass
1.0
0.8
0.6
0.4
0.2
0.0
25
50
75
100
125
150
175

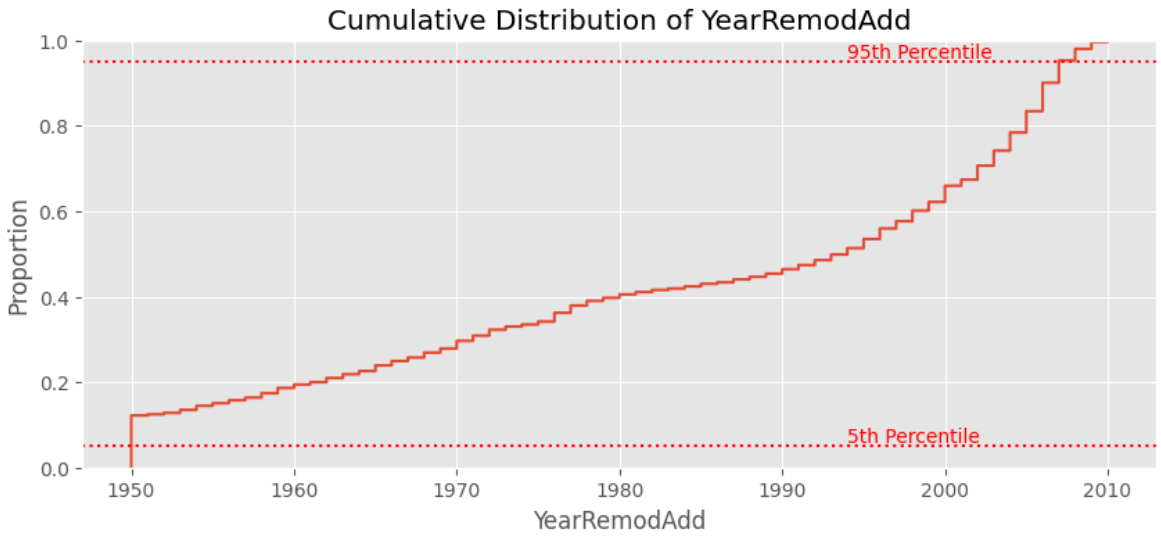

Cumulative Distribution of LotFrontage
95th Percentile
5th Percentile
Proportion
LotFrontage
1.0
0.8
0.6
0.4
0.2
0.0
50
100
150
200
250
300

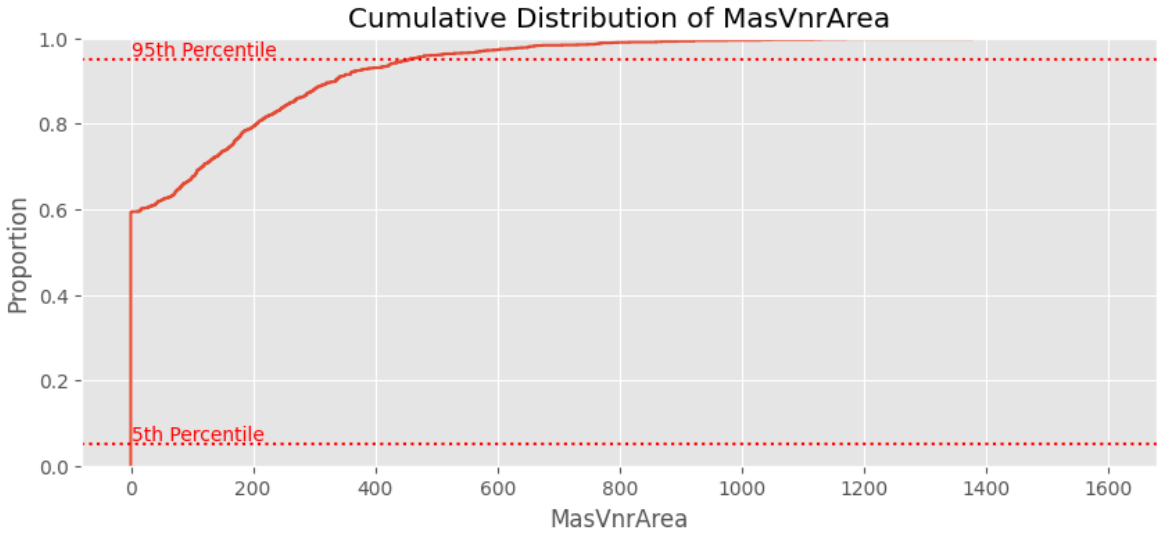

Cumulative Distribution of LotArea
95th Percentile
5th Percentile
Proportion
LotArea
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
200000

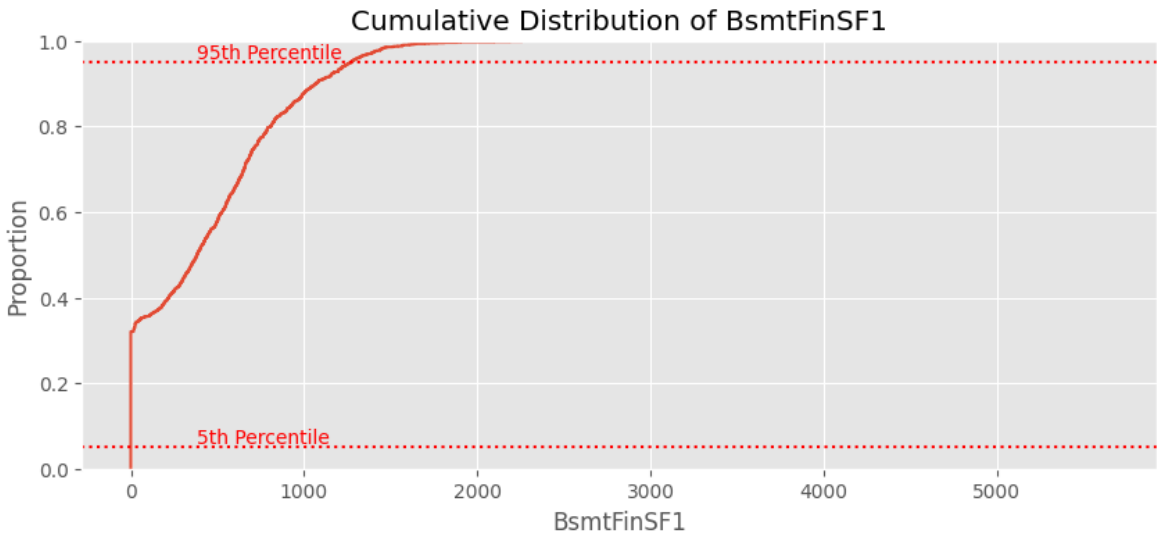

Cumulative Distribution of OverallQual
95th Percentile
5th Percentile
Proportion
OverallQual
1.0
0.8
0.6
0.4
0.2
0.0
2
4
6
8
10

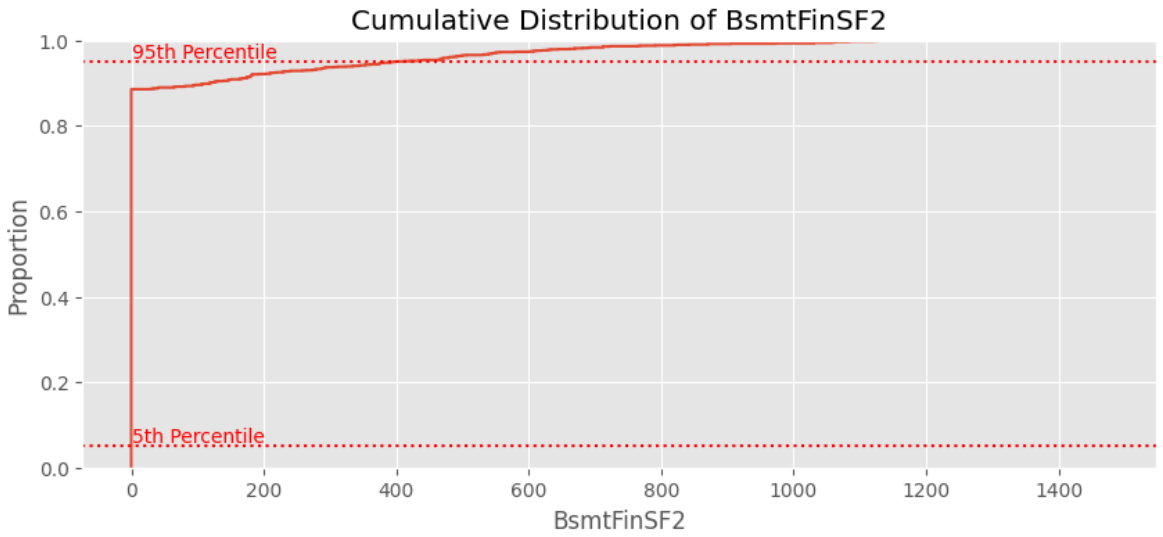

Cumulative Distribution of OverallCond
95th Percentile
5th Percentile
Proportion
OverallCond
1.0
0.8
0.6
0.4
0.2
0.0
1
2
3
4
5
6
7
8
9

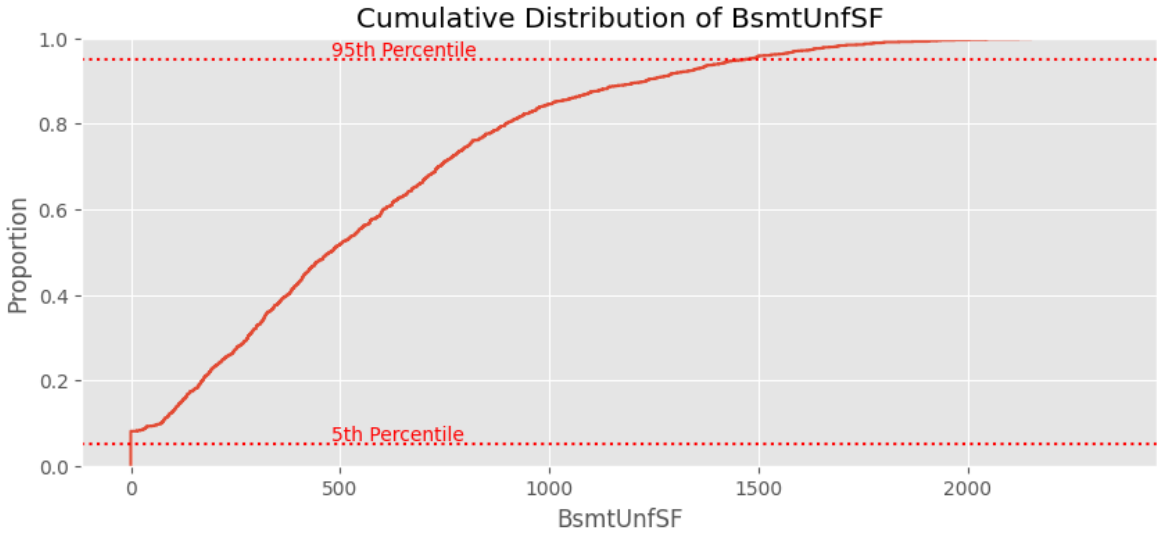

Cumulative Distribution of YearBuilt
95th Percentile
5th Percentile
Proportion
YearBuilt
1.0
0.8
0.6
0.4
0.2
0.0
1880
1900
1920
1940
1960
1980
2000

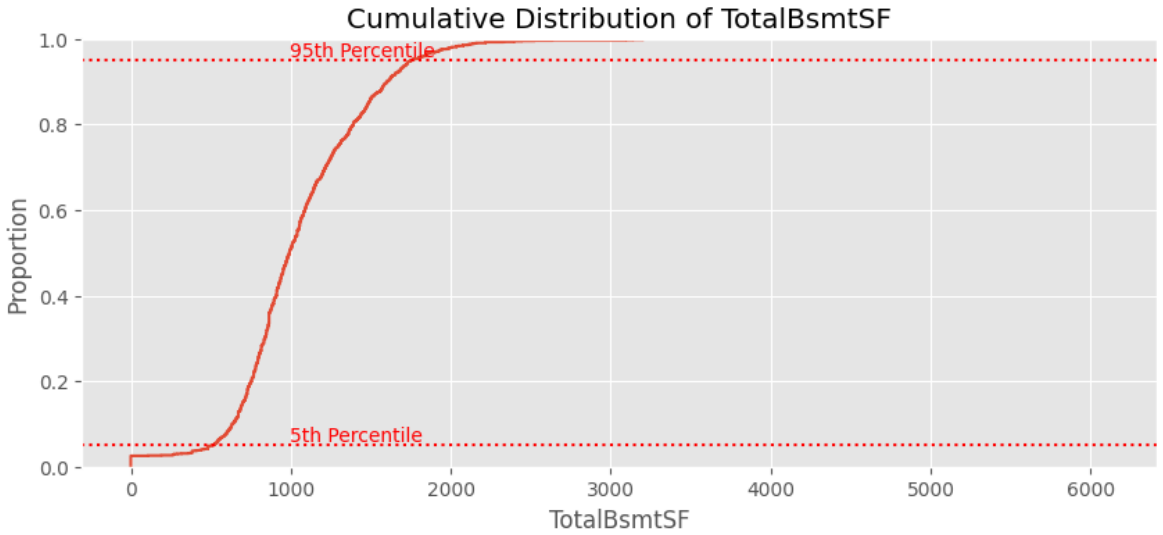

Cumulative Distribution of YearRemodAdd
95th Percentile
5th Percentile
Proportion
YearRemodAdd
1.0
0.8
0.6
0.4
0.2
0.0
1950
1960
1970
1980
1990
2000
2010

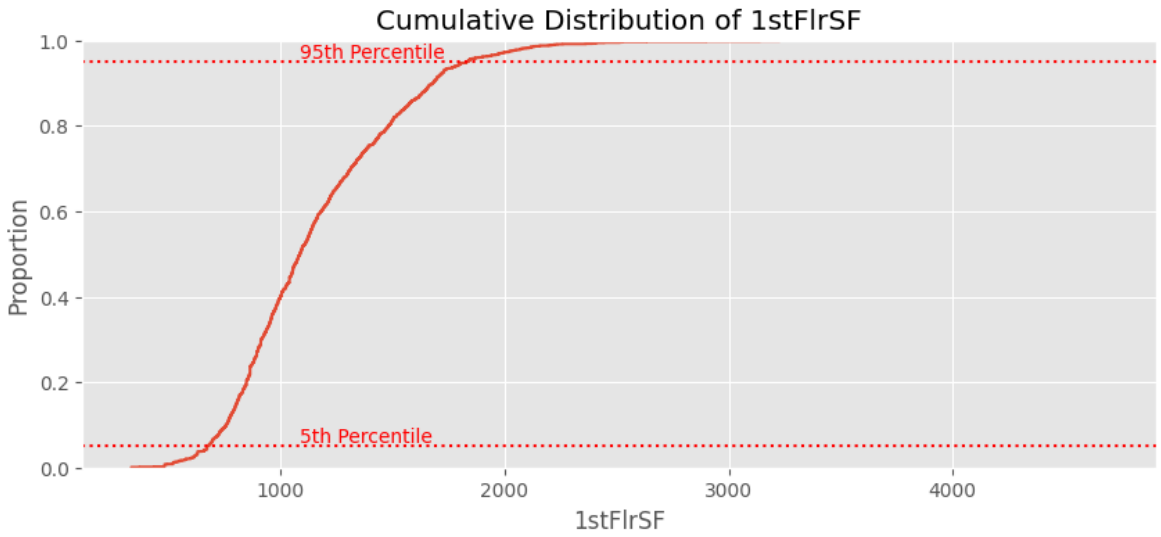

Cumulative Distribution of MasVnrArea
95th Percentile
5th Percentile
Proportion
MasVnrArea
1.0
0.8
0.6
0.4
0.2
0.0
0
200
400
600
800
1000
1200
1400
1600

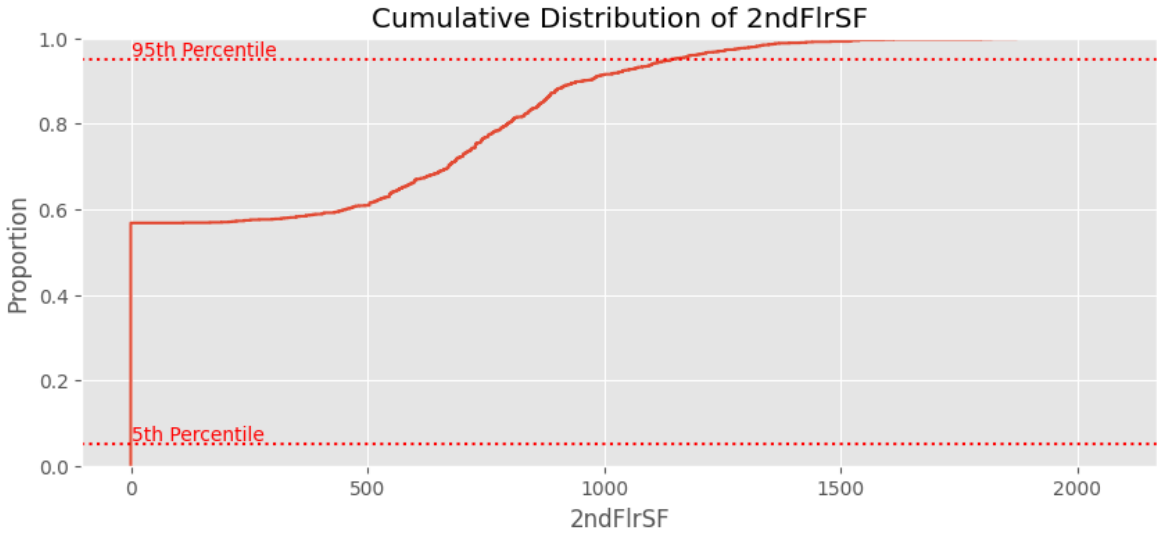

Cumulative Distribution of BsmtFinSF1
95th Percentile
5th Percentile
Proportion
BsmtFinSF1
1.0
0.8
0.6
0.4
0.2
0.0
0
1000
2000
3000
4000
5000

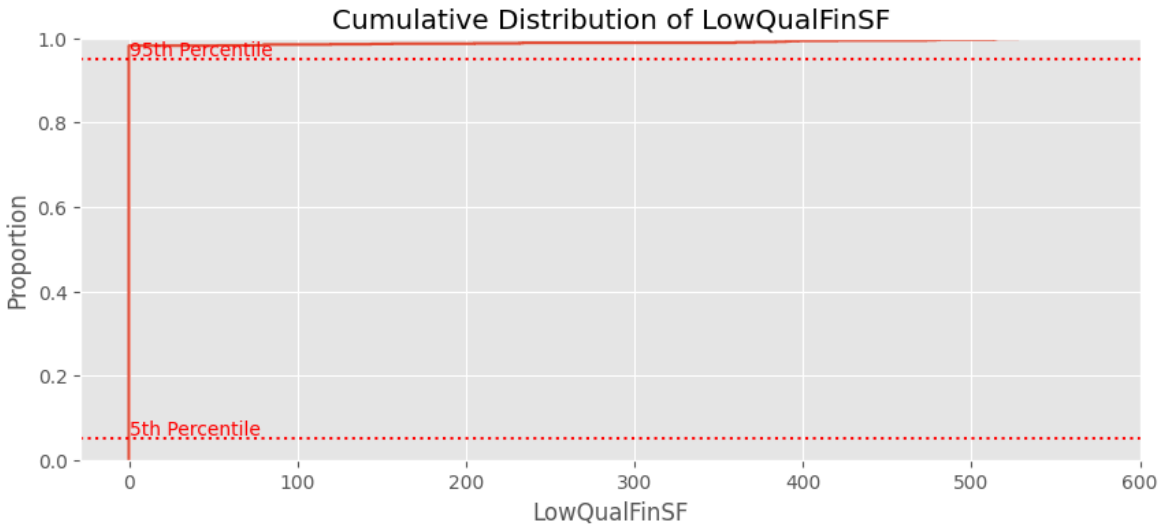

Cumulative Distribution of BsmtFinSF2
95th Percentile
5th Percentile
Proportion
BsmtFinSF2
0.0
0.2
0.4
0.6
0.8
1.0
0
200
400
600
800
1000
1200
1400

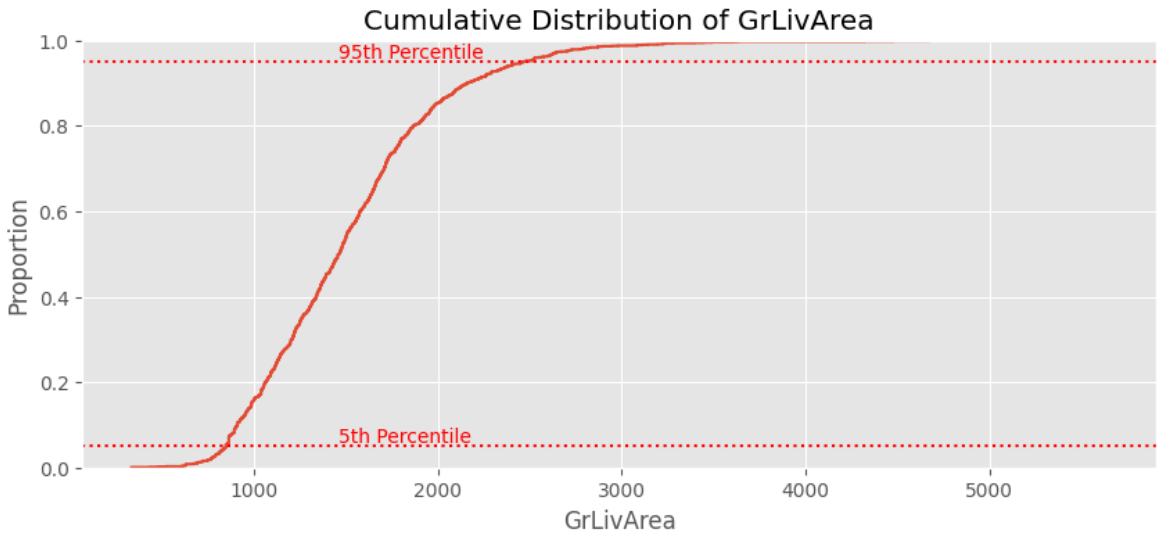

Cumulative Distribution of BsmtUnfSF
95th Percentile
5th Percentile
Proportion
BsmtUnfSF
0.0
0.2
0.4
0.6
0.8
1.0
0
500
1000
1500
2000

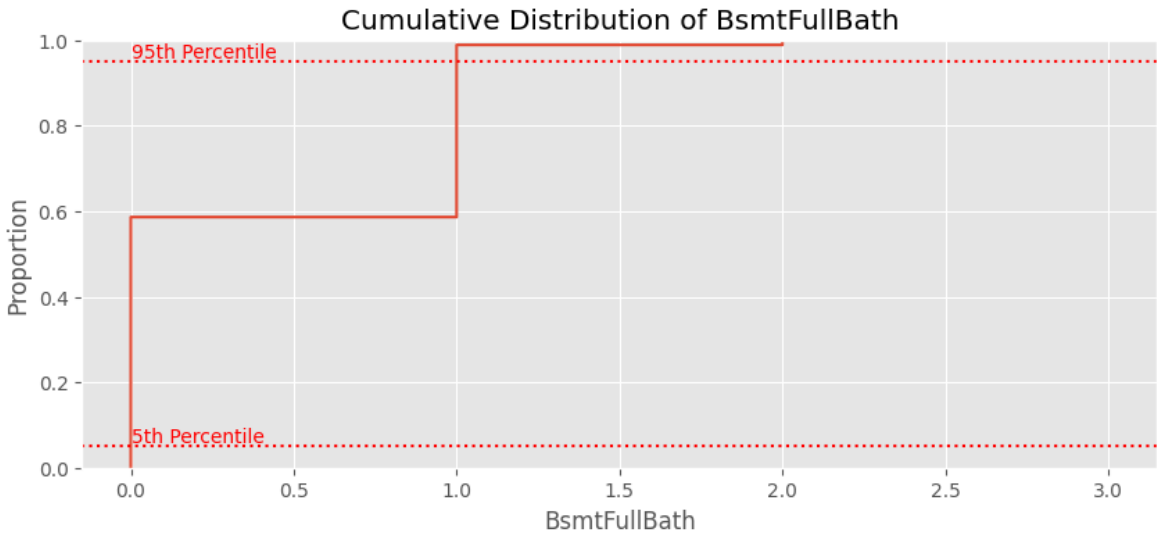

Cumulative Distribution of TotalBsmtSF
95th Percentile
5th Percentile
Proportion
TotalBsmtSF
0.0
0.2
0.4
0.6
0.8
1.0
0
1000
2000
3000
4000
5000
6000

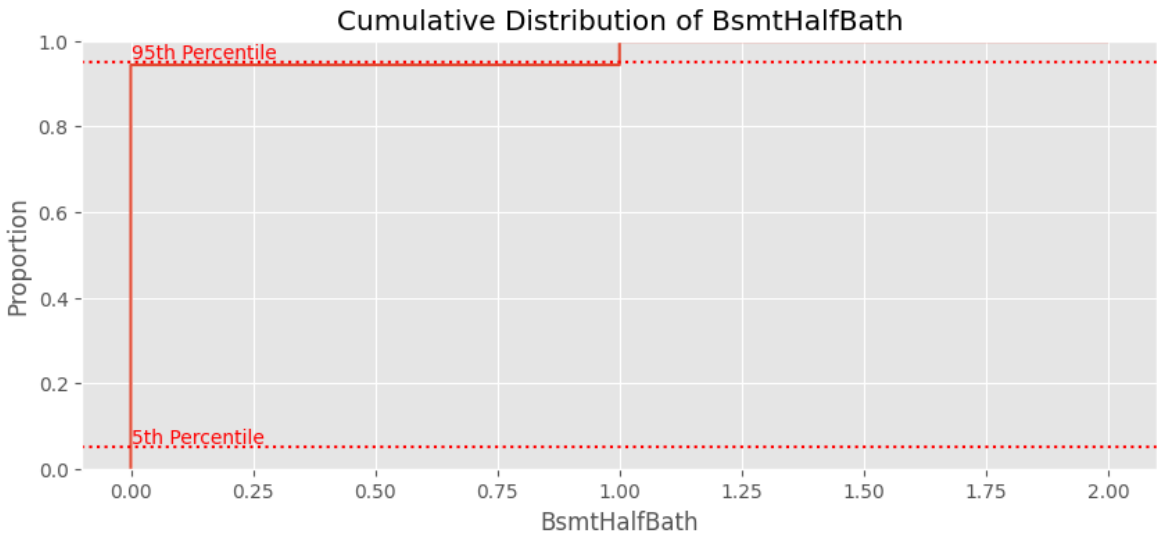

Cumulative Distribution of 1stFlrSF
95th Percentile
5th Percentile
Proportion
1stFlrSF
0.0
0.2
0.4
0.6
0.8
1.0
1000
2000
3000
4000

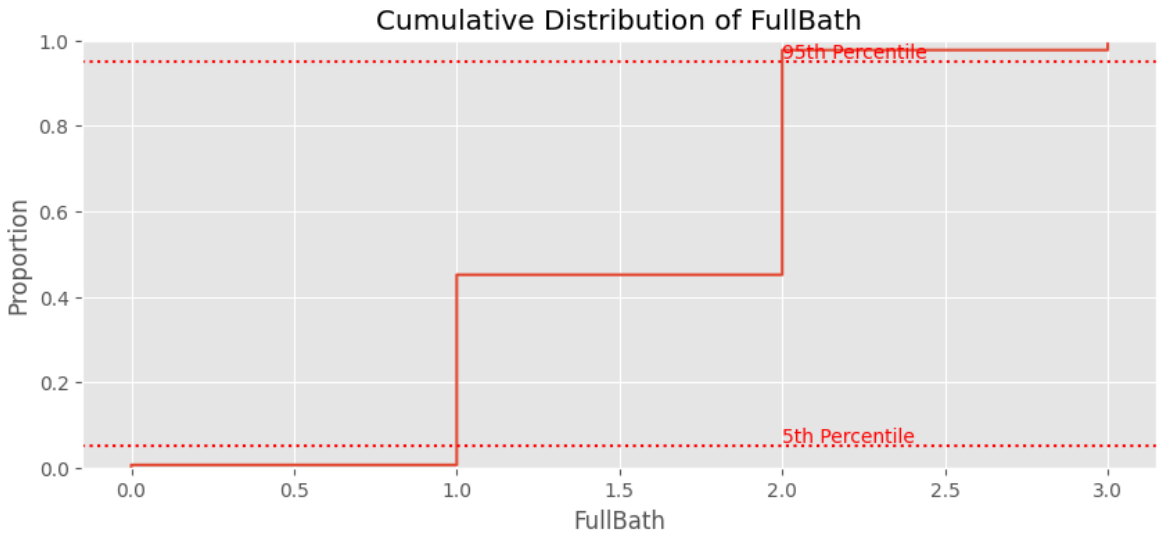

Cumulative Distribution of 2ndFlrSF
95th Percentile
5th Percentile
Proportion
2ndFlrSF
0.0
0.2
0.4
0.6
0.8
1.0
0
500
1000
1500
2000

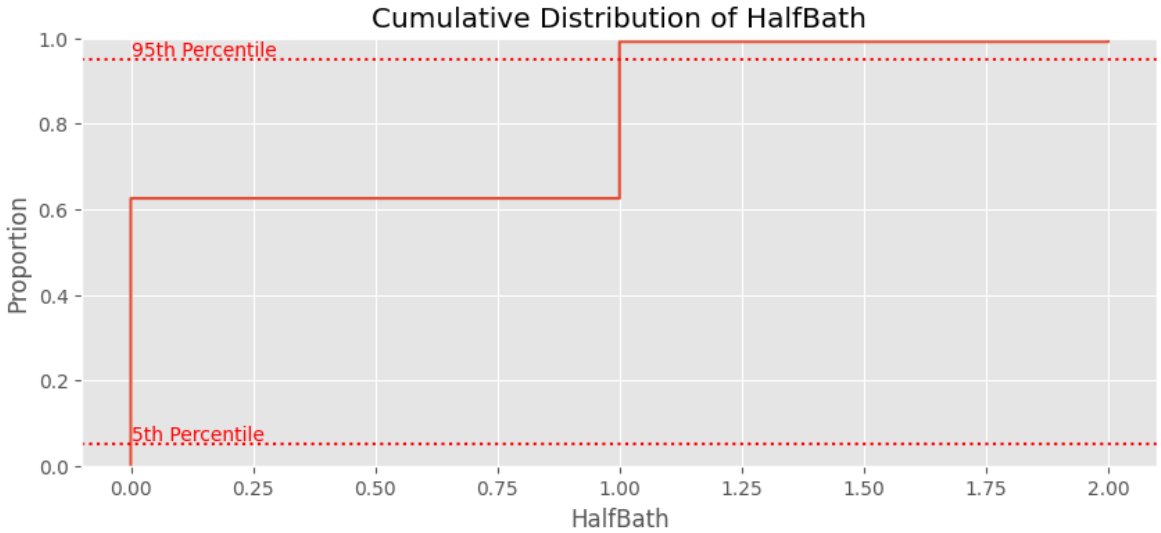

Cumulative Distribution of LowQualFinSF
95th Percentile
5th Percentile
Proportion
LowQualFinSF
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500
600

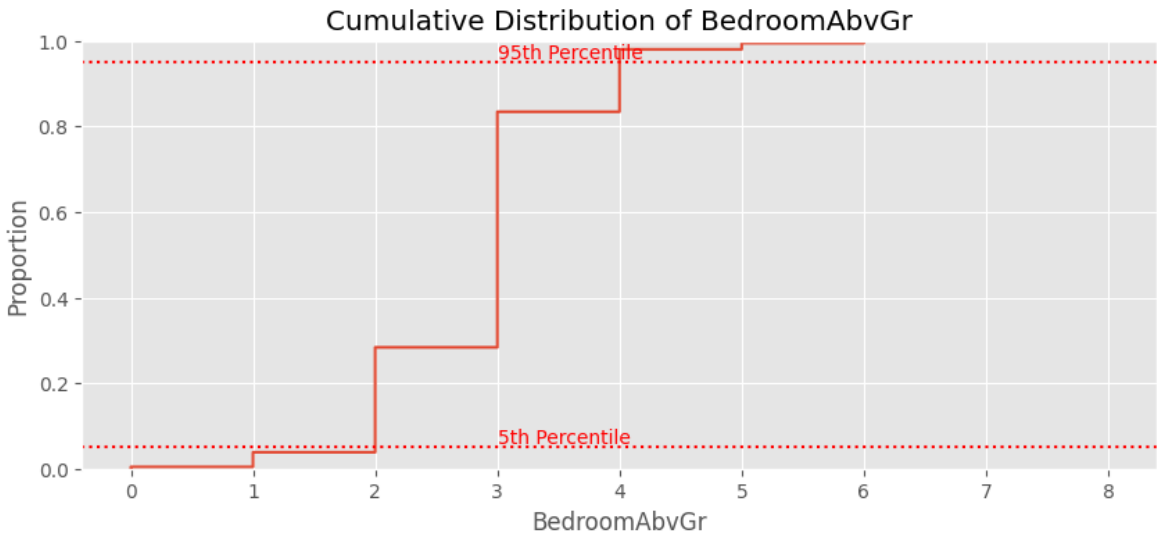

Cumulative Distribution of GrLivArea
95th Percentile
5th Percentile
Proportion
GrLivArea
1.0
0.8
0.6
0.4
0.2
0.0
1000
2000
3000
4000
5000

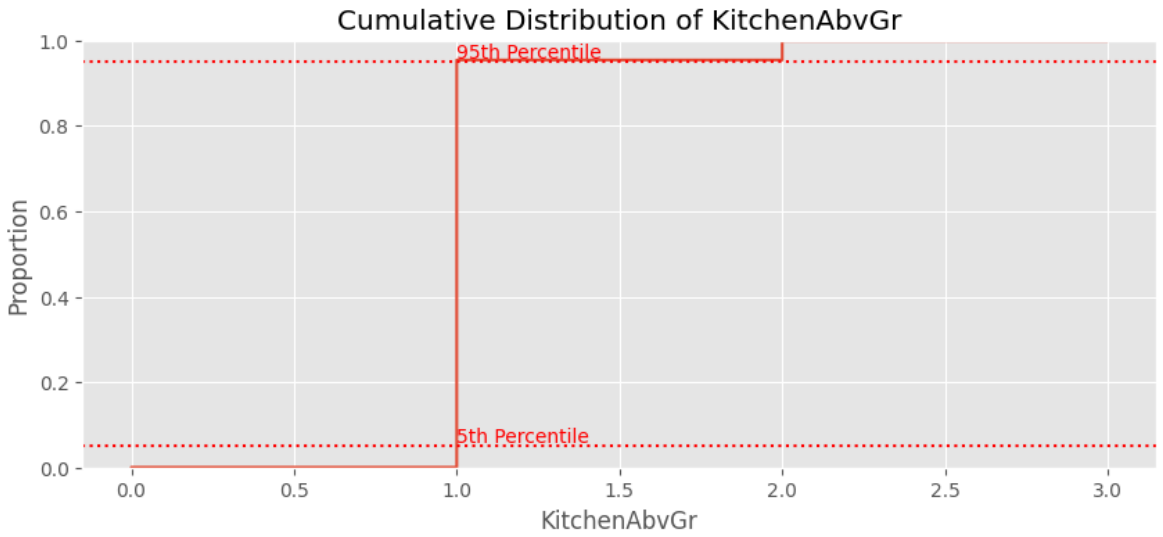

Cumulative Distribution of BsmtFullBath
95th Percentile
5th Percentile
Proportion
BsmtFullBath
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0

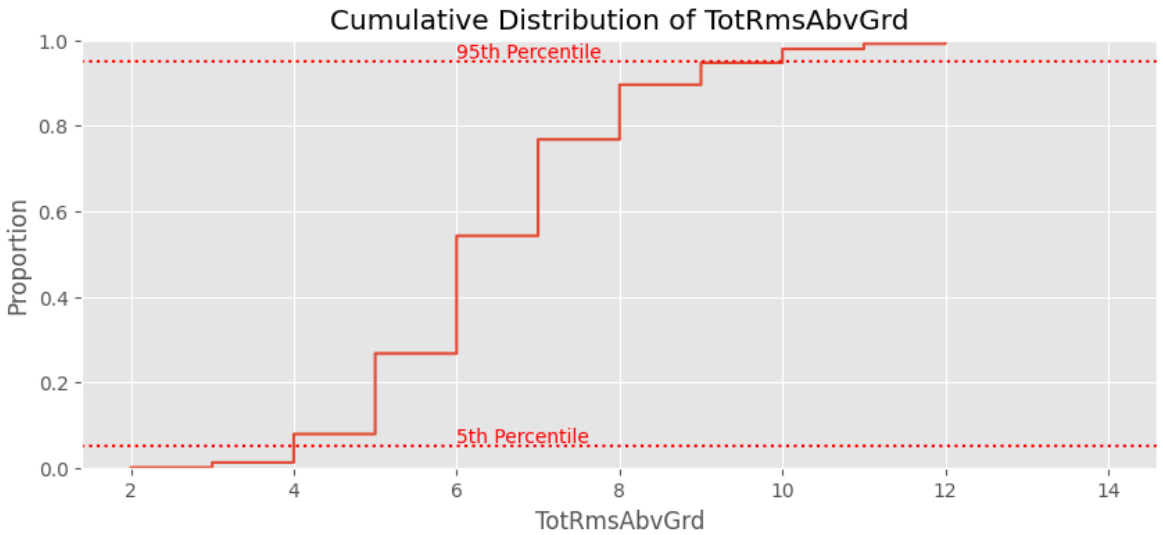

Cumulative Distribution of BsmtHalfBath
95th Percentile
5th Percentile
Proportion
BsmtHalfBath
1.0
0.8
0.6
0.4
0.2
0.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

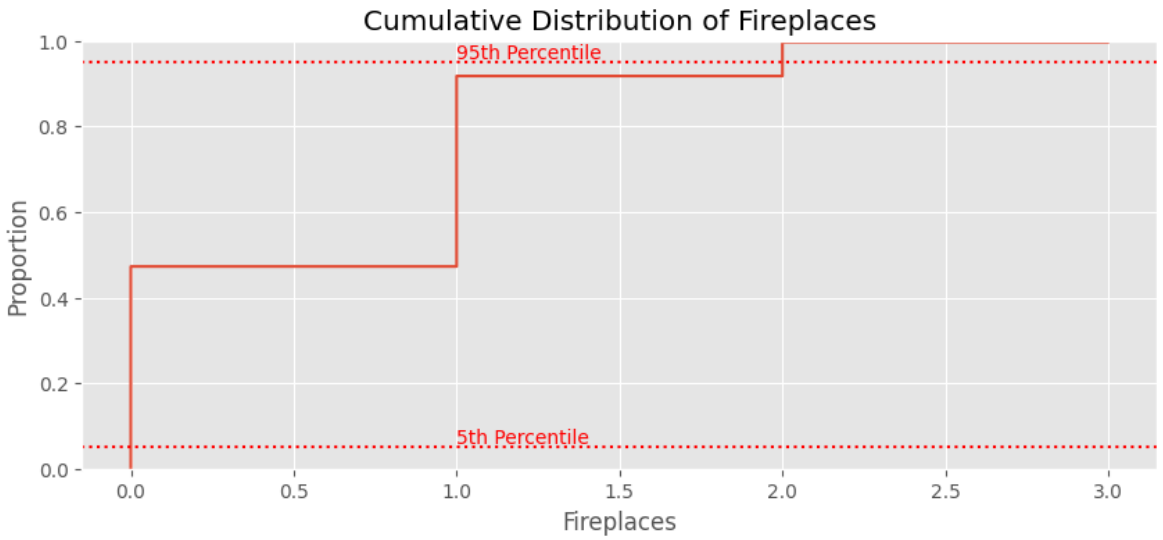

Cumulative Distribution of FullBath
95th Percentile
5th Percentile
Proportion
FullBath
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0

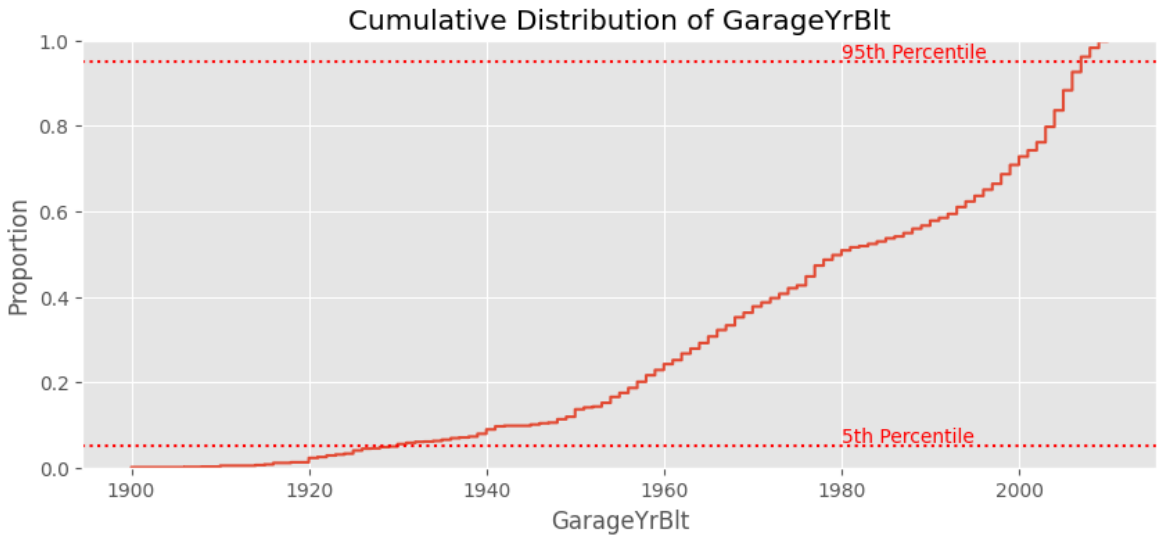

Cumulative Distribution of HalfBath
95th Percentile
5th Percentile
Proportion
HalfBath
0.0
0.2
0.4
0.6
0.8
1.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

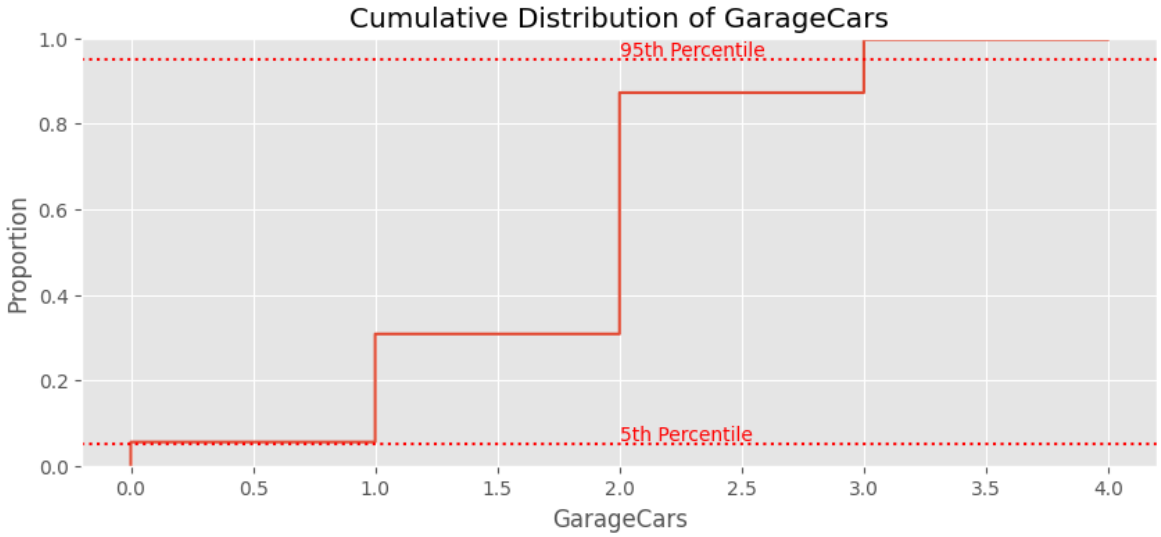

Cumulative Distribution of BedroomAbvGr
95th Percentile
5th Percentile
Proportion
BedroomAbvGr
0.0
0.2
0.4
0.6
0.8
1.0
0
1
2
3
4
5
6
7
8

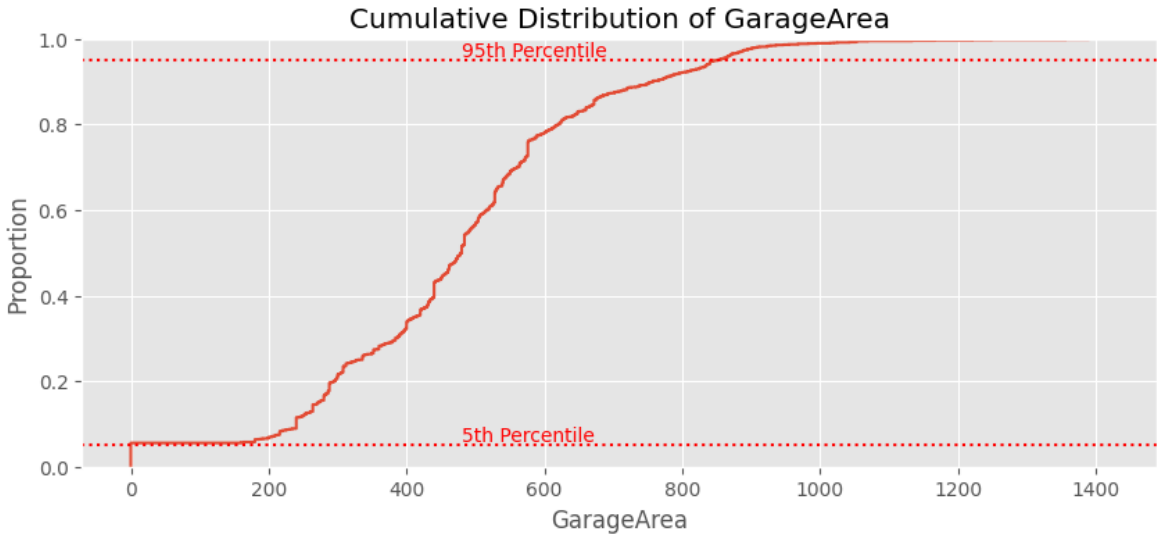

Cumulative Distribution of KitchenAbvGr
95th Percentile
5th Percentile
Proportion
KitchenAbvGr
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0

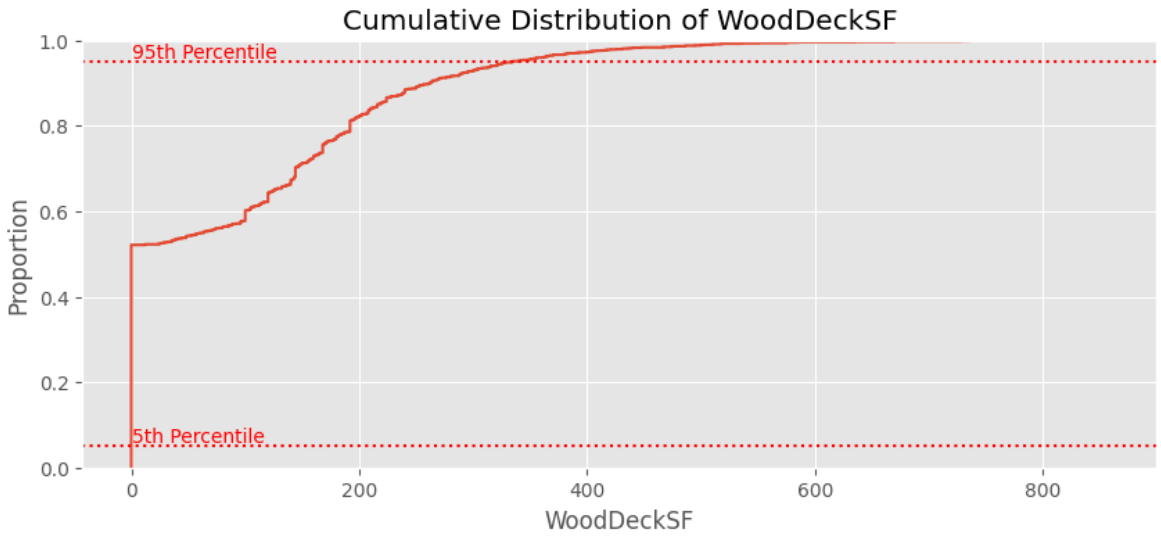

Cumulative Distribution of TotRmsAbvGrd
95th Percentile
5th Percentile
Proportion
TotRmsAbvGrd
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
10
12
14

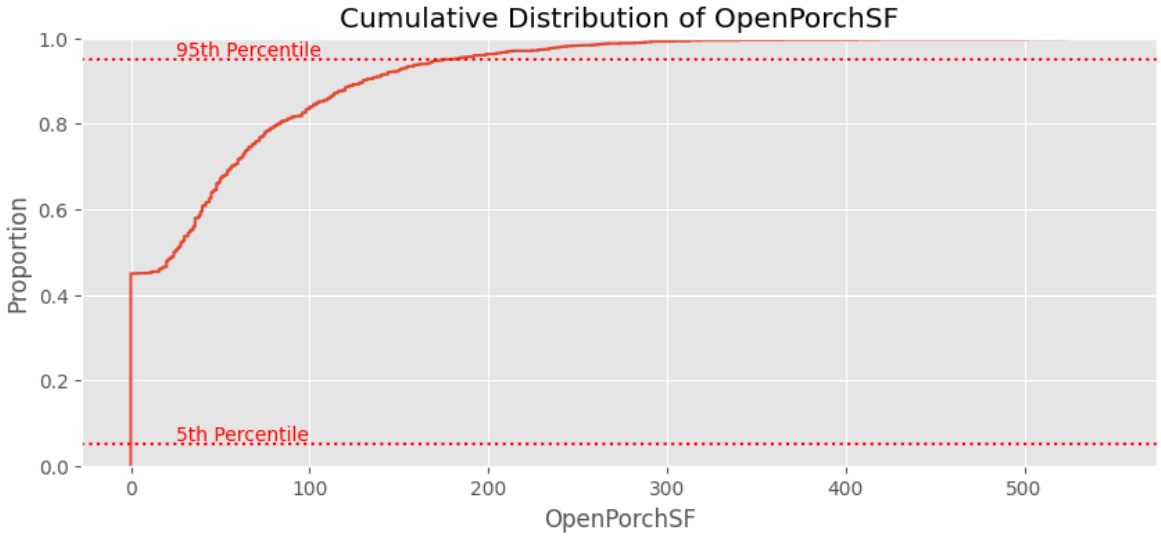

Cumulative Distribution of Fireplaces
95th Percentile
5th Percentile
Proportion
Fireplaces
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0

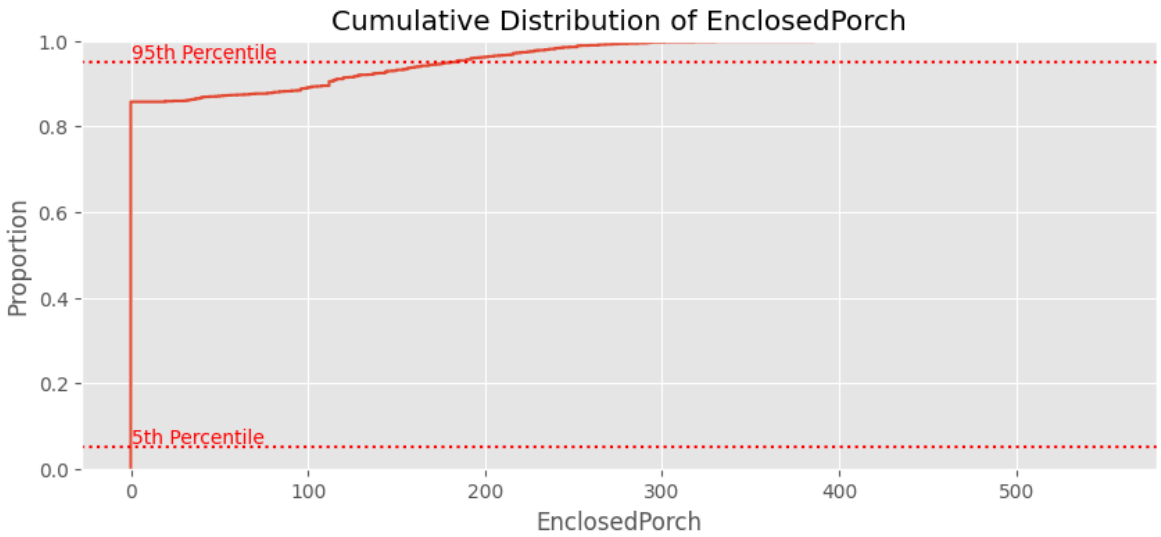

Cumulative Distribution of GarageYrBlt
95th Percentile
5th Percentile
Proportion
GarageYrBlt
1.0
0.8
0.6
0.4
0.2
0.0
1900
1920
1940
1960
1980
2000

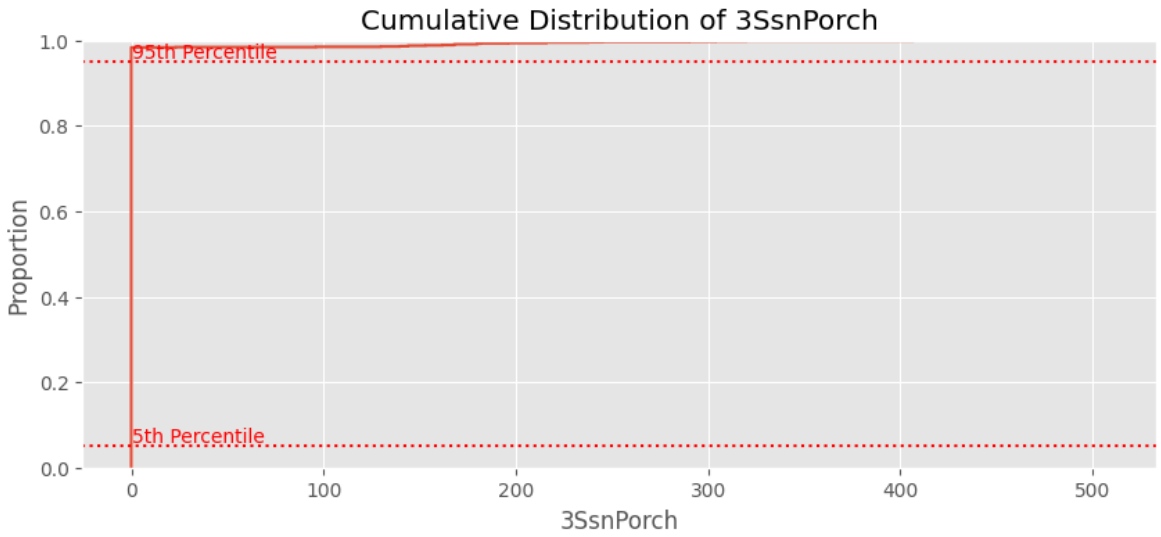

Cumulative Distribution of GarageCars
95th Percentile
5th Percentile
Proportion
GarageCars
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

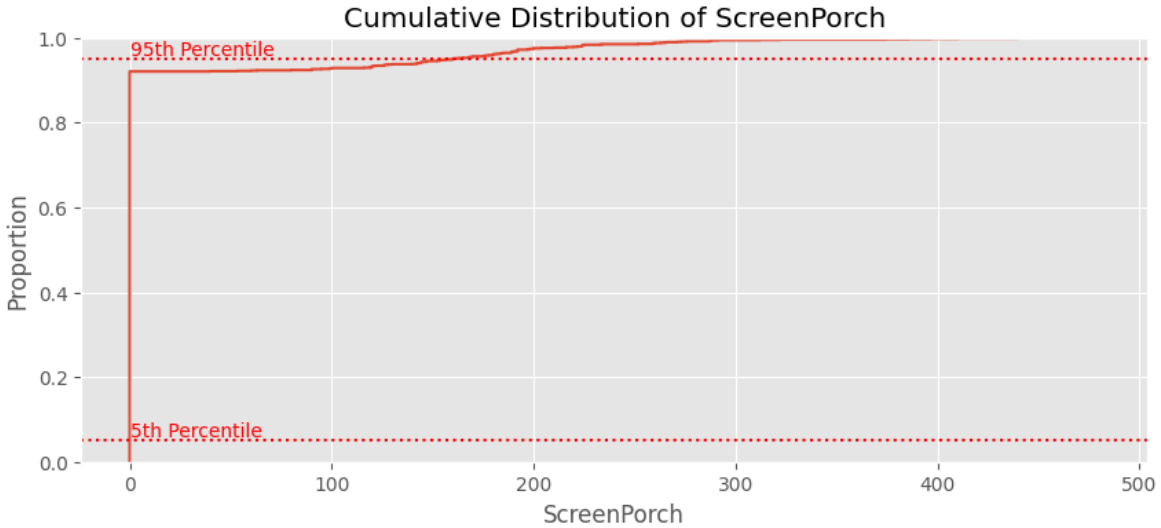

Cumulative Distribution of GarageArea
95th Percentile
5th Percentile
Proportion
GarageArea
1.0
0.8
0.6
0.4
0.2
0.0
0
200
400
600
800
1000
1200
1400

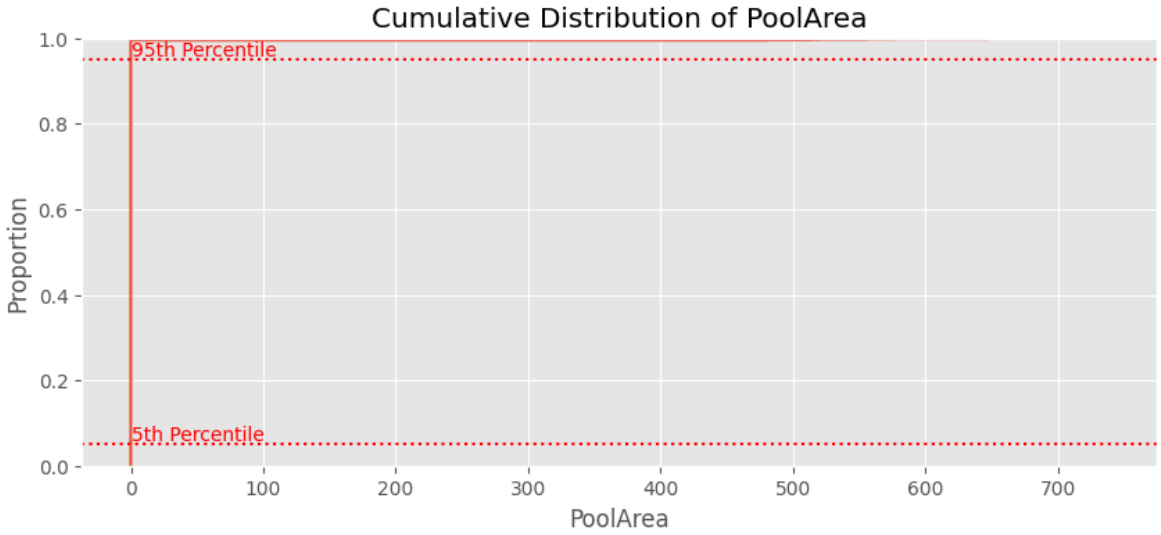

Cumulative Distribution of WoodDeckSF
95th Percentile
5th Percentile
Proportion
WoodDeckSF
0.0
0.2
0.4
0.6
0.8
1.0
0
200
400
600
800

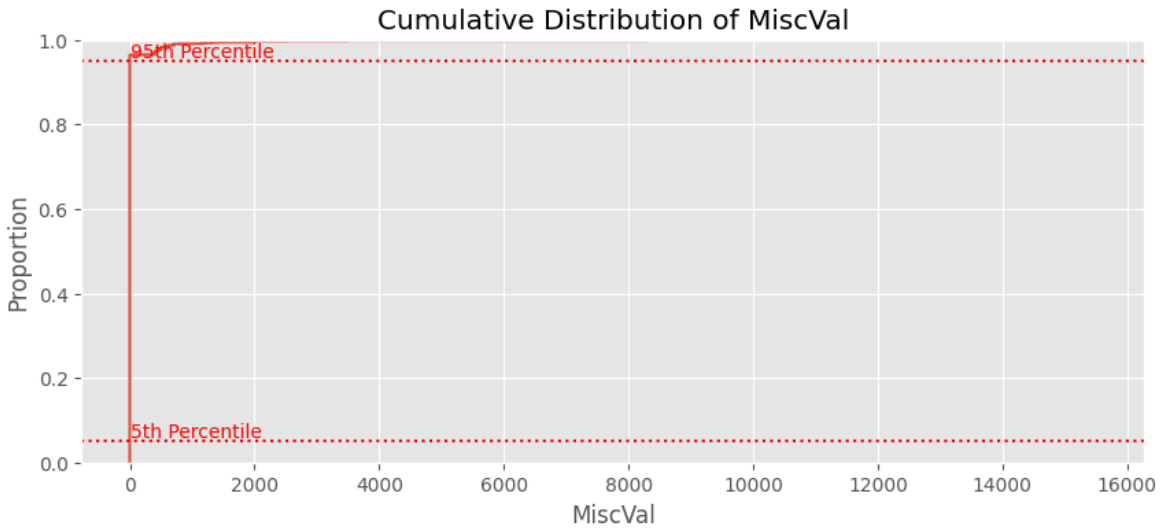

Cumulative Distribution of OpenPorchSF
95th Percentile
5th Percentile
Proportion
OpenPorchSF
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500

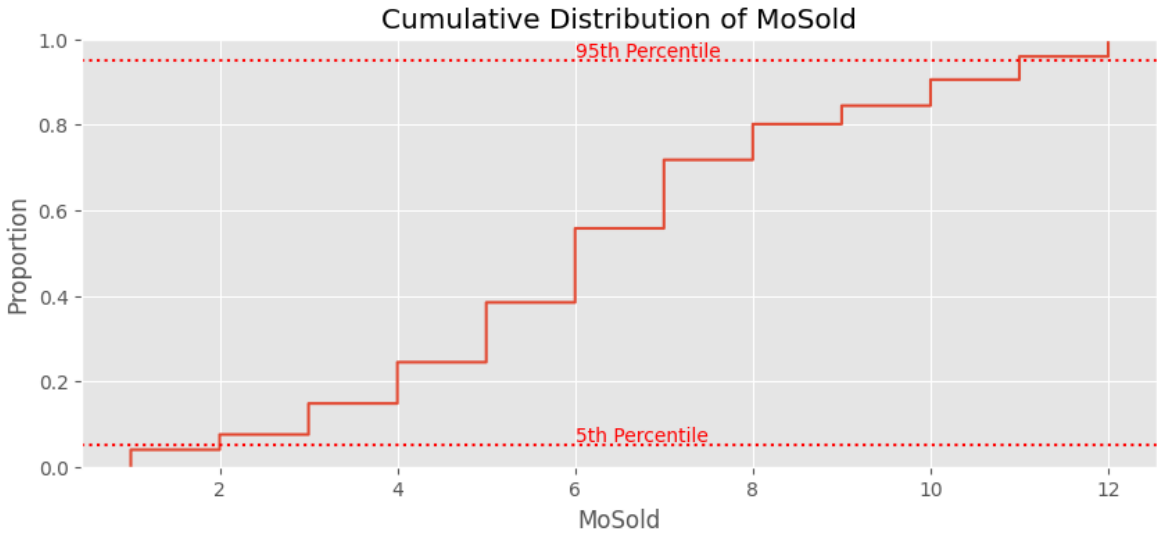

Cumulative Distribution of EnclosedPorch
95th Percentile
5th Percentile
Proportion
EnclosedPorch
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500

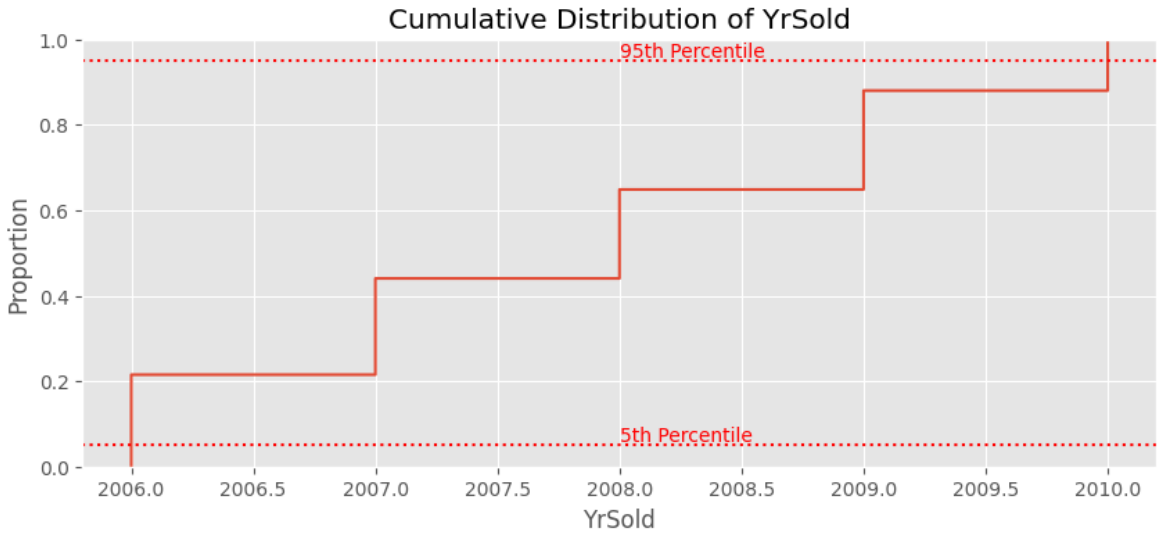

Cumulative Distribution of 3SsnPorch
95th Percentile
5th Percentile
Proportion
3SsnPorch
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500

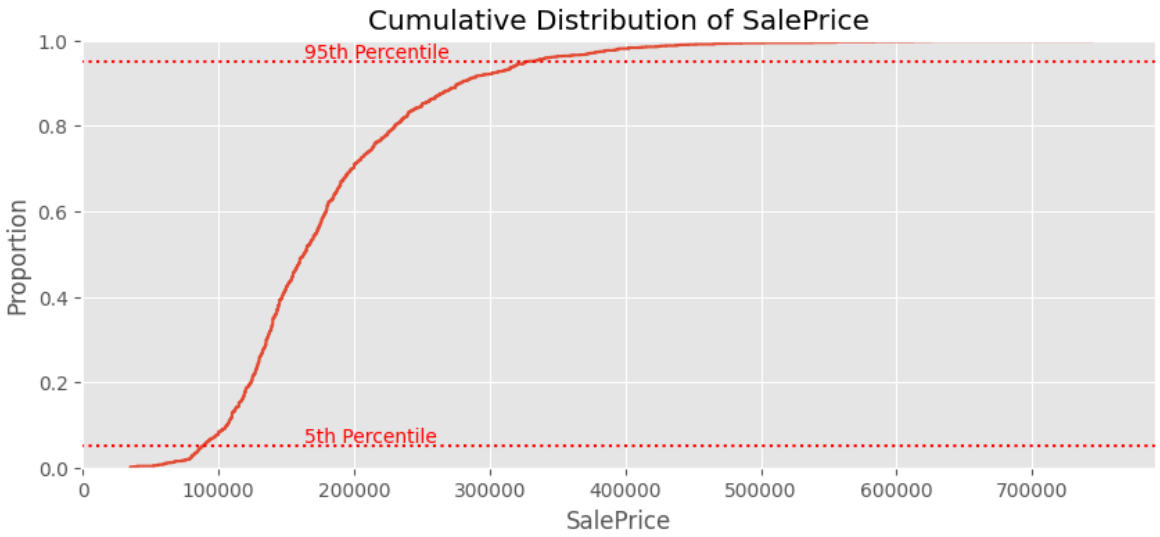

Cumulative Distribution of ScreenPorch
95th Percentile
5th Percentile
Proportion
ScreenPorch
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500


Cumulative Distribution of PoolArea
95th Percentile
5th Percentile
Proportion
PoolArea
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500
600
700


Cumulative Distribution of MiscVal
95th Percentile
5th Percentile
Proportion
MiscVal
0.0
0.2
0.4
0.6
0.8
1.0
0
2000
4000
6000
8000
10000
12000
14000
16000


Cumulative Distribution of MoSold
95th Percentile
5th Percentile
Proportion
MoSold
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
10
12


Cumulative Distribution of YrSold
95th Percentile
5th Percentile
Proportion
YrSold
0.0
0.2
0.4
0.6
0.8
1.0
2006.0
2006.5
2007.0
2007.5
2008.0
2008.5
2009.0
2009.5
2010.0



In [10]:
```
# # Seeing it with boxplots
# for column in numerical_df:
#     plt.figure(figsize=(16, 4))
#     sns.set_theme(style='whitegrid')
#     sns.boxplot(numerical_df[column], orient='h')
#     plt.xlabel(column)
#     plt.show()
```

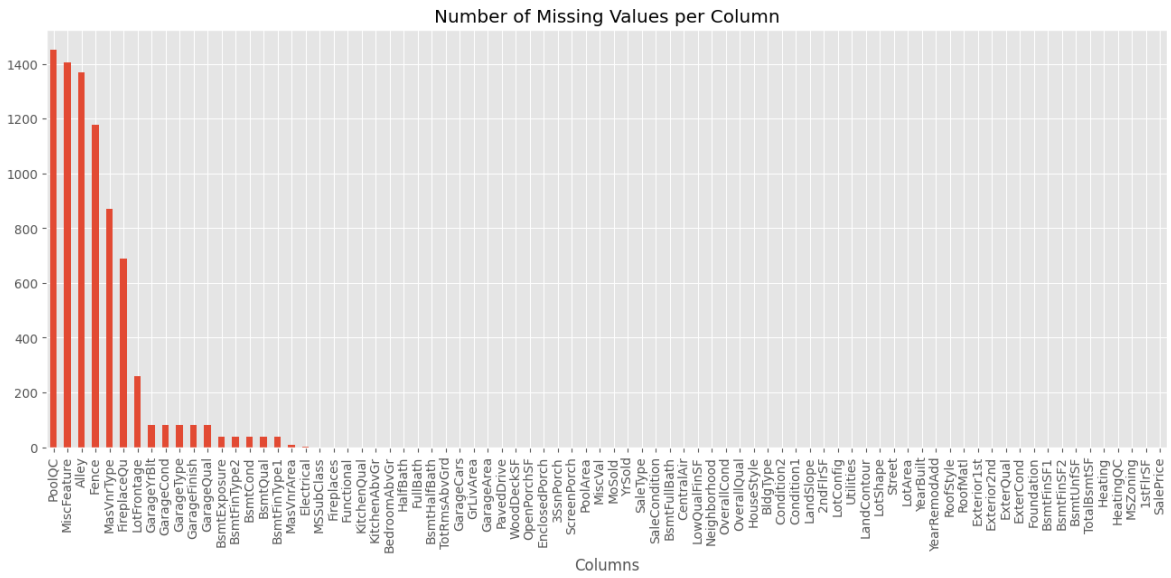
In [11]:
```
# # With violin plots

# for column in numerical_df:
#     plt.figure(figsize=(12, 6))

#     # Violin plot gives the shape; inner='box' adds the mini-boxplot in
#     sns.violinplot(data=numerical_df, x=column, inner="box", palette="P

#     # Stripplot adds the "dots" so you can see if outliers are scattere
#     sns.stripplot(data=numerical_df, x=column, color="black", size=2, a

#     plt.title(f'Raincloud-style Distribution of {column}')
#     plt.show()
```

Outlier conclusion: Whether we need to do somethign about it will depend on what model we want to use later. For example, linear regression is very sensitive to outliers, so we might want to remove them or transform the data. On the other hand, tree-based models like random forests and gradient boosting are more robust to outliers, so we might not need to do anything.

Check Nan values

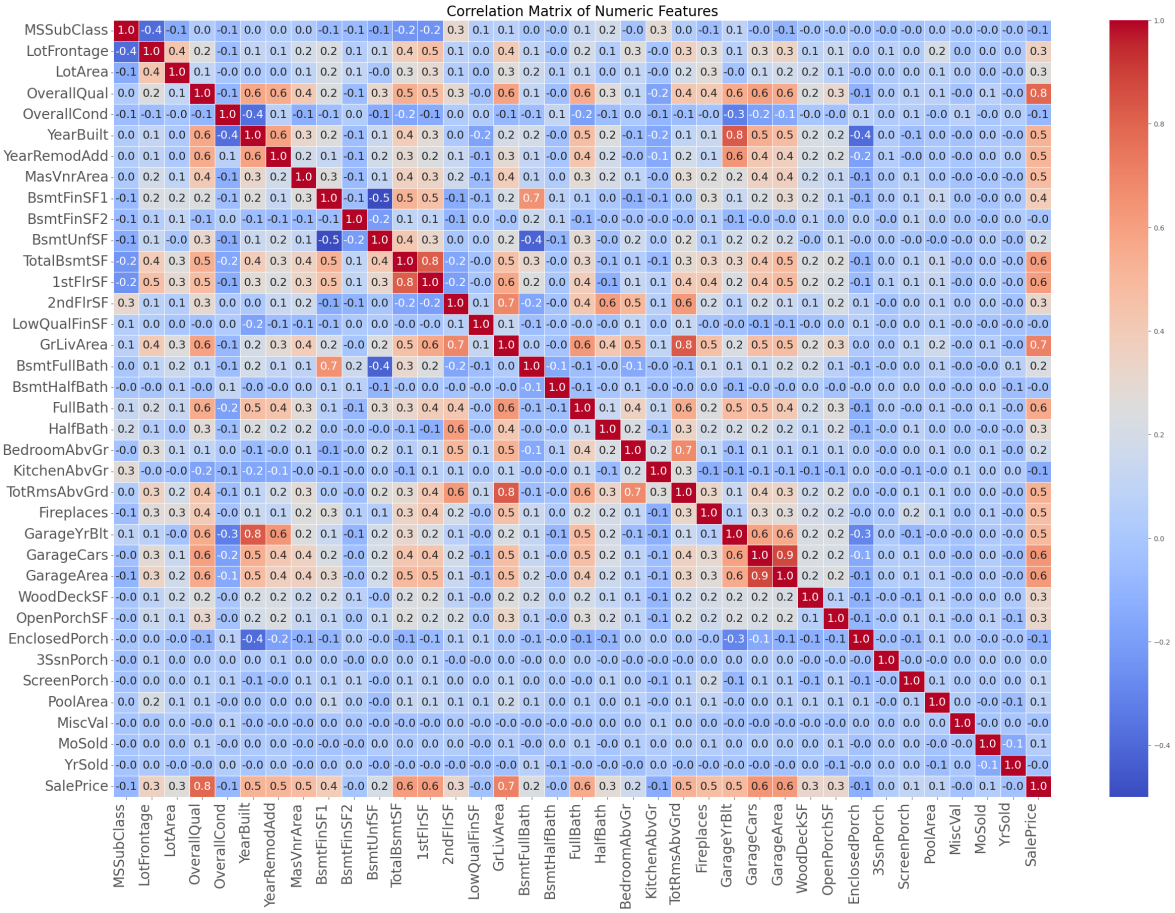
In [12]:
```
df_train.isna().sum().sort_values(ascending=False).plot(kind='bar', figsi
plt.title('Number of Missing Values per Column')
plt.xlabel('Columns')
```

Out[12]:
```
Text(0.5, 0, 'Columns')
```



Check correlations

In [13]:
```
# Look up correlation between numeric features

# Select only numeric columns for calculating the correlation matrix
numeric_df = df_train.select_dtypes(include='number')

# Calculate the correlation matrix
df_corr = numeric_df.corr().round(2)

# Create a heatmap
plt.figure(figsize=(30, 20))
ax = sns.heatmap(df_corr, annot=True, cmap='coolwarm', fmt="0.1f", linewi
ax.set_xticks(ax.get_xticks(), df_corr.columns, rotation=90, ha='right',
ax.set_yticks(ax.get_yticks(), df_corr.columns, rotation=0, va='center',
plt.title('Correlation Matrix of Numeric Features', fontsize=20)
plt.show()
```



Data Cleanup

Merge df_train and df_inference for simplicity

In [14]:
```
# Merge the dataframes together
df = pd.concat([df_train, df_inference])

# Reset the index
df.reset_index(drop=True, inplace=True)

df.shape
```

Out[14]: (2919, 80)

In [15]:
```
df
```

Out[15]:

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandCor
0	60	RL	65.0	8450	Pave	NaN	Reg	
1	20	RL	80.0	9600	Pave	NaN	Reg	
2	60	RL	68.0	11250	Pave	NaN	IR1	
3	70	RL	60.0	9550	Pave	NaN	IR1	
4	60	RL	84.0	14260	Pave	NaN	IR1	
...	...	...	...	...	...	...	...	
2914	160	RM	21.0	1936	Pave	NaN	Reg	
2915	160	RM	21.0	1894	Pave	NaN	Reg	
2916	20	RL	160.0	20000	Pave	NaN	Reg	
2917	85	RL	62.0	10441	Pave	NaN	Reg	
2918	60	RL	74.0	9627	Pave	NaN	Reg	

2919 rows × 80 columns

Rename Columns

In [16]:
```
df.columns
```

Out[16]:
```
Index(['MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street', 'Al
ley',
       'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope',
       'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseSty
le',
       'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'RoofS
tyle',
       'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'MasVnrAr
ea',
       'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond',
       'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1', 'BsmtFinType2',
       'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating', 'HeatingQC',
       'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF', 'LowQualFinS
F',
       'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBat
h',
       'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual', 'TotRmsAbvGrd',
       'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType', 'GarageY
rBlt',
       'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual', 'Garage
Cond',
       'PavedDrive', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3Ssn
Porch',
       'ScreenPorch', 'PoolArea', 'PoolQC', 'Fence', 'MiscFeature', 'Mis
cVal',
       'MoSold', 'YrSold', 'SaleType', 'SaleCondition', 'SalePrice'],
      dtype='object')
```

In [17]:
```
df = df.rename(columns = {"1stFlrSF" : "FirstFlrSF", "2ndFlrSF" : "Second
```

Imput missing data

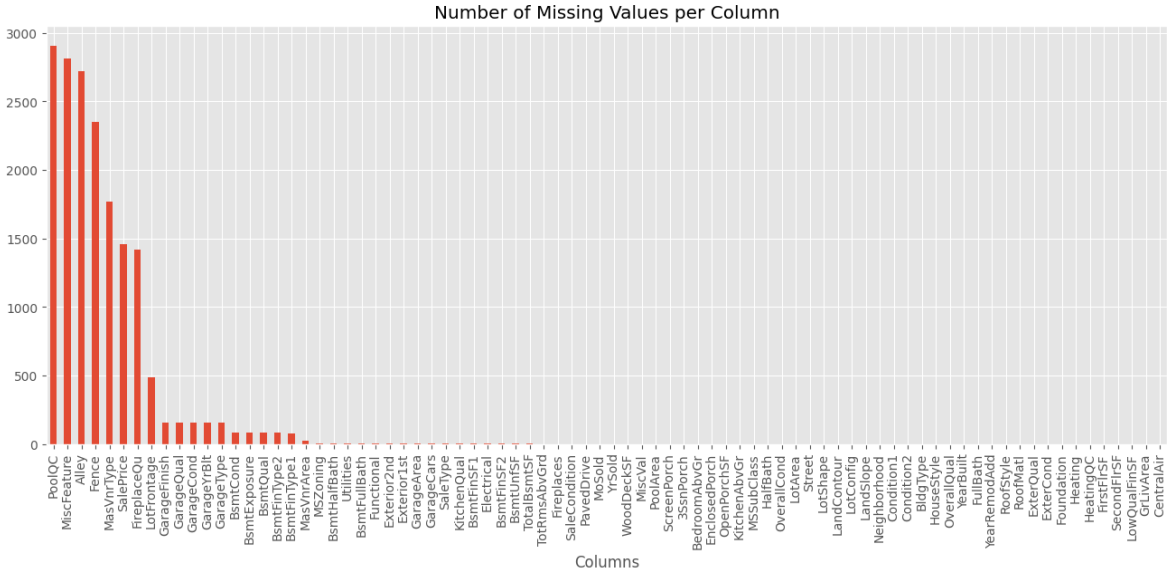
Strategy	One-Liner Code	Best Case Scenario
Fixed/ Constant	`df[col] = df[col].fillna(0)`	When missing inherently means "None" or "Zero".
Median	`df[col] = df[col].fillna(df[col].median())`	Numerical data with outliers (skewed distributions).
Mode	`df[col] = df[col].fillna(df[col].mode()[0])`	Categorical features (Nominal data).
Grouped	`df[col] = df.groupby('Key')[col].transform(lambda x: x.fillna(x.median()))`	Values that depend on a context (e.g., Neighborhood).
Forward Fill	`df[col] = df[col].ffill()`	Time-series or sequentially ordered data.
Indicator	`df['col_is_missing'] = df[col].isnull().astype(int)`	When the "fact of being missing" is a predictive signal.

In [ ]:

In [18]:
```
df.isna().sum().sort_values(ascending=False).head(36)
```

Out[18]:
```
PoolQC          2909
MiscFeature     2814
Alley           2721
Fence           2348
MasVnrType      1766
SalePrice       1459
FireplaceQu     1420
LotFrontage      486
GarageFinish     159
GarageQual       159
GarageCond       159
GarageYrBlt      159
GarageType       157
BsmtCond          82
BsmtExposure      82
BsmtQual          81
BsmtFinType2      80
BsmtFinType1      79
MasVnrArea        23
MSZoning           4
BsmtHalfBath       2
Utilities          2
BsmtFullBath       2
Functional         2
Exterior2nd        1
Exterior1st        1
GarageArea         1
GarageCars         1
SaleType           1
KitchenQual        1
BsmtFinSF1         1
Electrical         1
BsmtFinSF2         1
BsmtUnfSF          1
TotalBsmtSF        1
TotRmsAbvGrd       0
dtype: int64
```

In [19]:
```
df.isna().sum().sort_values(ascending=False).plot(kind='bar', figsize=(16
plt.title('Number of Missing Values per Column')
plt.xlabel('Columns')
plt.show()
```



Dealing with the missing values

In [20]:
```
# Dealing with PoolQC
df["PoolQC"].value_counts(dropna=False)
```

Out[20]:
```
PoolQC
NaN    2909
Ex        4
Gd        4
Fa        2
Name: count, dtype: int64
```

In [21]:
```
df["PoolQC"] = df["PoolQC"].fillna("None")
df["PoolQC"].value_counts(dropna=False)
```

Out[21]:
```
PoolQC
None    2909
Ex         4
Gd         4
Fa         2
Name: count, dtype: int64
```

In [ ]:

In [22]:
```
# Dealing with MiscFeature
df["MiscFeature"].value_counts(dropna=False)
```

Out[22]:
```
MiscFeature
NaN     2814
Shed      95
Gar2       5
Othr       4
TenC       1
Name: count, dtype: int64
```

In [23]:
```
df["MiscFeature"] = df["MiscFeature"].fillna("None")
df["MiscFeature"].value_counts(dropna=False)
```

Out[23]:
```
MiscFeature
None    2814
Shed      95
Gar2       5
Othr       4
TenC       1
Name: count, dtype: int64
```

In [ ]:

In [24]:
```
# Dealing with Alley
df["Alley"].value_counts(dropna=False)
```

Out[24]:
```
Alley
NaN     2721
Grvl     120
Pave      78
Name: count, dtype: int64
```

In [25]:
```
df["Alley"] = df["Alley"].fillna("None")
df["Alley"].value_counts(dropna=False)
```

Out[25]:
```
Alley
None    2721
Grvl     120
Pave      78
Name: count, dtype: int64
```

In [ ]:

In [26]:
```
# Dealing with Fence
df["Fence"].value_counts(dropna=False)
```

Out[26]:
```
Fence
NaN      2348
MnPrv     329
GdPrv     118
GdWo      112
MnWw       12
Name: count, dtype: int64
```

In [27]:
```
df["Fence"] = df["Fence"].fillna("None")
df["Fence"].value_counts(dropna=False)
```

Out[27]:
```
Fence
None     2348
MnPrv     329
GdPrv     118
GdWo      112
MnWw       12
Name: count, dtype: int64
```

In [ ]:

In [28]:
```
# Dealing with MasVnrType
df["MasVnrType"].value_counts(dropna=False)
```

Out[28]:
```
MasVnrType
NaN        1766
BrkFace     879
Stone       249
BrkCmn       25
Name: count, dtype: int64
```

In [29]:
```
df["MasVnrType"] = df["MasVnrType"].fillna("None")
df["MasVnrType"].value_counts(dropna=False)
```

Out[29]:
```
MasVnrType
None       1766
BrkFace     879
Stone       249
BrkCmn       25
Name: count, dtype: int64
```

In [ ]:

In [ ]:

In [30]:
```
# Dealing with FireplaceQu
df["FireplaceQu"].value_counts(dropna=False)
```

Out[30]:
```
FireplaceQu
NaN    1420
Gd      744
TA      592
Fa       74
Po       46
Ex       43
Name: count, dtype: int64
```

In [31]:
```
df["FireplaceQu"] = df["FireplaceQu"].fillna("None")
df["FireplaceQu"].value_counts(dropna=False)
```

Out[31]:
```
FireplaceQu
None    1420
Gd       744
TA       592
Fa        74
Po        46
Ex        43
Name: count, dtype: int64
```

In [ ]:

In [32]:
```
# Dealing with LotFrontage
df["LotFrontage"].value_counts(dropna=False)
```

Out[32]:
```
LotFrontage
NaN      486
60.0     276
80.0     137
70.0     133
50.0     117
        ...
111.0      1
138.0      1
182.0      1
168.0      1
133.0      1
Name: count, Length: 129, dtype: int64
```

In [33]:
```
df["LotFrontage"] = df["LotFrontage"].fillna(df_train["LotFrontage"].medi
df["LotFrontage"].value_counts(dropna=False)
```

Out[33]:
```
LotFrontage
69.0     506
60.0     276
80.0     137
70.0     133
50.0     117
        ...
152.0      1
138.0      1
182.0      1
168.0      1
133.0      1
Name: count, Length: 128, dtype: int64
```

In [ ]:

In [34]:
```
# Dealing with GarageFinish
df["GarageFinish"].value_counts(dropna=False)
```

Out[34]:
```
GarageFinish
Unf    1230
RFn     811
Fin     719
NaN     159
Name: count, dtype: int64
```

In [35]:
```
df["GarageFinish"] = df["GarageFinish"].fillna("None")
df["GarageFinish"].value_counts(dropna=False)
```

Out[35]:
```
GarageFinish
Unf     1230
RFn      811
Fin      719
None     159
Name: count, dtype: int64
```

In [ ]:

In [36]:
```
# Dealing with GarageQual
df["GarageQual"].value_counts(dropna=False)
```

Out[36]:
```
GarageQual
TA      2604
NaN      159
Fa       124
Gd        24
Po         5
Ex         3
Name: count, dtype: int64
```

In [37]:
```
df["GarageQual"] = df["GarageQual"].fillna("None")
df["GarageQual"].value_counts(dropna=False)
```

Out[37]:
```
GarageQual
TA      2604
None     159
Fa       124
Gd        24
Po         5
Ex         3
Name: count, dtype: int64
```

In [ ]:

In [38]:
```
# Dealing with GarageCond
df["GarageCond"].value_counts(dropna=False)
```

Out[38]:
```
GarageCond
TA      2654
NaN      159
Fa        74
Gd        15
Po        14
Ex         3
Name: count, dtype: int64
```

In [39]:
```
df["GarageCond"] = df["GarageCond"].fillna("None")
df["GarageCond"].value_counts(dropna=False)
```

Out[39]:
```
GarageCond
TA      2654
None     159
Fa        74
Gd        15
Po        14
Ex         3
Name: count, dtype: int64
```

In [ ]:

In [40]:
```
# Dealing with GarageYrBlt
df["GarageYrBlt"].value_counts(dropna=False)
```

Out[40]:
```
GarageYrBlt
NaN       159
2005.0    142
2007.0    115
2006.0    115
2004.0     99
         ...
1908.0      1
1896.0      1
1933.0      1
1906.0      1
1919.0      1
Name: count, Length: 104, dtype: int64
```

In [41]:
```
df["GarageYrBlt"] = df["GarageYrBlt"].fillna(df_train["GarageYrBlt"].medi
df["GarageYrBlt"].value_counts(dropna=False)
```

Out[41]:
```
GarageYrBlt
1980.0    191
2005.0    142
2006.0    115
2007.0    115
2004.0     99
         ...
1896.0      1
1895.0      1
2207.0      1
1943.0      1
1919.0      1
Name: count, Length: 103, dtype: int64
```

In [42]:
```
#replace error
df["GarageYrBlt"].min(), df["GarageYrBlt"].max()
```

Out[42]:
```
(1895.0, 2207.0)
```

In [43]:
```
df['GarageYrBlt'] = df['GarageYrBlt'].replace(2207, 2007)
```

In [ ]:

In [44]:
```
# Dealing with GarageType
df["GarageType"].value_counts(dropna=False)
```

Out[44]:
```
GarageType
Attchd     1723
Detchd      779
BuiltIn     186
NaN         157
Basment      36
2Types       23
CarPort      15
Name: count, dtype: int64
```

In [45]:
```
df["GarageType"] = df["GarageType"].fillna("None")
df["GarageType"].value_counts(dropna=False)
```

Out[45]:
```
GarageType
Attchd     1723
Detchd      779
BuiltIn     186
None        157
Basment      36
2Types       23
CarPort      15
Name: count, dtype: int64
```

In [ ]:

In [46]:
```
# Dealing with BsmtCond
df["BsmtCond"].value_counts(dropna=False)
```

Out[46]:
```
BsmtCond
TA     2606
Gd      122
Fa      104
NaN      82
Po        5
Name: count, dtype: int64
```

In [47]:
```
df["BsmtCond"] = df["BsmtCond"].fillna("None")
df["BsmtCond"].value_counts(dropna=False)
```

Out[47]:
```
BsmtCond
TA      2606
Gd       122
Fa       104
None      82
Po         5
Name: count, dtype: int64
```

In [ ]:

In [ ]:

In [48]:
```
# Dealing with BsmtExposure
df["BsmtExposure"].value_counts(dropna=False)
```

Out[48]:
```
BsmtExposure
No     1904
Av      418
Gd      276
Mn      239
NaN      82
Name: count, dtype: int64
```

In [49]:
```
df["BsmtExposure"] = df["BsmtExposure"].fillna("None")
df["BsmtExposure"].value_counts(dropna=False)
```

Out[49]:
```
BsmtExposure
No      1904
Av       418
Gd       276
Mn       239
None      82
Name: count, dtype: int64
```

In [ ]:

In [ ]:

In [50]:
```
# Dealing with BsmtQual
df["BsmtQual"].value_counts(dropna=False)
```

Out[50]:
```
BsmtQual
TA     1283
Gd     1209
Ex      258
Fa       88
NaN      81
Name: count, dtype: int64
```

In [51]:
```
df["BsmtQual"] = df["BsmtQual"].fillna("None")
df["BsmtQual"].value_counts(dropna=False)
```

Out[51]:
```
BsmtQual
TA      1283
Gd      1209
Ex       258
Fa        88
None      81
Name: count, dtype: int64
```

In [ ]:

In [52]:
```
# Dealing with BsmtFinType2
df["BsmtFinType2"].value_counts(dropna=False)
```

Out[52]:
```
BsmtFinType2
Unf    2493
Rec     105
LwQ      87
NaN      80
BLQ      68
ALQ      52
GLQ      34
Name: count, dtype: int64
```

In [53]:
```
df["BsmtFinType2"] = df["BsmtFinType2"].fillna("None")
df["BsmtFinType2"].value_counts(dropna=False)
```

Out[53]:
```
BsmtFinType2
Unf     2493
Rec      105
LwQ       87
None      80
BLQ       68
ALQ       52
GLQ       34
Name: count, dtype: int64
```

In [ ]:

In [54]:
```
# Dealing with BsmtFinType1
df["BsmtFinType1"].value_counts(dropna=False)
```

Out[54]:
```
BsmtFinType1
Unf    851
GLQ    849
ALQ    429
Rec    288
BLQ    269
LwQ    154
NaN     79
Name: count, dtype: int64
```

In [55]:
```
df["BsmtFinType1"] = df["BsmtFinType1"].fillna("None")
df["BsmtFinType1"].value_counts(dropna=False)
```

Out[55]:
```
BsmtFinType1
Unf     851
GLQ     849
ALQ     429
Rec     288
BLQ     269
LwQ     154
None     79
Name: count, dtype: int64
```

In [ ]:

In [56]:
```
# Dealing with MasVnrArea
df["MasVnrArea"].value_counts(dropna=False)
```

Out[56]:
```
MasVnrArea
0.0      1738
NaN        23
120.0      15
176.0      13
200.0      13
         ...
788.0       1
796.0       1
391.0       1
604.0       1
382.0       1
Name: count, Length: 445, dtype: int64
```

In [57]:
```
df["MasVnrArea"] = df["MasVnrArea"].fillna(0)
df["MasVnrArea"].value_counts(dropna=False)
```

Out[57]:
```
MasVnrArea
0.0      1761
120.0      15
176.0      13
200.0      13
216.0      12
         ...
137.0       1
274.0       1
415.0       1
760.0       1
382.0       1
Name: count, Length: 444, dtype: int64
```

In [ ]:

In [58]:
```
# Dealing with MSZoning
df["MSZoning"].value_counts(dropna=False)
```

Out[58]:
```
MSZoning
RL         2265
RM          460
FV          139
RH           26
C (all)      25
NaN           4
Name: count, dtype: int64
```

In [59]:
```
df["MSZoning"] = df["MSZoning"].fillna("None")
df["MSZoning"].value_counts(dropna=False)
```

Out[59]:
```
MSZoning
RL         2265
RM          460
FV          139
RH           26
C (all)      25
None          4
Name: count, dtype: int64
```

In [ ]:

In [60]:
```
# Dealing with BsmtHalfBath
df["BsmtHalfBath"].value_counts(dropna=False)
```

Out[60]:
```
BsmtHalfBath
0.0    2742
1.0     171
2.0       4
NaN       2
Name: count, dtype: int64
```

In [61]:
```
df["BsmtHalfBath"] = df["BsmtHalfBath"].fillna(0)
df["BsmtHalfBath"].value_counts()
```

Out[61]:
```
BsmtHalfBath
0.0    2744
1.0     171
2.0       4
Name: count, dtype: int64
```

In [62]:
```
# # If you do something wrong
# df["BsmtHalfBath"].replace("None", 0, inplace=True)
# df["BsmtHalfBath"].value_counts()
```

In [ ]:

In [63]:
```
# Dealing with Utilities
df["Utilities"].value_counts(dropna=False)
```

Out[63]:
```
Utilities
AllPub    2916
NaN          2
NoSeWa       1
Name: count, dtype: int64
```

In [64]:
```
df["Utilities"] = df["Utilities"].fillna("AllPub")
df["Utilities"].value_counts()
```

Out[64]:
```
Utilities
AllPub    2918
NoSeWa       1
Name: count, dtype: int64
```

In [65]:
```
# # If you do something wrong
# df["Utilities"].replace("None", "AllPub", inplace=True)
# df["Utilities"].value_counts()
```

In [ ]:

In [66]:
```
# Dealing with BsmtFullBath
df["BsmtFullBath"].value_counts(dropna=False)
```

Out[66]:
```
BsmtFullBath
0.0    1705
1.0    1172
2.0      38
3.0       2
NaN       2
Name: count, dtype: int64
```

In [67]:
```
df["BsmtFullBath"] = df["BsmtFullBath"].fillna(0)
df["BsmtFullBath"].value_counts()
```

Out[67]:
```
BsmtFullBath
0.0    1707
1.0    1172
2.0      38
3.0       2
Name: count, dtype: int64
```

In [ ]:

In [68]:
```
# Dealing with Functional
df["Functional"].value_counts(dropna=False)
```

Out[68]:
```
Functional
Typ     2717
Min2      70
Min1      65
Mod       35
Maj1      19
Maj2       9
Sev        2
NaN        2
Name: count, dtype: int64
```

In [69]:
```
df["Functional"] = df["Functional"].fillna("Typ")
df["Functional"].value_counts(dropna=False)
```

Out[69]:
```
Functional
Typ     2719
Min2      70
Min1      65
Mod       35
Maj1      19
Maj2       9
Sev        2
Name: count, dtype: int64
```

In [ ]:

In [70]:
```
# Dealing with Exterior2nd
df["Exterior2nd"].value_counts(dropna=False)
```

Out[70]:
```
Exterior2nd
VinylSd    1014
MetalSd     447
HdBoard     406
Wd Sdng     391
Plywood     270
CmentBd     126
Wd Shng      81
Stucco       47
BrkFace      47
AsbShng      38
Brk Cmn      22
ImStucc      15
Stone         6
AsphShn       4
CBlock        3
Other         1
NaN           1
Name: count, dtype: int64
```

In [71]:
```
df["Exterior2nd"] = df["Exterior2nd"].fillna("None")
df["Exterior2nd"].value_counts(dropna=False)
```

Out[71]:
```
Exterior2nd
VinylSd    1014
MetalSd     447
HdBoard     406
Wd Sdng     391
Plywood     270
CmentBd     126
Wd Shng      81
Stucco       47
BrkFace      47
AsbShng      38
Brk Cmn      22
ImStucc      15
Stone         6
AsphShn       4
CBlock        3
Other         1
None          1
Name: count, dtype: int64
```

In [ ]:

In [72]:
```
# Dealing with Exterior1st
df["Exterior1st"].value_counts(dropna=False)
```

Out[72]:
```
Exterior1st
VinylSd    1025
MetalSd     450
HdBoard     442
Wd Sdng     411
Plywood     221
CemntBd     126
BrkFace      87
WdShing      56
AsbShng      44
Stucco       43
BrkComm       6
AsphShn       2
Stone         2
CBlock        2
ImStucc       1
NaN           1
Name: count, dtype: int64
```

In [73]:
```
df["Exterior1st"] = df["Exterior1st"].fillna("None")
df["Exterior1st"].value_counts(dropna=False)
```

Out[73]:
```
Exterior1st
VinylSd    1025
MetalSd     450
HdBoard     442
Wd Sdng     411
Plywood     221
CemntBd     126
BrkFace      87
WdShing      56
AsbShng      44
Stucco       43
BrkComm       6
AsphShn       2
Stone         2
CBlock        2
ImStucc       1
None          1
Name: count, dtype: int64
```

In [ ]:

In [74]:
```
# Dealing with GarageArea
df["GarageArea"].value_counts(dropna=False)
```

Out[74]:
```
GarageArea
0.0       157
576.0      97
440.0      96
240.0      69
484.0      68
          ...
459.0       1
954.0       1
1014.0      1
753.0       1
272.0       1
Name: count, Length: 604, dtype: int64
```

In [75]:
```
df["GarageArea"] = df["GarageArea"].fillna(0)
df["GarageArea"].value_counts(dropna=False)
```

Out[75]:
```
GarageArea
0.0       158
576.0      97
440.0      96
240.0      69
484.0      68
          ...
872.0       1
923.0       1
192.0       1
1025.0      1
272.0       1
Name: count, Length: 603, dtype: int64
```

In [ ]:

In [76]:
```
# Dealing with GarageCars
df["GarageCars"].value_counts(dropna=False)
```

Out[76]:
```
GarageCars
2.0    1594
1.0     776
3.0     374
0.0     157
4.0      16
5.0       1
NaN       1
Name: count, dtype: int64
```

In [77]:
```
df["GarageCars"] = df["GarageCars"].fillna(0)
df["GarageCars"].value_counts(dropna=False)
```

Out[77]:
```
GarageCars
2.0    1594
1.0     776
3.0     374
0.0     158
4.0      16
5.0       1
Name: count, dtype: int64
```

In [ ]:

In [78]:
```
# Dealing with SaleType
df["SaleType"].value_counts(dropna=False)
```

Out[78]:
```
SaleType
WD       2525
New       239
COD        87
ConLD      26
CWD        12
ConLI       9
ConLw       8
Oth         7
Con         5
NaN         1
Name: count, dtype: int64
```

In [79]:
```
df["SaleType"] = df["SaleType"].fillna("None")
df["SaleType"].value_counts(dropna=False)
```

Out[79]:
```
SaleType
WD       2525
New       239
COD        87
ConLD      26
CWD        12
ConLI       9
ConLw       8
Oth         7
Con         5
None        1
Name: count, dtype: int64
```

In [ ]:

In [80]:
```
# Dealing with KitchenQual
df["KitchenQual"].value_counts(dropna=False)
```

Out[80]:
```
KitchenQual
TA     1492
Gd     1151
Ex      205
Fa       70
NaN       1
Name: count, dtype: int64
```

In [81]:
```
df["KitchenQual"] = df["KitchenQual"].fillna("TA")
df["KitchenQual"].value_counts(dropna=False)
```

Out[81]:
```
KitchenQual
TA    1493
Gd    1151
Ex     205
Fa      70
Name: count, dtype: int64
```

In [ ]:

In [82]:
```
# Dealing with BsmtFinSF1
df["BsmtFinSF1"].value_counts(dropna=False)
```

Out[82]:
```
BsmtFinSF1
0.0       929
24.0       27
16.0       14
300.0       9
20.0        8
          ...
383.0       1
1085.0      1
900.0       1
1022.0      1
337.0       1
Name: count, Length: 992, dtype: int64
```

In [83]:
```
df["BsmtFinSF1"] = df["BsmtFinSF1"].fillna(0)
df["BsmtFinSF1"].value_counts(dropna=False)
```

Out[83]:
```
BsmtFinSF1
0.0       930
24.0       27
16.0       14
300.0       9
288.0       8
          ...
383.0       1
1085.0      1
900.0       1
1022.0      1
337.0       1
Name: count, Length: 991, dtype: int64
```

In [ ]:

In [84]:
```
# Dealing with Electrical
df["Electrical"].value_counts(dropna=False)
```

Out[84]:
```
Electrical
SBrkr    2671
FuseA     188
FuseF      50
FuseP       8
Mix         1
NaN         1
Name: count, dtype: int64
```

In [85]:
```
df["Electrical"] = df["Electrical"].fillna("SBrkr")
df["Electrical"].value_counts(dropna=False)
```

Out[85]:
```
Electrical
SBrkr    2672
FuseA     188
FuseF      50
FuseP       8
Mix         1
Name: count, dtype: int64
```

In [ ]:

In [86]:
```
# Dealing with BsmtFinSF2
```

```
df["BsmtFinSF2"].value_counts(dropna=False)
```

Out[86]:
```
BsmtFinSF2
0.0       2571
180.0        5
294.0        5
483.0        3
168.0        3
          ...
600.0        1
211.0        1
1031.0       1
438.0        1
297.0        1
Name: count, Length: 273, dtype: int64
```

In [87]:
```
df["BsmtFinSF2"] = df["BsmtFinSF2"].fillna(0)
df["BsmtFinSF2"].value_counts(dropna=False)
```

Out[87]:
```
BsmtFinSF2
0.0       2572
180.0        5
294.0        5
435.0        3
483.0        3
          ...
600.0        1
211.0        1
1031.0       1
438.0        1
297.0        1
Name: count, Length: 272, dtype: int64
```

In [ ]:

In [88]:
```
# Dealing with BsmtUnfSF
df["BsmtUnfSF"].value_counts(dropna=False)
```

Out[88]:
```
BsmtUnfSF
0.0       241
384.0      19
728.0      14
672.0      13
600.0      12
          ...
574.0       1
1502.0      1
568.0       1
26.0        1
1503.0      1
Name: count, Length: 1136, dtype: int64
```

In [89]:
```
df["BsmtUnfSF"] = df["BsmtUnfSF"].fillna(0)
df["BsmtUnfSF"].value_counts(dropna=False)
```

Out[89]:
```
BsmtUnfSF
0.0       242
384.0      19
728.0      14
672.0      13
600.0      12
          ...
257.0       1
1217.0      1
539.0       1
39.0        1
1503.0      1
Name: count, Length: 1135, dtype: int64
```

In [ ]:

In [90]:
```
# Dealing with TotalBsmtSF
df["TotalBsmtSF"].value_counts(dropna=False)
```

Out[90]:
```
TotalBsmtSF
0.0       78
864.0     74
672.0     29
912.0     26
1040.0    25
          ..
1571.0     1
2633.0     1
757.0      1
873.0      1
1381.0     1
Name: count, Length: 1059, dtype: int64
```

In [91]:
```
df["TotalBsmtSF"] = df["TotalBsmtSF"].fillna(0)
df["TotalBsmtSF"].value_counts(dropna=False)
```

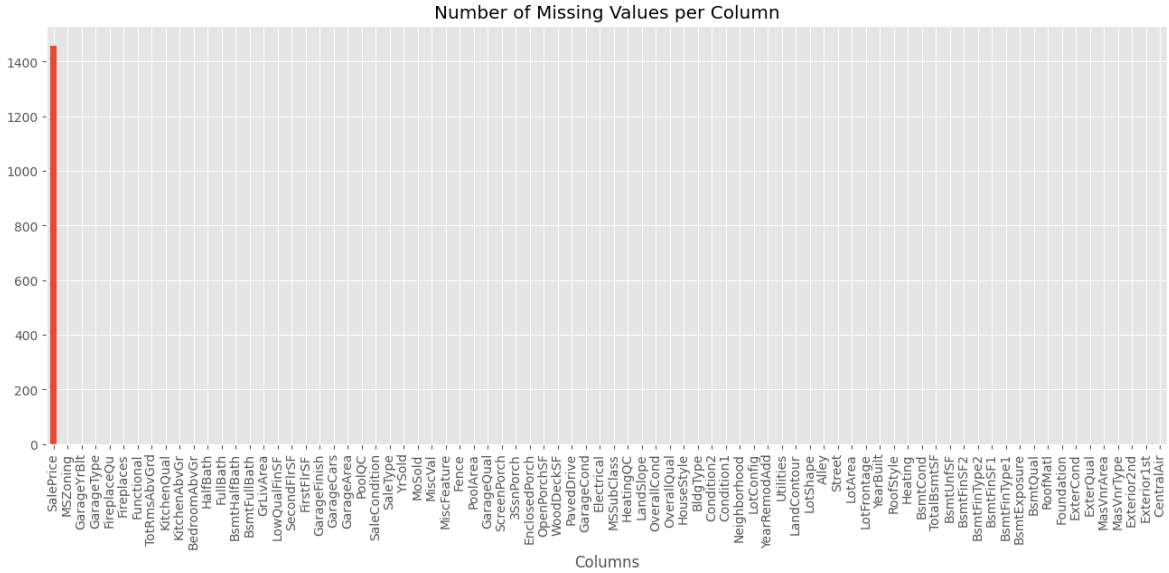
Out[91]:
```
TotalBsmtSF
0.0       79
864.0     74
672.0     29
912.0     26
1040.0    25
          ..
1571.0     1
2633.0     1
757.0      1
873.0      1
1381.0     1
Name: count, Length: 1058, dtype: int64
```

Outcome

In [92]:
```
df.isna().sum().sort_values(ascending=False).head(36)
```

Out[92]:
```
SalePrice        1459
MSZoning            0
GarageYrBlt         0
GarageType          0
FireplaceQu         0
Fireplaces          0
Functional          0
TotRmsAbvGrd        0
KitchenQual         0
KitchenAbvGr        0
BedroomAbvGr        0
HalfBath            0
FullBath            0
BsmtHalfBath        0
BsmtFullBath        0
GrLivArea           0
LowQualFinSF        0
SecondFlrSF         0
FirstFlrSF          0
GarageFinish        0
GarageCars          0
GarageArea          0
PoolQC              0
SaleCondition       0
SaleType            0
YrSold              0
MoSold              0
MiscVal             0
MiscFeature         0
Fence               0
PoolArea            0
GarageQual          0
ScreenPorch         0
3SsnPorch           0
EnclosedPorch       0
OpenPorchSF         0
dtype: int64
```

In [93]:
```
df.isna().sum().sort_values(ascending=False).plot(kind='bar', figsize=(16
plt.title('Number of Missing Values per Column')
plt.xlabel('Columns')
plt.show()
```



Change datatypes

In [94]:
```
df
```

Out[94]:

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandCor
0	60	RL	65.0	8450	Pave	None	Reg	
1	20	RL	80.0	9600	Pave	None	Reg	
2	60	RL	68.0	11250	Pave	None	IR1	
3	70	RL	60.0	9550	Pave	None	IR1	
4	60	RL	84.0	14260	Pave	None	IR1	
...	...	...	...	...	...	...	...	
2914	160	RM	21.0	1936	Pave	None	Reg	
2915	160	RM	21.0	1894	Pave	None	Reg	
2916	20	RL	160.0	20000	Pave	None	Reg	
2917	85	RL	62.0	10441	Pave	None	Reg	
2918	60	RL	74.0	9627	Pave	None	Reg	

2919 rows × 80 columns

In [95]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2919 entries, 0 to 2918
Data columns (total 80 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   MSSubClass     2919 non-null   int64
 1   MSZoning       2919 non-null   object
 2   LotFrontage    2919 non-null   float64
 3   LotArea        2919 non-null   int64
 4   Street         2919 non-null   object
 5   Alley          2919 non-null   object
 6   LotShape       2919 non-null   object
 7   LandContour    2919 non-null   object
 8   Utilities      2919 non-null   object
 9   LotConfig      2919 non-null   object
 10  LandSlope      2919 non-null   object
 11  Neighborhood   2919 non-null   object
 12  Condition1     2919 non-null   object
 13  Condition2     2919 non-null   object
 14  BldgType       2919 non-null   object
 15  HouseStyle     2919 non-null   object
 16  OverallQual    2919 non-null   int64
 17  OverallCond    2919 non-null   int64
 18  YearBuilt      2919 non-null   int64
 19  YearRemodAdd   2919 non-null   int64
 20  RoofStyle      2919 non-null   object
 21  RoofMatl       2919 non-null   object
 22  Exterior1st    2919 non-null   object
 23  Exterior2nd    2919 non-null   object
 24  MasVnrType     2919 non-null   object
 25  MasVnrArea     2919 non-null   float64
 26  ExterQual      2919 non-null   object
 27  ExterCond      2919 non-null   object
 28  Foundation     2919 non-null   object
 29  BsmtQual       2919 non-null   object
 30  BsmtCond       2919 non-null   object
 31  BsmtExposure   2919 non-null   object
 32  BsmtFinType1   2919 non-null   object
 33  BsmtFinSF1     2919 non-null   float64
 34  BsmtFinType2   2919 non-null   object
 35  BsmtFinSF2     2919 non-null   float64
 36  BsmtUnfSF      2919 non-null   float64
 37  TotalBsmtSF    2919 non-null   float64
 38  Heating        2919 non-null   object
 39  HeatingQC      2919 non-null   object
 40  CentralAir     2919 non-null   object
 41  Electrical     2919 non-null   object
 42  FirstFlrSF     2919 non-null   int64
 43  SecondFlrSF    2919 non-null   int64
 44  LowQualFinSF   2919 non-null   int64
 45  GrLivArea      2919 non-null   int64
 46  BsmtFullBath   2919 non-null   float64
 47  BsmtHalfBath   2919 non-null   float64
 48  FullBath       2919 non-null   int64
 49  HalfBath       2919 non-null   int64
 50  BedroomAbvGr   2919 non-null   int64
 51  KitchenAbvGr   2919 non-null   int64
 52  KitchenQual    2919 non-null   object
 53  TotRmsAbvGrd   2919 non-null   int64
 54  Functional     2919 non-null   object
```

```
 55  Fireplaces     2919 non-null   int64
 56  FireplaceQu    2919 non-null   object
 57  GarageType     2919 non-null   object
 58  GarageYrBlt    2919 non-null   float64
 59  GarageFinish   2919 non-null   object
 60  GarageCars     2919 non-null   float64
 61  GarageArea     2919 non-null   float64
 62  GarageQual     2919 non-null   object
 63  GarageCond     2919 non-null   object
 64  PavedDrive     2919 non-null   object
 65  WoodDeckSF     2919 non-null   int64
 66  OpenPorchSF    2919 non-null   int64
 67  EnclosedPorch  2919 non-null   int64
 68  3SsnPorch      2919 non-null   int64
 69  ScreenPorch    2919 non-null   int64
 70  PoolArea       2919 non-null   int64
 71  PoolQC         2919 non-null   object
 72  Fence          2919 non-null   object
 73  MiscFeature    2919 non-null   object
 74  MiscVal        2919 non-null   int64
 75  MoSold         2919 non-null   int64
 76  YrSold         2919 non-null   int64
 77  SaleType       2919 non-null   object
 78  SaleCondition  2919 non-null   object
 79  SalePrice      1460 non-null   float64
dtypes: float64(12), int64(25), object(43)
memory usage: 1.8+ MB
```

In [96]:
```
# Changing floats that are actually ints to ints

df['BsmtFullBath'] = df['BsmtFullBath'].astype(int)
df['BsmtHalfBath'] = df['BsmtHalfBath'].astype(int)
df['GarageYrBlt'] = df['GarageYrBlt'].astype(int)
df['GarageCars'] = df['GarageCars'].astype(int)
```

Feature engineering

Creating new features

from the existing ones

In [97]:
```
# Features that have years in them
df['GarageYrBlt'] = df['YrSold'] - df['GarageYrBlt']
df['YearBuilt'] = df['YrSold'] - df['YearBuilt']
df['YearRemodAdd'] = df['YrSold'] - df['YearRemodAdd']

df.drop(['YrSold'], axis=1, inplace=True)
df.drop(['MoSold'], axis=1, inplace=True) #Isn't necessary
```

In [98]:
```
# Features that have square feet
df['BsmtFinSF'] = df['BsmtFinSF1'] + df['BsmtFinSF2']
df['TotalFlrSF'] = df['FirstFlrSF'] + df['SecondFlrSF']

df.drop(['BsmtFinSF1'], axis=1, inplace=True)
df.drop(['BsmtFinSF2'], axis=1, inplace=True)
df.drop(['FirstFlrSF'], axis=1, inplace=True)
```

```
df.drop(['SecondFlrSF'], axis=1, inplace=True)
df.drop(['TotalBsmtSF'], axis=1, inplace=True) # Isn't necessary since it
```

In [99]:
```
# Features that are about bathrooms
df['TotalBaths'] = df['FullBath'] + (0.5*df['HalfBath']) + df['BsmtFullBa

df.drop(['FullBath'], axis=1, inplace=True)
df.drop(['HalfBath'], axis=1, inplace=True)
df.drop(['BsmtFullBath'], axis=1, inplace=True)
df.drop(['BsmtHalfBath'], axis=1, inplace=True)
```

In [100…
```
# Features about the garage
df['GarageAreaPerCar'] = df['GarageArea'] / df['GarageCars']

# Fill in any nulls from feature engineering
df['GarageAreaPerCar'].fillna(0, inplace=True)

df.drop(['GarageArea'], axis=1, inplace=True)
df.drop(['GarageCars'], axis=1, inplace=True)
```

Skewness of the columns:

NOT NEEDED IF USING TREE-BASED MODELS, BUT GOOD TO KNOW

In [101…
```
# Create a list of the numerical data types after the feature engineering
numerical_cols = [cname for cname in df.columns if df[cname].dtypes!='obj

# Create new dataframe with the numerical columns
skew_df = pd.DataFrame(numerical_cols, columns=['Feature'])

# This finction used to compute the skewness of a dataset
skew_df['Skew'] = skew_df['Feature'].apply(lambda feature: scipy.stats.sk

# Change Skew to a postive number
skew_df['Absolute Skew'] = skew_df['Skew'].apply(abs)

# Create true/false columns based on if Absolute Skew is >=0.5
skew_df['Skewed'] = skew_df['Absolute Skew'].apply(lambda x: True if x >=

skew_df
```

Out[101…

	Feature	Skew	Absolute Skew	Skewed
0	MSSubClass	1.375457	1.375457	True
1	LotFrontage	1.652671	1.652671	True
2	LotArea	12.822431	12.822431	True
3	OverallQual	0.197110	0.197110	False
4	OverallCond	0.570312	0.570312	True
5	YearBuilt	0.598609	0.598609	True
6	YearRemodAdd	0.450226	0.450226	False
7	MasVnrArea	2.613592	2.613592	True
8	BsmtUnfSF	0.919339	0.919339	True
9	LowQualFinSF	12.088761	12.088761	True
10	GrLivArea	1.269358	1.269358	True
11	BedroomAbvGr	0.326324	0.326324	False
12	KitchenAbvGr	4.302254	4.302254	True
13	TotRmsAbvGrd	0.758367	0.758367	True
14	Fireplaces	0.733495	0.733495	True
15	GarageYrBlt	0.688345	0.688345	True
16	WoodDeckSF	1.842433	1.842433	True
17	OpenPorchSF	2.535114	2.535114	True
18	EnclosedPorch	4.003891	4.003891	True
19	3SsnPorch	11.376065	11.376065	True
20	ScreenPorch	3.946694	3.946694	True
21	PoolArea	16.898328	16.898328	True
22	MiscVal	21.947195	21.947195	True
23	BsmtFinSF	1.178646	1.178646	True
24	TotalFlrSF	1.252036	1.252036	True
25	TotalBaths	0.492247	0.492247	False
26	GarageAreaPerCar	-0.342971	0.342971	False

In [102…

```
if False:
    # Apply log1p transformation to df whose above a 0.5 absolute skew
    for column in skew_df.query("Skewed == True")['Feature'].values:
        df[column] = np.log1p(df[column])
```

Deal with skew in the target variable

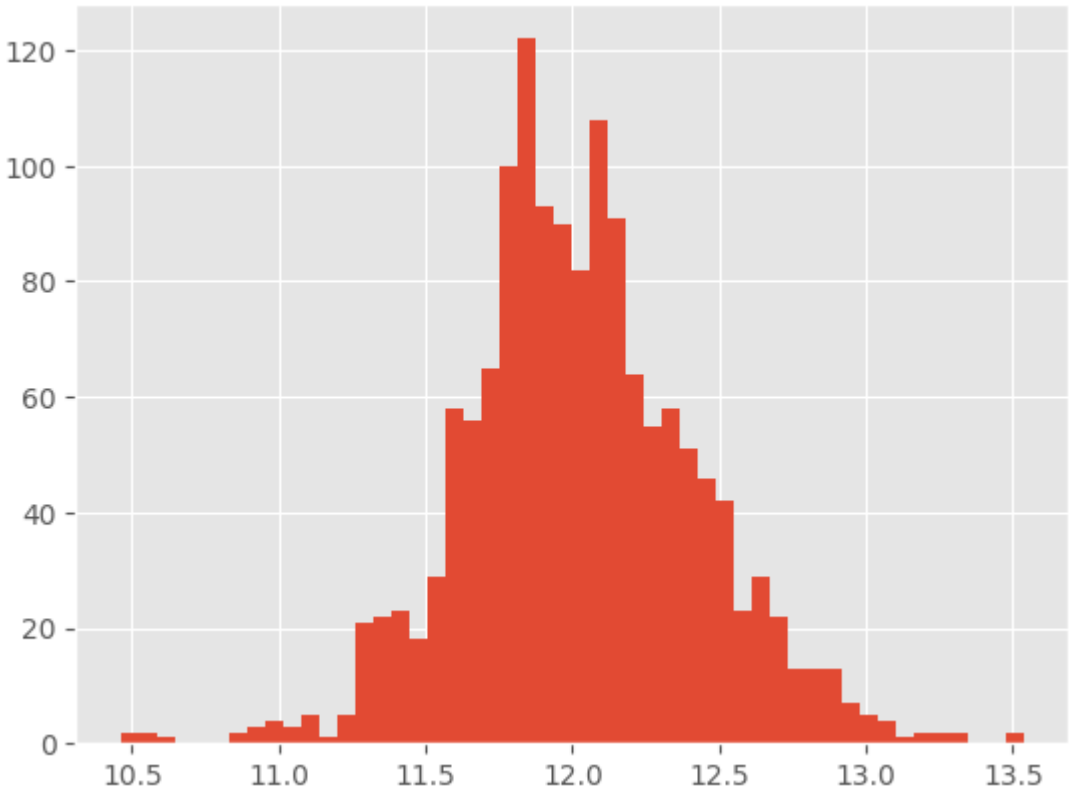
In [103…

```
df["LogSalePrice"] = np.log(df["SalePrice"])

df = df.drop(['SalePrice'], axis=1)
```

In [104…
```
print(df["LogSalePrice"].value_counts(dropna=False))
df["LogSalePrice"].hist(bins=50)
```

```
LogSalePrice
NaN          1459
11.849398      20
11.813030      17
11.884489      14
11.951180      14
             ...
12.219310       1
12.013095       1
12.109011       1
12.574182       1
12.246735       1
Name: count, Length: 664, dtype: int64
```

Out[104…
```
<Axes: >
```



Encode categorical variables

In [105…
```
numerical_cols = df.select_dtypes(include=['int64', 'float64'],exclude=['
categorical_cols = df.select_dtypes(include='object').columns
numerical_cols, categorical_cols
```

Out[105…

```
(Index(['MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallC
ond',
        'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtUnfSF', 'LowQual
FinSF',
        'GrLivArea', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd',
        'Fireplaces', 'GarageYrBlt', 'WoodDeckSF', 'OpenPorchSF',
        'EnclosedPorch', '3SsnPorch', ' screenPorch', 'PoolArea', 'MiscVa
l',
        'BsmtFinSF', 'TotalFlrSF', 'TotalBaths', 'GarageAreaPerCar',
        'LogSalePrice'],
       dtype='object'),
 Index(['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utili
ties',
        'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Conditi
on2',
        'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1s
t',
        'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundati
on',
        'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFin
Type2',
        'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQua
l',
        'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'Gara
geQual',
        'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature',
        'SaleType', 'SaleCondition'],
       dtype='object'))
```

Ordinal encoding for ordered categories

In [106…

```
# Encoding ordered categorical variables

import pandas as pd

ordinal_1 = [
    'ExterQual', 'ExterCond', 'BsmtQual', 'BsmtCond', 'HeatingQC',
    'KitchenQual', 'FireplaceQu', 'GarageQual', 'GarageCond', 'PoolQC'
]

quality_map = {
    'None': 0,
    'Po': 1,
    'Fa': 2,
    'TA': 3,
    'Gd': 4,
    'Ex': 5
}

for col in ordinal_1:
    df[col] = df[col].map(quality_map).astype('int')
```

In [107…

```
# Ordered encode all the ordinal data individually, and add to list
ordinal_2 = ['LotShape', 'LandContour', 'LandSlope', 'BsmtExposure', 'Bsm
                 'Utilities', 'CentralAir', 'Functional', 'GarageFinish',

df['LotShape'] = df['LotShape'].map({"None":0,"IR3":1,"IR2":2,"IR1":3,"Re
df['LandContour'] = df['LandContour'].map({"None":0,"Low":1,"Bnk":2,"HLS"
```

```
df['LandSlope'] = df['LandSlope'].map({"None":0,"Sev":1,"Mod":2,"Gtl":3})
df['BsmtExposure'] = df['BsmtExposure'].map({"None":0, "No":1,"Mn":2,"Av"
df['BsmtFinType1'] = df['BsmtFinType1'].map({"None":0, "Unf":1,"LwQ":2,"R
df['BsmtFinType2'] = df['BsmtFinType2'].map({"None":0, "Unf":1,"LwQ":2,"R
df['Utilities'] = df['Utilities'].map({"None":0,"ELO":1,"NoSeWa":2,"NoSew
df['CentralAir'] = df['CentralAir'].map({"None":0,"N":1,"Y":2}).astype('i
df['Functional'] = df['Functional'].map({"None":0,"Sal":1,"Sev":2,"Maj2":
                                        "Min1":7,"Typ":8}).astype('int')
df['GarageFinish'] = df['GarageFinish'].map({"None":0,"Unf":1,"RFn":2,"Fi
df['PavedDrive'] = df['PavedDrive'].map({"None":0,"N":1,"P":2,"Y":3}).ast
df['Fence'] = df['Fence'].map({"None":0, "MnWw":1,"GdWo":2,"MnPrv":3,"GdP
```

Label encoding for non-ordered categories

In [108…

```
# For both xgboost and lightgbm you just need to use the category datatyp

labeled_cols = df.select_dtypes(include='object').columns

for col in labeled_cols:
    df[col] = df[col].astype('category')


temp = {}
for col in labeled_cols:
    temp[col] = df[col].value_counts()

temp
```

Out[108…

```
{'MSZoning': MSZoning
RL         2265
RM          460
FV          139
RH           26
C (all)      25
None          4
Name: count, dtype: int64,
'Street': Street
Pave    2907
Grvl      12
Name: count, dtype: int64,
'Alley': Alley
None    2721
Grvl     120
Pave      78
Name: count, dtype: int64,
'LotConfig': LotConfig
Inside     2133
Corner      511
CulDSac     176
FR2          85
FR3          14
Name: count, dtype: int64,
'Neighborhood': Neighborhood
NAmes      443
CollgCr    267
OldTown    239
Edwards    194
Somerst    182
NridgHt    166
Gilbert    165
Sawyer     151
NWAmes     131
SawyerW    125
Mitchel    114
BrkSide    108
Crawfor    103
IDOTRR      93
Timber      72
NoRidge     71
StoneBr     51
SWISU       48
ClearCr     44
MeadowV     37
BrDale      30
Blmngtn     28
Veenker     24
NPkVill     23
Blueste     10
Name: count, dtype: int64,
'Condition1': Condition1
Norm      2511
Feedr      164
Artery      92
RRAn        50
PosN        39
RRAe        28
PosA        20
RRNn         9
```

```
RRNe         6
Name: count, dtype: int64,
'Condition2': Condition2
Norm      2889
Feedr       13
Artery       5
PosA         4
PosN         4
RRNn         2
RRAe         1
RRAn         1
Name: count, dtype: int64,
'BldgType': BldgType
1Fam      2425
TwnhsE     227
Duplex     109
Twnhs       96
2fmCon      62
Name: count, dtype: int64,
'HouseStyle': HouseStyle
1Story    1471
2Story     872
1.5Fin     314
SLvl       128
SFoyer      83
2.5Unf      24
1.5Unf      19
2.5Fin       8
Name: count, dtype: int64,
'RoofStyle': RoofStyle
Gable      2310
Hip         551
Gambrel      22
Flat         20
Mansard      11
Shed          5
Name: count, dtype: int64,
'RoofMatl': RoofMatl
CompShg    2876
Tar&Grv      23
WdShake       9
WdShngl       7
ClyTile       1
Membran       1
Metal         1
Roll          1
Name: count, dtype: int64,
'Exterior1st': Exterior1st
VinylSd    1025
MetalSd     450
HdBoard     442
Wd Sdng     411
Plywood     221
CemntBd     126
BrkFace      87
WdShing      56
AsbShng      44
Stucco       43
BrkComm       6
AsphShn       2
```

```
CBlock        2
Stone         2
ImStucc       1
None          1
Name: count, dtype: int64,
'Exterior2nd': Exterior2nd
VinylSd    1014
MetalSd     447
HdBoard     406
Wd Sdng     391
Plywood     270
CmentBd     126
Wd Shng      81
Stucco       47
BrkFace      47
AsbShng      38
Brk Cmn      22
ImStucc      15
Stone         6
AsphShn       4
CBlock        3
None          1
Other         1
Name: count, dtype: int64,
'MasVnrType': MasVnrType
None       1766
BrkFace     879
Stone       249
BrkCmn       25
Name: count, dtype: int64,
'Foundation': Foundation
PConc     1308
CBlock    1235
BrkTil     311
Slab        49
Stone       11
Wood         5
Name: count, dtype: int64,
'Heating': Heating
GasA     2874
GasW       27
Grav        9
Wall        6
OthW        2
Floor       1
Name: count, dtype: int64,
'Electrical': Electrical
SBrkr    2672
FuseA     188
FuseF      50
FuseP       8
Mix         1
Name: count, dtype: int64,
'GarageType': GarageType
Attchd     1723
Detchd      779
BuiltIn     186
None        157
Basment      36
2Types       23
```

```
CarPort      15
Name: count, dtype: int64,
'MiscFeature': MiscFeature
None    2814
Shed      95
Gar2       5
Othr       4
TenC       1
Name: count, dtype: int64,
'SaleType': SaleType
WD       2525
New       239
COD        87
ConLD      26
CWD        12
ConLI       9
ConLw       8
Oth         7
Con         5
None        1
Name: count, dtype: int64,
'SaleCondition': SaleCondition
Normal     2402
Partial     245
Abnorml     190
Family       46
Alloca       24
AdjLand      12
Name: count, dtype: int64}
```

Target encoding with **Leave-one-out (LOO)** instead the non-ordered labels

Simple version for just one dataset

In [109…

```
if False:
    tempdf = df.copy()
    labeled_cols = df.select_dtypes(include=['object','category']).column


    def loo_target_encode(feature, target='SalePrice'):
        # 1. Calculate the sum and count per category
        grp = df.groupby(feature)[target]
        cat_sum = grp.transform('sum')
        cat_count = grp.transform('count')

        # 2. Apply the LOO formula: (Total Sum - Current Row Value) / (To
        loo_encoded = (cat_sum - df[target]) / (cat_count - 1)

        # 3. Handle the "Single Observation" case (division by zero)
        # Most common approach: Fill with the global mean of the target
        global_mean = df[target].mean()
        loo_encoded = loo_encoded.fillna(global_mean)

        return loo_encoded

    for col in labeled_cols:
        tempdf[col] = loo_target_encode(col).astype('float')
```

```
    tempdf[labeled_cols]
```

More complicated version for when the inference dataset (with Nan targets) is merget into the same dataframe

In [110…

```
if False:
    labeled_cols = df.select_dtypes(include=['object','category']).column
    labeled_cols

    tempdf = df.copy()

    def loo_target_encode(feature, target='LogSalePrice'):

        train_mask = df[target].notnull()
        train_df = df[train_mask]

        # 2. Calculate training-only stats
        # We use a dictionary or a series to map these to the main datafr
        cat_stats = train_df.groupby(feature)[target].agg(['sum', 'count'
        global_sum = train_df[target].sum()
        global_count = len(train_df)
        global_mean = global_sum / global_count

        # 3. Map these training stats to the entire dataframe (including
        # This creates temporary helper columns
        df_sum = df[feature].map(cat_stats['sum']).astype(float)  # float
        df_count = df[feature].map(cat_stats['count']).astype(float)  # f

        # 4. Initialize the output column
        # We'll use the "Standard Mean" for everything first
        encoded = df_sum / df_count

        # 5. Apply LOO Logic specifically to Training rows
        # Formula: (Sum - Current Y) / (Count - 1)
        train_loo = (df_sum[train_mask] - df[target][train_mask]) / (df_c
        encoded.update(train_loo)

        # 6. Handle Singletons and New Categories in inference
        # If a category in inference wasn't in Train, df_count is NaN.
        # If a category in Train had count 1, train_loo is inf/NaN.

        # Calculate LOO Global Mean for Training singletons
        loo_global_mean = (global_sum - df[target][train_mask]) / (global_

        # Fill gaps:
        # A) Training rows with 1 observation -> LOO Global Mean
        encoded.loc[train_mask] = encoded.loc[train_mask].fillna(loo_glob

        # B) inference rows with categories not seen in training -> Globa
        encoded = encoded.fillna(global_mean)

        return encoded



    for col in labeled_cols:
        tempdf[col] = loo_target_encode(col).astype('float')
```

```
    tempdf
```

In [111…

```
target_means = df.groupby("SaleCondition")['LogSalePrice'].mean()

target_means
```

Out[111…

```
SaleCondition
Abnorml    11.788774
AdjLand    11.529147
Alloca     11.914458
Family     11.869655
Normal     12.005034
Partial    12.447477
Name: LogSalePrice, dtype: float64
```

One-hot encoding for linear regression and other models

In [112…

```
labeled_cols = df.select_dtypes(include=['object','category']).columns
labeled_cols
```

Out[112…

```
Index(['MSZoning', 'Street', 'Alley', 'LotConfig', 'Neighborhood',
       'Condition1', 'Condition2', 'BldgType', 'HouseStyle', 'RoofStyl
e',
       'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'Foundati
on',
       'Heating', 'Electrical', 'GarageType', 'MiscFeature', 'SaleType',
       'SaleCondition'],
      dtype='object')
```

In [113…

```
one_hot_columns = labeled_cols
df = pd.get_dummies(df, columns=one_hot_columns, prefix=one_hot_columns,
```

In [114…

```
if False:
    dftemp = df.copy()
    one_hot_columns = ['Alley']

    dftemp = pd.get_dummies(dftemp, columns=one_hot_columns, prefix=one_h
    dftemp
```

Train Model

Split df_train and into df_validate df_test for model

In [115…
```
# Typical train/test split
if False:
    columns = [c for c in df.columns if c not in ['targe']]

    X = df[columns]
    y = df['target']
    X_validate, X_test, y_validate, y_test = train_test_split(X, y, test_

    X_validate
```

In [116…
```
df_train = df.iloc[:df_train.shape[0]]
df_inference = df.iloc[df_train.shape[0]:]

log_target = df_train["LogSalePrice"]

df_train = df_train.drop(['LogSalePrice'], axis=1)
df_inference = df_inference.drop(['LogSalePrice'], axis=1)
```

In [117…
```
df_inference
```

Out[117…

	MSSubClass	LotFrontage	LotArea	LotShape	LandContour	Utilities	LandSlope
1460	20	80.0	11622	4	4	4	3
1461	20	81.0	14267	3	4	4	3
1462	60	74.0	13830	3	4	4	3
1463	60	78.0	9978	3	4	4	3
1464	120	43.0	5005	3	3	4	3
...	...	...	...	...	...	...	...
2914	160	21.0	1936	4	4	4	3
2915	160	21.0	1894	4	4	4	3
2916	20	160.0	20000	4	4	4	3
2917	85	62.0	10441	4	4	4	3
2918	60	74.0	9627	4	4	4	2

1459 rows × 195 columns

Tune hyperparameters with optuna

XGBoost

In [118…
```
# Define objective function for Optuna for xgboost
def objective_xgb(trial):
    # Define hyperparameters to optimize
    xgboost_params = {
        'n_estimators': trial.suggest_int('n_estimators', 1000, 8000),
        'learning_rate': trial.suggest_loguniform('learning_rate', 0.005,
        'colsample_bytree': trial.suggest_uniform('colsample_bytree', 0.2
        'subsample': trial.suggest_uniform('subsample', 0.4, 0.8),
        'min_child_weight': trial.suggest_int('min_child_weight', 2, 5),
```

```
    }

    # Initialize models with suggested parameters
    xgb_model = XGBRegressor(**xgboost_params, verbosity=0)

    # Train models
    xgb_model.fit(df_train, log_target)

    # Calculate RMSE
    kf = KFold(n_splits=10)
    xgb_rmse = np.exp(np.sqrt(-cross_val_score(xgb_model, df_train, log_t

    # Return average RMSE
    return np.mean(xgb_rmse)
```

In [119…

```
# # Optimize hyperparameters xgboost
# study_xgb = optuna.create_study(direction='minimize')
# study_xgb.optimize(objective_xgb, n_trials=50)
```

In [120…

```
# best_params_xgb = study_xgb.best_params
# best_params_xgb



# {'n_estimators': 2794,
#  'learning_rate': 0.017363039293560027,
#  'colsample_bytree': 0.39670401505406117,
#  'subsample': 0.5816789348238951,
#  'min_child_weight': 2}
```

In [121…

```
best_params_xgb = {'n_estimators': 2794,
 'learning_rate': 0.017363039293560027,
 'colsample_bytree': 0.39670401505406117,
 'subsample': 0.5816789348238951,
 'min_child_weight': 2}
```

In [122…

```
# study_xgb.best_value

# 1.1253873295166898
```

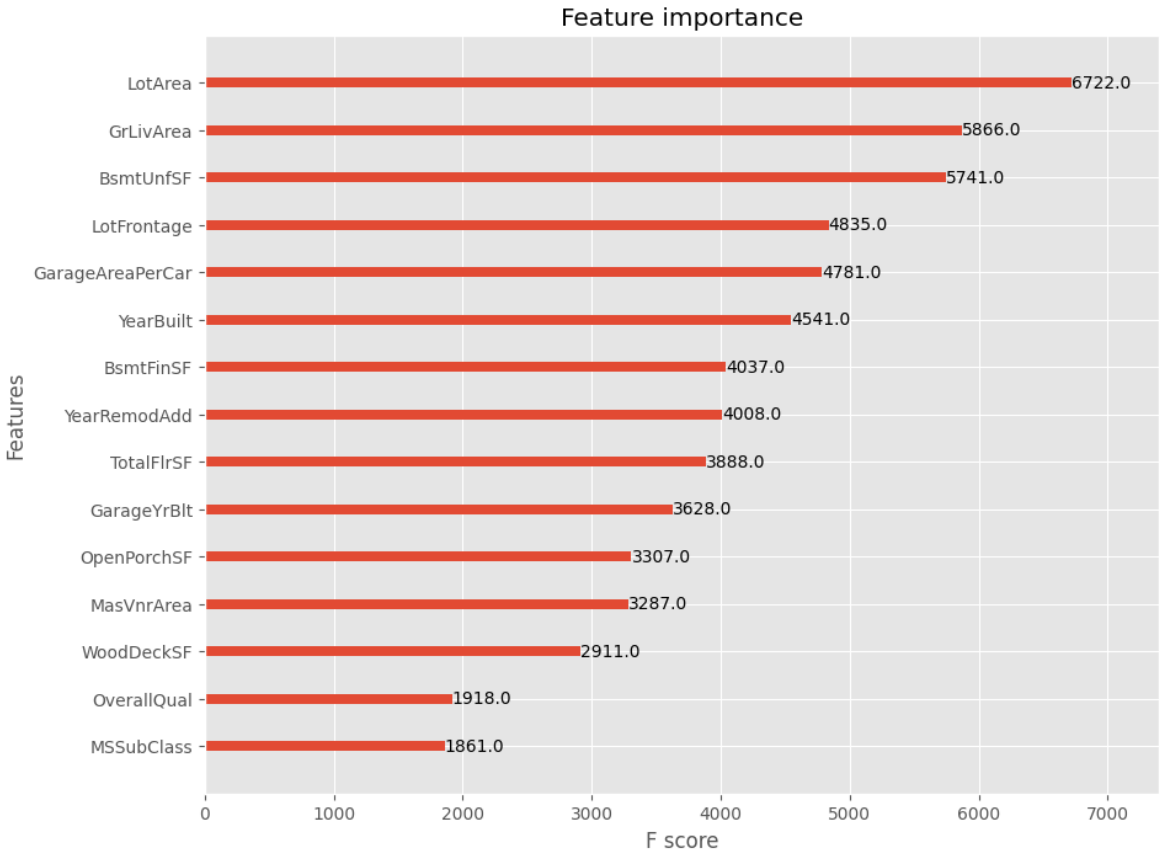
In [123…

```
# Use best parameters to train final models
xgb_model = XGBRegressor(**best_params_xgb, verbosity=0)

xgb_model.fit(df_train, log_target)

kf = KFold(n_splits=10)

xgb_rmse = np.exp(np.sqrt(-cross_val_score(xgb_model, df_train, log_targe
print(np.mean(xgb_rmse))
```

```
1.1253873295166898
```

In [124…

```
# Assuming your model is named xgb_model
fig, ax = plt.subplots(figsize=(10, 8))
xgb.plot_importance(xgb_model, max_num_features=15, ax=ax,importance_type
plt.show()
```



LightGBM

In [125…

```
def objective_lgbm(trial):
    # Define hyperparameters to optimize
    lgbm_params = {
        'n_estimators': trial.suggest_int('n_estimators', 1000, 8000),
        'learning_rate': trial.suggest_float('learning_rate', 0.005, 0.08
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.2,
        'subsample': trial.suggest_float('subsample', 0.4, 0.8),
        # In LightGBM, bagging_freq is required to enable subsampling
        'bagging_freq': trial.suggest_int('bagging_freq', 1, 7),
        'min_child_weight': trial.suggest_int('min_child_weight', 2, 5),
        'num_leaves': trial.suggest_int('num_leaves', 20, 150), # LGBM is
        'verbosity': -1
    }

    # Initialize model
    lgbm_model = LGBMRegressor(**lgbm_params)

    # Calculate RMSE via Cross-Validation
    kf = KFold(n_splits=10, shuffle=True, random_state=42)

    # We evaluate in log-space, then transform back
    # scoring='neg_mean_squared_error' returns negative values, so we use
    cv_scores = cross_val_score(lgbm_model, df_train, log_target, scoring

    # Calculate the exponentiated RMSE (consistent with your log-transfor
    lgbm_rmse = np.exp(np.sqrt(-cv_scores))

    return np.mean(lgbm_rmse)
```

In [ ]:

```
# study_lgbm = optuna.create_study(direction='minimize')
```

```
# study_lgbm.optimize(objective_lgbm, n_trials=50)

# best_params_lgbm = study_lgbm.best_params
# print(f"Best Params: {best_params_lgbm}")
```

```
[I 2026-04-23 17:28:29,452] A new study created in memory with name: no-na
me-7a89b6dd-9ccb-4802-bd2b-686b3ce85c48
[I 2026-04-23 17:29:10,266] Trial 0 finished with value: 1.131121598081910
3 and parameters: {'n_estimators': 7787, 'learning_rate': 0.00826837281497
0894, 'colsample_bytree': 0.4070841204114981, 'subsample': 0.7335068999043
002, 'bagging_freq': 4, 'min_child_weight': 3, 'num_leaves': 57}. Best is
trial 0 with value: 1.1311215980819103.
[I 2026-04-23 17:29:25,162] Trial 1 finished with value: 1.133949644397192
1 and parameters: {'n_estimators': 3007, 'learning_rate': 0.03407848301699
224, 'colsample_bytree': 0.41677239740181415, 'subsample': 0.6958686264388
848, 'bagging_freq': 1, 'min_child_weight': 2, 'num_leaves': 144}. Best is
trial 0 with value: 1.1311215980819103.
[I 2026-04-23 17:29:30,538] Trial 2 finished with value: 1.133055358272010
6 and parameters: {'n_estimators': 1028, 'learning_rate': 0.01133068888661
9183, 'colsample_bytree': 0.5539492521685674, 'subsample': 0.7287243967478
937, 'bagging_freq': 7, 'min_child_weight': 5, 'num_leaves': 44}. Best is
trial 0 with value: 1.1311215980819103.
[I 2026-04-23 17:29:45,779] Trial 3 finished with value: 1.138248396640156
4 and parameters: {'n_estimators': 4973, 'learning_rate': 0.01857521513407
895, 'colsample_bytree': 0.5941664902570725, 'subsample': 0.40818232781164
56, 'bagging_freq': 7, 'min_child_weight': 5, 'num_leaves': 25}. Best is t
rial 0 with value: 1.1311215980819103.
[I 2026-04-23 17:30:08,769] Trial 4 finished with value: 1.130967325953004
2 and parameters: {'n_estimators': 5645, 'learning_rate': 0.00676085090955
3951, 'colsample_bytree': 0.3131648074981249, 'subsample': 0.5900250754121
547, 'bagging_freq': 3, 'min_child_weight': 3, 'num_leaves': 99}. Best is
trial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:30:20,771] Trial 5 finished with value: 1.132282901914388
7 and parameters: {'n_estimators': 2639, 'learning_rate': 0.00508377827550
6407, 'colsample_bytree': 0.574477395976949, 'subsample': 0.63556697786306
05, 'bagging_freq': 6, 'min_child_weight': 3, 'num_leaves': 68}. Best is t
rial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:30:38,660] Trial 6 finished with value: 1.133868035525323
2 and parameters: {'n_estimators': 3964, 'learning_rate': 0.06081160374785
3916, 'colsample_bytree': 0.30512418705484523, 'subsample': 0.685998366040
6505, 'bagging_freq': 6, 'min_child_weight': 3, 'num_leaves': 48}. Best is
trial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:30:55,295] Trial 7 finished with value: 1.134727507217141
4 and parameters: {'n_estimators': 5177, 'learning_rate': 0.02437436230523
505, 'colsample_bytree': 0.2345843555509673, 'subsample': 0.48655499554956
383, 'bagging_freq': 4, 'min_child_weight': 4, 'num_leaves': 42}. Best is
trial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:31:13,866] Trial 8 finished with value: 1.139426838566891
5 and parameters: {'n_estimators': 4666, 'learning_rate': 0.05824031104121
887, 'colsample_bytree': 0.5214331266737469, 'subsample': 0.46782924730424
32, 'bagging_freq': 1, 'min_child_weight': 3, 'num_leaves': 98}. Best is t
rial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:31:31,614] Trial 9 finished with value: 1.138531329660724
5 and parameters: {'n_estimators': 4520, 'learning_rate': 0.04434163795458
377, 'colsample_bytree': 0.5888117074003822, 'subsample': 0.52072217665674
18, 'bagging_freq': 7, 'min_child_weight': 2, 'num_leaves': 54}. Best is t
rial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:32:03,363] Trial 10 finished with value: 1.13104853438223
32 and parameters: {'n_estimators': 7142, 'learning_rate': 0.0064938667911
83841, 'colsample_bytree': 0.3227660507416948, 'subsample': 0.587292008107
3062, 'bagging_freq': 3, 'min_child_weight': 4, 'num_leaves': 114}. Best i
s trial 4 with value: 1.1309673259530042.
[I 2026-04-23 17:32:31,635] Trial 11 finished with value: 1.13090309343992
38 and parameters: {'n_estimators': 7060, 'learning_rate': 0.0051120263811
867215, 'colsample_bytree': 0.3171831674943699, 'subsample': 0.56507177891
```

```
38966, 'bagging_freq': 3, 'min_child_weight': 4, 'num_leaves': 108}. Best
is trial 11 with value: 1.1309030934399238.
[I 2026-04-23 17:32:57,137] Trial 12 finished with value: 1.13209364057644
24 and parameters: {'n_estimators': 6361, 'learning_rate': 0.0107618892200
57557, 'colsample_bytree': 0.31088609928447714, 'subsample': 0.57200439850
25254, 'bagging_freq': 3, 'min_child_weight': 4, 'num_leaves': 120}. Best
is trial 11 with value: 1.1309030934399238.
[I 2026-04-23 17:33:29,618] Trial 13 finished with value: 1.13083419509025
7 and parameters: {'n_estimators': 6387, 'learning_rate': 0.01405402235465
0024, 'colsample_bytree': 0.2046790888724749, 'subsample': 0.7905809408336
579, 'bagging_freq': 2, 'min_child_weight': 4, 'num_leaves': 86}. Best is
trial 13 with value: 1.130834195090257.
[I 2026-04-23 17:34:01,931] Trial 14 finished with value: 1.13082974616620
4 and parameters: {'n_estimators': 6452, 'learning_rate': 0.01582138209398
2695, 'colsample_bytree': 0.20882210811563165, 'subsample': 0.789735273231
1342, 'bagging_freq': 2, 'min_child_weight': 5, 'num_leaves': 80}. Best is
trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:34:32,413] Trial 15 finished with value: 1.13091453388760
54 and parameters: {'n_estimators': 6086, 'learning_rate': 0.0169217512697
96993, 'colsample_bytree': 0.20275891457507267, 'subsample': 0.79979684746
0663, 'bagging_freq': 2, 'min_child_weight': 5, 'num_leaves': 81}. Best is
trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:35:06,493] Trial 16 finished with value: 1.13100996939766
67 and parameters: {'n_estimators': 6650, 'learning_rate': 0.0137328027486
31714, 'colsample_bytree': 0.264526122095242, 'subsample': 0.7964467398499
687, 'bagging_freq': 2, 'min_child_weight': 5, 'num_leaves': 79}. Best is
trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:35:46,130] Trial 17 finished with value: 1.13155469625400
26 and parameters: {'n_estimators': 7628, 'learning_rate': 0.0271807166385
12577, 'colsample_bytree': 0.2011537330052702, 'subsample': 0.752016372318
0178, 'bagging_freq': 2, 'min_child_weight': 5, 'num_leaves': 134}. Best i
s trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:36:14,073] Trial 18 finished with value: 1.13234651516908
66 and parameters: {'n_estimators': 5735, 'learning_rate': 0.0143891764985
208, 'colsample_bytree': 0.485366292213547, 'subsample': 0.643852010492619
, 'bagging_freq': 1, 'min_child_weight': 4, 'num_leaves': 92}. Best is tri
al 14 with value: 1.130829746166204.
[I 2026-04-23 17:36:32,678] Trial 19 finished with value: 1.13198020816400
5 and parameters: {'n_estimators': 3805, 'learning_rate': 0.02354434122223
9824, 'colsample_bytree': 0.26501953731732175, 'subsample': 0.766192531909
6798, 'bagging_freq': 5, 'min_child_weight': 5, 'num_leaves': 73}. Best is
trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:37:06,663] Trial 20 finished with value: 1.13133907836698
28 and parameters: {'n_estimators': 6837, 'learning_rate': 0.0100160593240
89959, 'colsample_bytree': 0.36586966855068814, 'subsample': 0.68399775557
55663, 'bagging_freq': 2, 'min_child_weight': 4, 'num_leaves': 66}. Best i
s trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:37:33,644] Trial 21 finished with value: 1.13186074067231
11 and parameters: {'n_estimators': 7286, 'learning_rate': 0.0079246965310
23817, 'colsample_bytree': 0.24628316321122454, 'subsample': 0.54538762834
04341, 'bagging_freq': 4, 'min_child_weight': 4, 'num_leaves': 116}. Best
is trial 14 with value: 1.130829746166204.
[I 2026-04-23 17:38:02,611] Trial 22 finished with value: 1.13060398358400
03 and parameters: {'n_estimators': 6314, 'learning_rate': 0.0050660039044
28194, 'colsample_bytree': 0.35052112772733446, 'subsample': 0.65202945621
19592, 'bagging_freq': 3, 'min_child_weight': 4, 'num_leaves': 90}. Best i
s trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:38:28,747] Trial 23 finished with value: 1.13249017014279
05 and parameters: {'n_estimators': 5606, 'learning_rate': 0.0309148428162
34987, 'colsample_bytree': 0.34792170016639634, 'subsample': 0.63636129767
```

```
93298, 'bagging_freq': 2, 'min_child_weight': 4, 'num_leaves': 86}. Best i
s trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:39:07,141] Trial 24 finished with value: 1.13267842941730
27 and parameters: {'n_estimators': 6297, 'learning_rate': 0.0161766035698
37324, 'colsample_bytree': 0.45630330692130777, 'subsample': 0.77006740209
82807, 'bagging_freq': 3, 'min_child_weight': 5, 'num_leaves': 103}. Best
is trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:39:31,828] Trial 25 finished with value: 1.13178472906064
38 and parameters: {'n_estimators': 5244, 'learning_rate': 0.0209393593690
91602, 'colsample_bytree': 0.22990177010892213, 'subsample': 0.71180770818
53488, 'bagging_freq': 1, 'min_child_weight': 4, 'num_leaves': 127}. Best
is trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:40:04,183] Trial 26 finished with value: 1.13130192721040
95 and parameters: {'n_estimators': 6119, 'learning_rate': 0.0129155640414
06024, 'colsample_bytree': 0.2780901948881098, 'subsample': 0.768117191120
3393, 'bagging_freq': 2, 'min_child_weight': 5, 'num_leaves': 88}. Best is
trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:40:40,644] Trial 27 finished with value: 1.13303170098787
59 and parameters: {'n_estimators': 7500, 'learning_rate': 0.0397933200911
9915, 'colsample_bytree': 0.3749444368717164, 'subsample': 0.6722716067369
91, 'bagging_freq': 5, 'min_child_weight': 4, 'num_leaves': 75}. Best is t
rial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:41:17,881] Trial 28 finished with value: 1.13606714073916
36 and parameters: {'n_estimators': 7892, 'learning_rate': 0.0766282713734
2834, 'colsample_bytree': 0.43030090719312064, 'subsample': 0.615770799976
1398, 'bagging_freq': 3, 'min_child_weight': 3, 'num_leaves': 64}. Best is
trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:41:50,832] Trial 29 finished with value: 1.13083947651145
64 and parameters: {'n_estimators': 6662, 'learning_rate': 0.0091108381784
55668, 'colsample_bytree': 0.284585859217248, 'subsample': 0.7286024890168
278, 'bagging_freq': 4, 'min_child_weight': 5, 'num_leaves': 94}. Best is
trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:42:09,810] Trial 30 finished with value: 1.13098273251713
75 and parameters: {'n_estimators': 3961, 'learning_rate': 0.0077768637811
74437, 'colsample_bytree': 0.22256103702917615, 'subsample': 0.73973156841
02067, 'bagging_freq': 2, 'min_child_weight': 2, 'num_leaves': 57}. Best i
s trial 22 with value: 1.1306039835840003.
[I 2026-04-23 17:42:42,700] Trial 31 finished with value: 1.13053894128881
32 and parameters: {'n_estimators': 6720, 'learning_rate': 0.0093030032253
39095, 'colsample_bytree': 0.284028746402162, 'subsample': 0.7272675527034
125, 'bagging_freq': 4, 'min_child_weight': 5, 'num_leaves': 91}. Best is
trial 31 with value: 1.1305389412888132.
[I 2026-04-23 17:43:21,766] Trial 32 finished with value: 1.13063233772779
34 and parameters: {'n_estimators': 7943, 'learning_rate': 0.0061169438862
60082, 'colsample_bytree': 0.2461060483028743, 'subsample': 0.784815852074
1911, 'bagging_freq': 5, 'min_child_weight': 5, 'num_leaves': 85}. Best is
trial 31 with value: 1.1305389412888132.
[I 2026-04-23 17:44:01,528] Trial 33 finished with value: 1.13088493959924
97 and parameters: {'n_estimators': 7847, 'learning_rate': 0.0064447115722
45814, 'colsample_bytree': 0.3490422839640961, 'subsample': 0.716235880336
078, 'bagging_freq': 5, 'min_child_weight': 5, 'num_leaves': 106}. Best is
trial 31 with value: 1.1305389412888132.
[I 2026-04-23 17:44:36,079] Trial 34 finished with value: 1.13052660654090
34 and parameters: {'n_estimators': 7986, 'learning_rate': 0.0058758882933
46555, 'colsample_bytree': 0.2534280318467705, 'subsample': 0.658269581127
221, 'bagging_freq': 5, 'min_child_weight': 5, 'num_leaves': 78}. Best is
trial 34 with value: 1.1305266065409034.
[I 2026-04-23 17:45:14,537] Trial 35 finished with value: 1.13129560846353
4 and parameters: {'n_estimators': 7968, 'learning_rate': 0.00592515187865
4387, 'colsample_bytree': 0.3892236460127569, 'subsample': 0.6646030990334
```

```
141, 'bagging_freq': 5, 'min_child_weight': 5, 'num_leaves': 94}. Best is
trial 34 with value: 1.1305266065409034.
[I 2026-04-23 17:45:49,111] Trial 36 finished with value: 1.13034185331331
43 and parameters: {'n_estimators': 7150, 'learning_rate': 0.0074738726304
694, 'colsample_bytree': 0.2903164459429457, 'subsample': 0.70405646609982
44, 'bagging_freq': 6, 'min_child_weight': 5, 'num_leaves': 146}. Best is
trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:45:54,449] Trial 37 finished with value: 1.13182168197956
52 and parameters: {'n_estimators': 1430, 'learning_rate': 0.0075749113013
60189, 'colsample_bytree': 0.29122656717125833, 'subsample': 0.61418566331
63967, 'bagging_freq': 6, 'min_child_weight': 5, 'num_leaves': 25}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:46:28,196] Trial 38 finished with value: 1.13123306738485
9 and parameters: {'n_estimators': 7344, 'learning_rate': 0.00906479487228
7136, 'colsample_bytree': 0.33009768573358844, 'subsample': 0.653652922074
2331, 'bagging_freq': 6, 'min_child_weight': 5, 'num_leaves': 149}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:47:02,993] Trial 39 finished with value: 1.13095975487009
36 and parameters: {'n_estimators': 6927, 'learning_rate': 0.0051722570635
36328, 'colsample_bytree': 0.3475699341927936, 'subsample': 0.707118926347
0156, 'bagging_freq': 4, 'min_child_weight': 5, 'num_leaves': 139}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:47:40,484] Trial 40 finished with value: 1.13164961225123
63 and parameters: {'n_estimators': 7448, 'learning_rate': 0.0071116042470
31617, 'colsample_bytree': 0.41950248185428873, 'subsample': 0.69869808125
86076, 'bagging_freq': 6, 'min_child_weight': 5, 'num_leaves': 35}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:48:18,147] Trial 41 finished with value: 1.13078608732726
55 and parameters: {'n_estimators': 7614, 'learning_rate': 0.0058272834754
57766, 'colsample_bytree': 0.25526154035166093, 'subsample': 0.74499458176
38383, 'bagging_freq': 5, 'min_child_weight': 5, 'num_leaves': 71}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:48:50,516] Trial 42 finished with value: 1.13055082977937
4 and parameters: {'n_estimators': 7080, 'learning_rate': 0.00586357802989
7811, 'colsample_bytree': 0.29483296381708435, 'subsample': 0.680979228851
0252, 'bagging_freq': 7, 'min_child_weight': 5, 'num_leaves': 58}. Best is
trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:49:17,310] Trial 43 finished with value: 1.13069439015091
57 and parameters: {'n_estimators': 5813, 'learning_rate': 0.0089518773428
09035, 'colsample_bytree': 0.29279115378349396, 'subsample': 0.68740124794
5487, 'bagging_freq': 7, 'min_child_weight': 5, 'num_leaves': 60}. Best is
trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:49:47,437] Trial 44 finished with value: 1.13152608835979
15 and parameters: {'n_estimators': 6957, 'learning_rate': 0.0054341695227
05774, 'colsample_bytree': 0.33035066600794455, 'subsample': 0.62133993884
10601, 'bagging_freq': 7, 'min_child_weight': 5, 'num_leaves': 43}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:49:58,466] Trial 45 finished with value: 1.13048338941750
53 and parameters: {'n_estimators': 2484, 'learning_rate': 0.0069479529486
67152, 'colsample_bytree': 0.298917724478714, 'subsample': 0.6669563944887
9, 'bagging_freq': 4, 'min_child_weight': 3, 'num_leaves': 52}. Best is tr
ial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:50:09,408] Trial 46 finished with value: 1.13105149969201
87 and parameters: {'n_estimators': 2437, 'learning_rate': 0.0115375455137
9648, 'colsample_bytree': 0.30221821753491945, 'subsample': 0.671261226521
152, 'bagging_freq': 4, 'min_child_weight': 3, 'num_leaves': 35}. Best is
trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:50:24,642] Trial 47 finished with value: 1.13053575633048
54 and parameters: {'n_estimators': 3211, 'learning_rate': 0.0068247628266
50905, 'colsample_bytree': 0.2746711694473665, 'subsample': 0.726713899659
```

```
1411, 'bagging_freq': 6, 'min_child_weight': 3, 'num_leaves': 51}. Best is
trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:50:33,820] Trial 48 finished with value: 1.13324258767776
16 and parameters: {'n_estimators': 3270, 'learning_rate': 0.0071266812173
49906, 'colsample_bytree': 0.2722622145242426, 'subsample': 0.400107865009
66433, 'bagging_freq': 6, 'min_child_weight': 3, 'num_leaves': 50}. Best i
s trial 36 with value: 1.1303418533133143.
[I 2026-04-23 17:50:42,669] Trial 49 finished with value: 1.13108350084583
46 and parameters: {'n_estimators': 1945, 'learning_rate': 0.0084531936415
02888, 'colsample_bytree': 0.24032451530728866, 'subsample': 0.72216448030
03975, 'bagging_freq': 6, 'min_child_weight': 3, 'num_leaves': 37}. Best i
s trial 36 with value: 1.1303418533133143.
Best Params: {'n_estimators': 7150, 'learning_rate': 0.0074738726304694, '
colsample_bytree': 0.2903164459429457, 'subsample': 0.7040564660998244, 'b
agging_freq': 6, 'min_child_weight': 5, 'num_leaves': 146}
Final LightGBM Mean RMSE: 1.1303418533133143
```

In [126…

```
best_params_lgbm = {'n_estimators': 7150, 'learning_rate': 0.007473872630
# Final LightGBM Mean RMSE: 1.1303418533133143
```

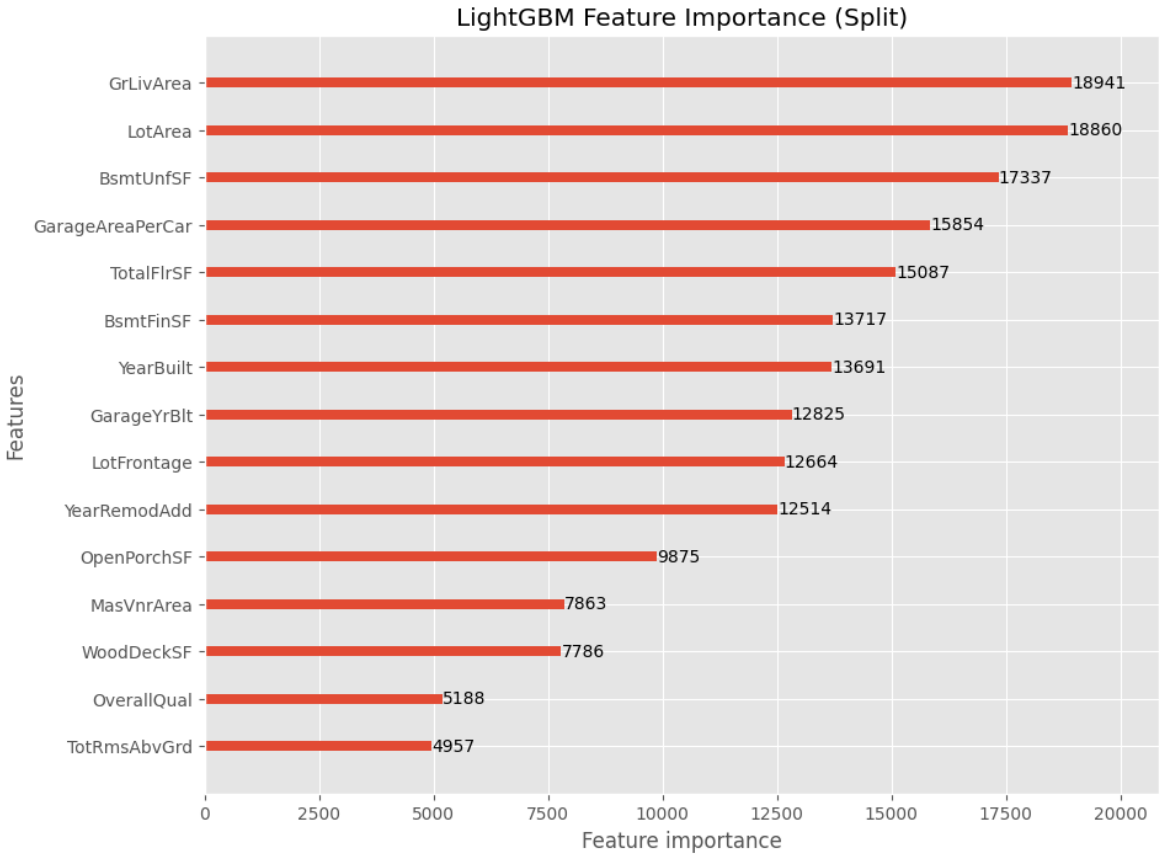
In [127…

```
# 3. Initialize the model with best parameters
# Note: verbosity=-1 is the LGBM equivalent to verbosity=0 in XGB
lgbm_model = LGBMRegressor(**best_params_lgbm, verbosity=-1)

# 4. Fit the final model
lgbm_model.fit(df_train, log_target)

# 5. Cross-validation to get the final reliable RMSE
kf = KFold(n_splits=10, shuffle=True, random_state=42)

lgbm_cv_scores = cross_val_score(lgbm_model, df_train, log_target, scorin
lgbm_rmse = np.exp(np.sqrt(-lgbm_cv_scores))

print(f"Final LightGBM Mean RMSE: {np.mean(lgbm_rmse)}")
```

```
Final LightGBM Mean RMSE: 1.1303418533133143
```

In [128…

```
# Assuming your model is named lgbm_model
fig, ax = plt.subplots(figsize=(10, 8))

# max_num_features works exactly the same way
lgb.plot_importance(lgbm_model, max_num_features=15, ax=ax, importance_ty

plt.title("LightGBM Feature Importance (Split)")
plt.show()
```



In [ ]:

OLS Linear regression

Lasso/Ridge needs feature scaling

In [129…

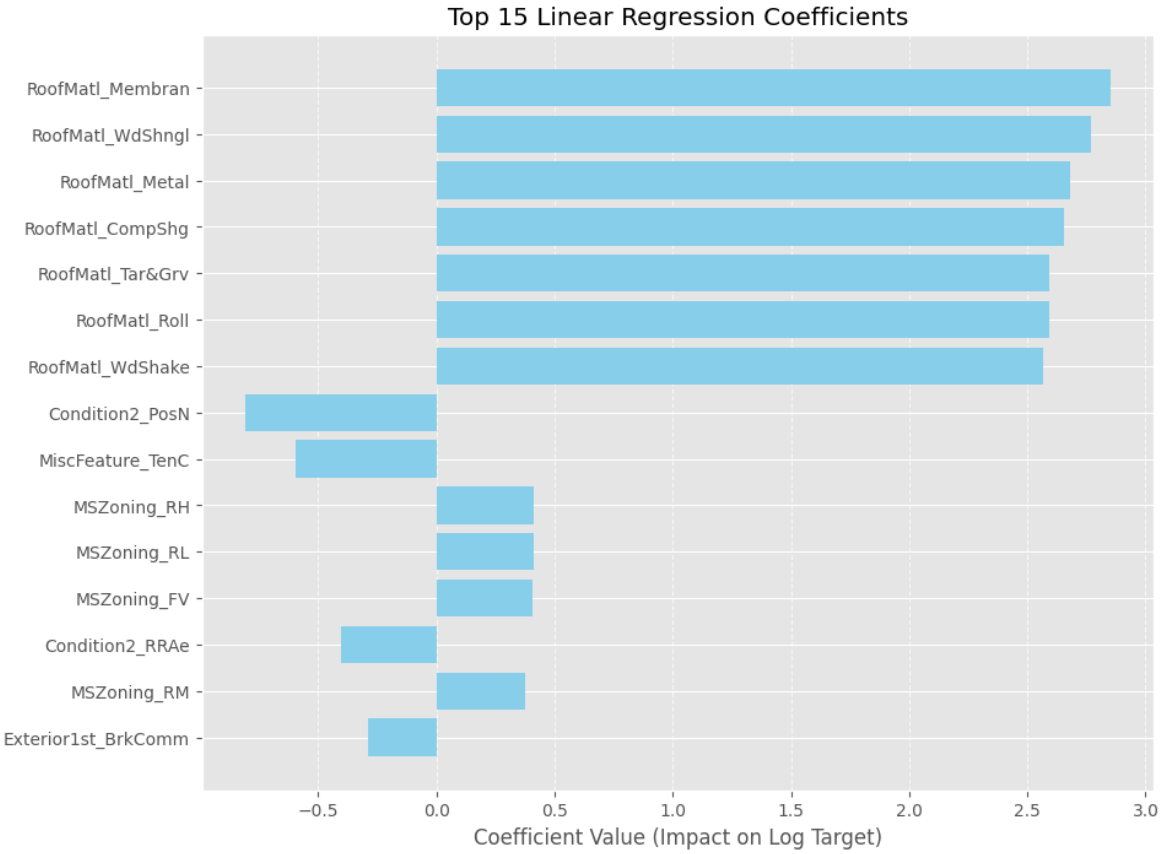
```
# 1. Instantiate the model
# (Linear Regression is usually 'plug-and-play' with no hyperparameters)
lr_model = LinearRegression()

# 2. Fit the model on the log-transformed target
lr_model.fit(df_train, log_target)

# 3. Setup the K-Fold (using 10 splits as in your XGB example)
# Note: It's usually a good idea to add shuffle=True
kf = KFold(n_splits=10, shuffle=True, random_state=42)

# 4. Calculate CV scores
# We calculate RMSE in the log-space, then apply np.exp as per your strat
lr_neg_mse = cross_val_score(lr_model, df_train, log_target, scoring='neg
lr_rmse = np.exp(np.sqrt(-lr_neg_mse))

print(f"Linear Regression Mean RMSE: {np.mean(lr_rmse)}")
```

```
Linear Regression Mean RMSE: 1.2057357524649677
```

In [130…

```
# 1. Extract coefficients and match them to feature names
# We use .coef_ for the weights and .feature_names_in_ if using a DataFra
coef_df = pd.DataFrame({
    'Feature': df_train.columns,
    'Coefficient': lr_model.coef_
})
```

```
# 2. Sort by absolute value to see the most influential features first
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False).head

# 3. Plot
plt.figure(figsize=(10, 8))
plt.barh(coef_df['Feature'], coef_df['Coefficient'], color='skyblue')
plt.xlabel('Coefficient Value (Impact on Log Target)')
plt.title('Top 15 Linear Regression Coefficients')
plt.gca().invert_yaxis() # Highest impact at the top
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()
```



Lasso

In [138…

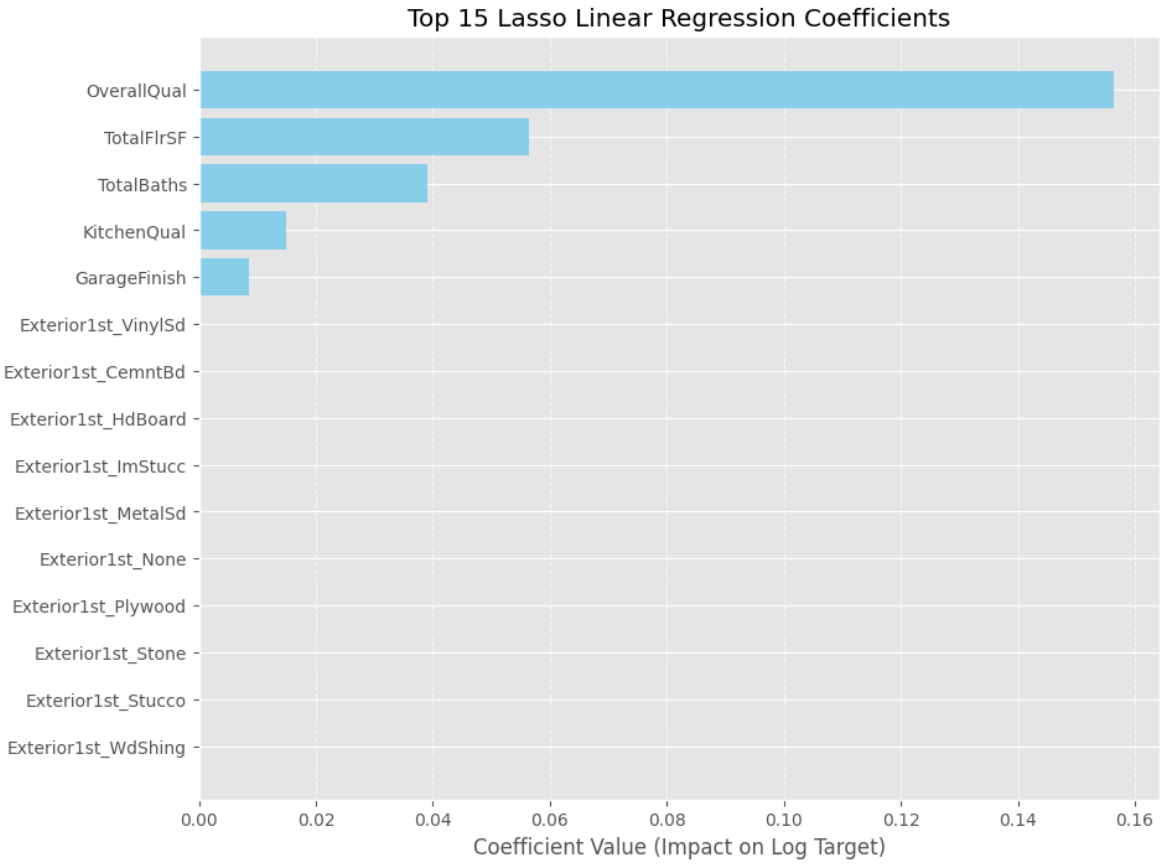
```
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# 1. Create a Pipeline
# This ensures StandardScaler is fit ONLY on the training folds and
# applied to the validation fold, preventing leakage.
lasso_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('lasso', Lasso(alpha=0.1)) # 'alpha' is the regularization strength
])

# 2. Fit the model (on log-transformed target)
lasso_pipe.fit(df_train, log_target)

# 3. Setup K-Fold
kf = KFold(n_splits=10, shuffle=True, random_state=42)
```

```
# 4. Calculate CV scores
# We use neg_mean_squared_error because cross_val_score maximizes a score
lasso_neg_mse = cross_val_score(lasso_pipe, df_train, log_target,
                                scoring='neg_mean_squared_error', cv=kf)

# Convert MSE to RMSE and then reverse the log-transform
lasso_rmse = np.exp(np.sqrt(-lasso_neg_mse))

print(f"Lasso Regression Mean RMSE: {np.mean(lasso_rmse):.4f}")
```

```
Lasso Regression Mean RMSE: 1.2469
```

In [141…

```
# 1. Extract coefficients and match them to feature names
# We use .coef_ for the weights and .feature_names_in_ if using a DataFra


coef_df = pd.DataFrame({
    'Feature': df_train.columns,
    'Coefficient': lasso_pipe.named_steps['lasso'].coef_
})

# 2. Sort by absolute value to see the most influential features first
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False).head

# 3. Plot
plt.figure(figsize=(10, 8))
plt.barh(coef_df['Feature'], coef_df['Coefficient'], color='skyblue')
plt.xlabel('Coefficient Value (Impact on Log Target)')
plt.title('Top 15 Lasso Linear Regression Coefficients')
plt.gca().invert_yaxis() # Highest impact at the top
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()
```



Ridge

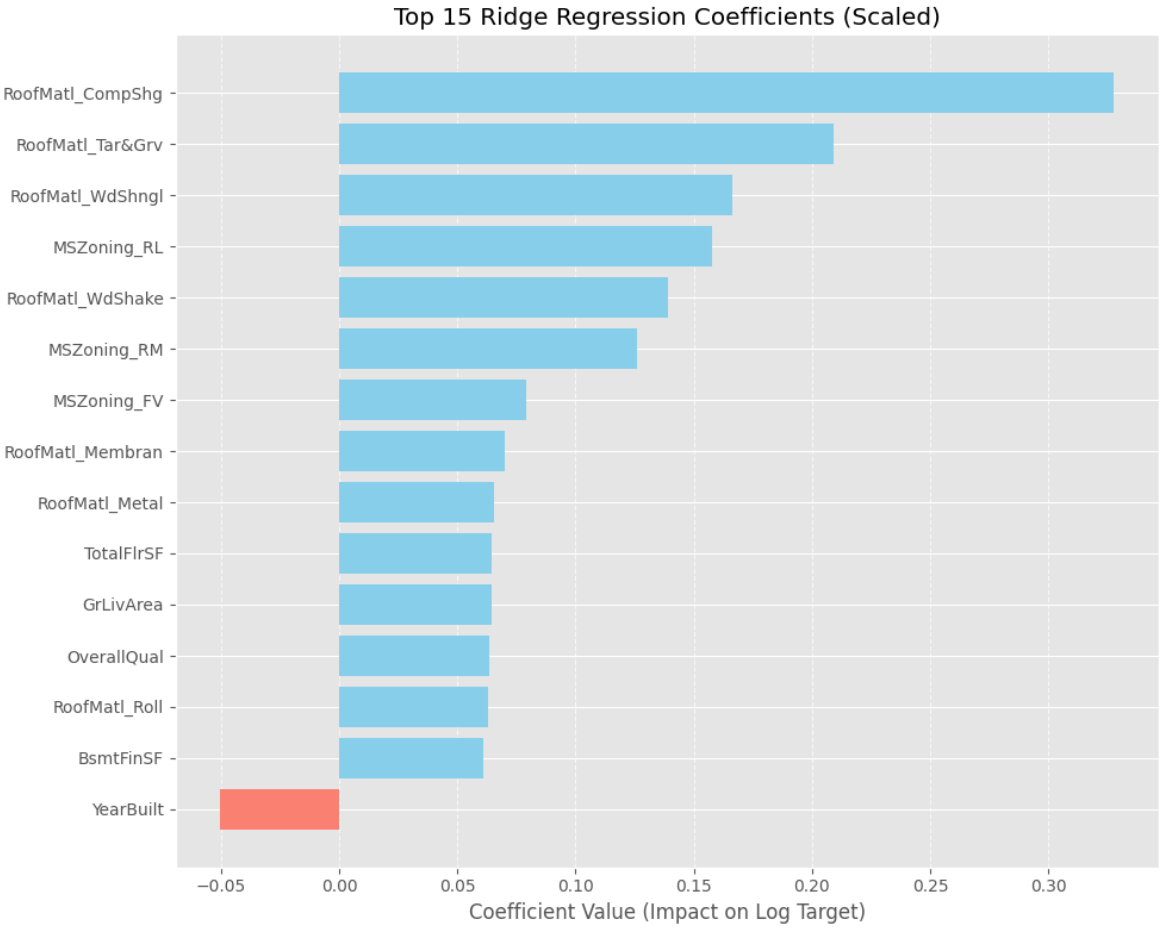
In [145…

```
from sklearn.linear_model import Ridge

# 1. Create a Pipeline
# Ridge adds a penalty proportional to the square of the magnitude of coe
ridge_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge(alpha=1.0)) # alpha is the regularization strength
])

# 2. Fit the model on log-transformed target
ridge_pipe.fit(df_train, log_target)

# 3. Setup K-Fold
kf = KFold(n_splits=10, shuffle=True, random_state=42)

# 4. Calculate CV scores
ridge_neg_mse = cross_val_score(ridge_pipe, df_train, log_target,
                                scoring='neg_mean_squared_error', cv=kf)

# Convert to RMSE and reverse log-transform
ridge_rmse = np.exp(np.sqrt(-ridge_neg_mse))

print(f"Ridge Regression Mean RMSE: {np.mean(ridge_rmse):.4f}")
```

```
Ridge Regression Mean RMSE: 1.1998
```

In [146…

```
# 1. Extract coefficients from the 'ridge' step
ridge_model = ridge_pipe.named_steps['ridge']

coef_df = pd.DataFrame({
    'Feature': df_train.columns,
    'Coefficient': ridge_model.coef_
})

# 2. Sort by absolute value
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False).head

# 3. Plot
plt.figure(figsize=(10, 8))
colors = ['skyblue' if c > 0 else 'salmon' for c in coef_df['Coefficient'

plt.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors)
plt.xlabel('Coefficient Value (Impact on Log Target)')
plt.title('Top 15 Ridge Regression Coefficients (Scaled)')
plt.gca().invert_yaxis()
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Elastic Net

In [147…

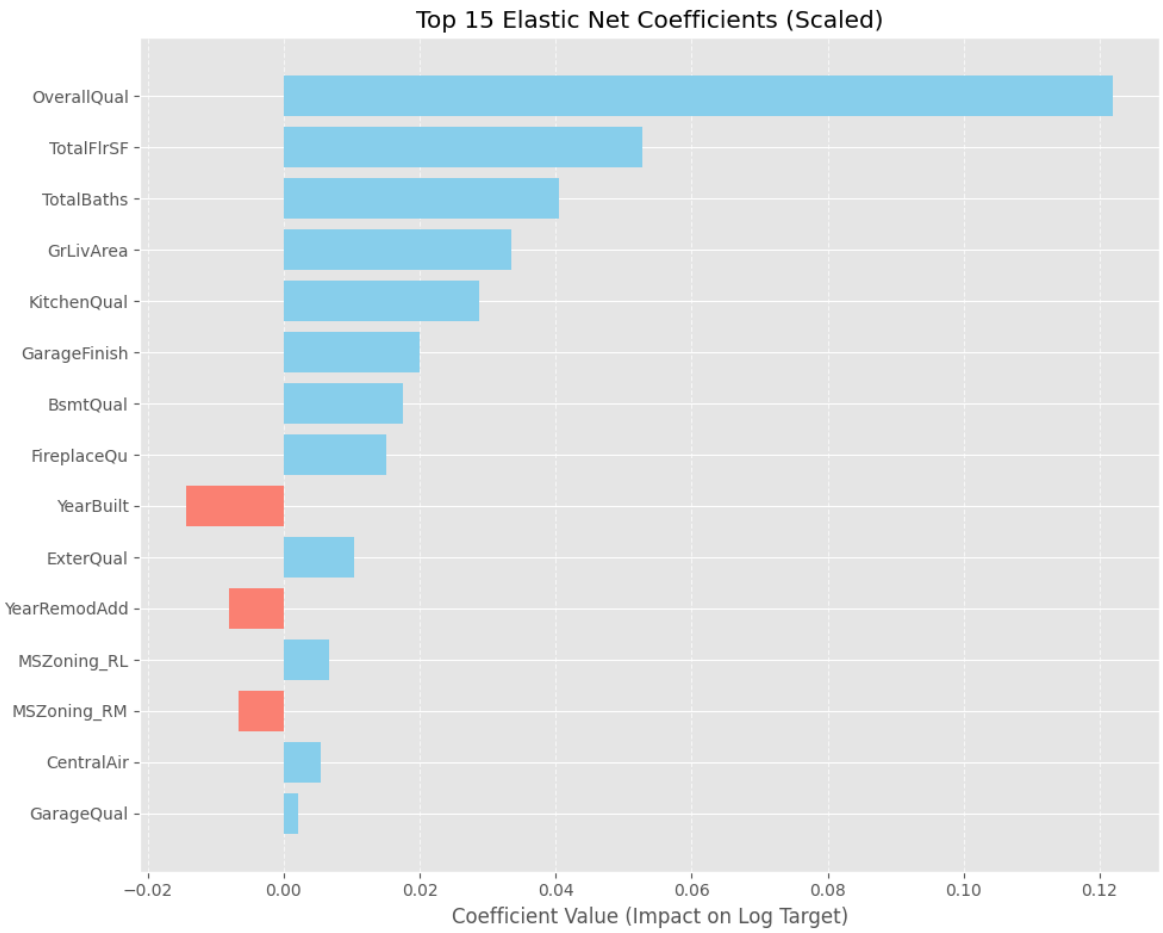
```
from sklearn.linear_model import ElasticNet

# 1. Create a Pipeline
# l1_ratio: 1.0 is pure Lasso, 0.0 is pure Ridge. 0.5 is a 50/50 mix.
en_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('elasticnet', ElasticNet(alpha=0.1, l1_ratio=0.5))
])

# 2. Fit the model
en_pipe.fit(df_train, log_target)

# 3. Setup K-Fold
kf = KFold(n_splits=10, shuffle=True, random_state=42)

# 4. Calculate CV scores
en_neg_mse = cross_val_score(en_pipe, df_train, log_target,
                             scoring='neg_mean_squared_error', cv=kf)

en_rmse = np.exp(np.sqrt(-en_neg_mse))

print(f"Elastic Net Mean RMSE: {np.mean(en_rmse):.4f}")
```

```
Elastic Net Mean RMSE: 1.2005
```

In [148…

```
# 1. Extract coefficients
en_model = en_pipe.named_steps['elasticnet']

coef_df = pd.DataFrame({
```

```
    'Feature': df_train.columns,
    'Coefficient': en_model.coef_
})

# 2. Sort by absolute value
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False).head

# 3. Plot
plt.figure(figsize=(10, 8))
colors = ['skyblue' if c > 0 else 'salmon' for c in coef_df['Coefficient'

plt.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors)
plt.xlabel('Coefficient Value (Impact on Log Target)')
plt.title('Top 15 Elastic Net Coefficients (Scaled)')
plt.gca().invert_yaxis()
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Holdout predictions

If I had split the data into validate and holdout, so here this is not super valid because the models have been trained with the data in the test as well

In [136…

```
# This is not important, I am just doing it to get the data frames alread

df_train2 = df.iloc[:df_train.shape[0]]
df_inference2 = df.iloc[df_train.shape[0]:]
```

In [143…

```
# Getting the validate/test split
# Here this is not proper because I am doing it after all the data proces
columns = [c for c in df_train2.columns if c not in ['LogSalePrice']]

X = df_train2[columns]
y = df_train2['LogSalePrice']

X_validate, X_test, y_validate, y_test = train_test_split(X, y, test_size

X_validate
```

Out[143…

	MSSubClass	LotFrontage	LotArea	LotShape	LandContour	Utilities	LandSlope
254	20	70.0	8400	4	4	4	3
1066	60	59.0	7837	3	4	4	3
638	30	67.0	8777	4	4	4	3
799	50	60.0	7200	4	4	4	3
380	50	50.0	5000	4	4	4	3
...	...	...	...	...	...	...	...
1095	20	78.0	9317	3	4	4	3
1130	50	65.0	7804	4	4	4	3
1294	20	60.0	8172	4	4	4	3
860	50	55.0	7642	4	4	4	3
1126	120	53.0	3684	4	4	4	3

1168 rows × 195 columns

In [149…

```
# 1. Run predictions on X_test
lr_preds = lr_model.predict(X_test)
lasso_pred = lasso_pipe.predict(X_test)
ridge_pred = ridge_pipe.predict(X_test)
en_pred = en_pipe.predict(X_test)
lgbm_preds = lgbm_model.predict(X_test)
xgb_preds = xgb_model.predict(X_test)

from sklearn.metrics import mean_squared_error
# 2. Calculate RMSE for each model
# Note: In sklearn 1.4+, use squared=False for RMSE
rmse_lr = mean_squared_error(y_test, lr_preds, squared=False)
rmse_lasso = mean_squared_error(y_test, lasso_pred, squared=False)
rmse_ridge = mean_squared_error(y_test, ridge_pred, squared=False)
rmse_en = mean_squared_error(y_test, en_pred, squared=False)
rmse_lgbm = mean_squared_error(y_test, lgbm_preds, squared=False)
rmse_xgb = mean_squared_error(y_test, xgb_preds, squared=False)

# 3. Print the results
```

```
print(f"Linear Regression RMSE: {np.exp(rmse_lr):.4f}")
print(f"Lasso Regression RMSE:  {np.exp(rmse_lasso):.4f}")
print(f"Ridge Regression RMSE:  {np.exp(rmse_ridge):.4f}")
print(f"Elastic Net RMSE:       {np.exp(rmse_en):.4f}")
print(f"LightGBM RMSE:          {np.exp(rmse_lgbm):.4f}")
print(f"XGBoost RMSE:           {np.exp(rmse_xgb):.4f}")
```

```
Linear Regression RMSE: 1.1043
Lasso Regression RMSE:  1.2614
Ridge Regression RMSE:  1.1047
Elastic Net RMSE:       1.2079
LightGBM RMSE:          1.0106
XGBoost RMSE:           1.0055
```

Final Prediction

In [137…

```
final_predictions = 0.6* np.exp(xgb_model.predict(df_inference)) + 0.4* n


output = pd.DataFrame({'Id': df_inference.index+1, 'SalePrice': final_pre
output.to_csv('submission2.csv', index=False)
```

In [ ]: